

Stage 03 — Threshold calibration

S03_threshold_calibration.ipynb

Configuración del Entorno

Importación de librerías

```
In [2]: import sys
import re
#Instalación de librería pandas_market_calendars
#{sys.executable} -m pip install -q pandas_market_calendars
#print("Librería instalada: pandas_market_calendars")

from functools import reduce
# Utilidades generales
from datetime import datetime, timedelta
import os
import glob
import requests
import warnings
warnings.filterwarnings('ignore')

# Manejo y procesamiento de datos
import ta
import pandas as pd
import numpy as np
from tabulate import tabulate
import matplotlib.pyplot as plt
# Calendario de mercados
import pandas_market_calendars as mcal

#from ta.momentum import StochasticOscillator, ROCIndicator
#from ta.volatility import BollingerBands, AverageTrueRange

#from scipy.stats import spearmanr
from __future__ import annotations # permite type hints modernos (Python < 3.11)
import json # para guardar el summary como JSON
import os # para leer variables de entorno
from pathlib import Path # manejo robusto de rutas
import pandas as pd
```

Rutas de archivos

```
In [3]: from pathlib import Path

# Buscar la raíz del proyecto
PROJECT_ROOT = Path.cwd()

while PROJECT_ROOT.name != "neural_profit_local":
    PROJECT_ROOT = PROJECT_ROOT.parent
```

```
print("Project root:", PROJECT_ROOT)
```

Project root: c:\Users\heguu\OneDrive\Escritorio\neural_profit_local

Carga de dataset mnq_intraday.parquet

```
In [4]: MNQ_INTRADAY_PATH = PROJECT_ROOT / "data" / "02_mnq_intraday"
MNQ_INTRADAY_PATH.mkdir(parents=True, exist_ok=True)
MNQ_INTRADAY_PARQUET = MNQ_INTRADAY_PATH / "mnq_intraday.parquet"
```

```
In [5]: def load_mnq_parquet(dataset_path):
os.path.exists(dataset_path)
print("Archivo encontrado en disco. Cargando dataset local...")
mnq_parquet = pd.read_parquet(dataset_path)
return mnq_parquet

mnq_intraday = load_mnq_parquet(MNQ_INTRADAY_PARQUET)
```

Archivo encontrado en disco. Cargando dataset local...

Función para ver información de dataset

```
In [6]: from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Dict, Optional, Tuple

import pandas as pd

def mnq_dataset_info(
    df: pd.DataFrame,
    *,
    name: str = "mnq_raw",
    tz_assume_if_naive: Optional[str] = None, # ej: "UTC" o "America/New_York"
    day_def: str = "calendar", # "calendar" (fecha calendario) o "trading" (día)
) -> Dict[str, Any]:
    """
    Resume un dataset OHLCV con DatetimeIndex (ideal para mnq_raw).

    - Si el índice es tz-naive:
        - Si tz_assume_if_naive != None, lo localiza a esa tz.
        - Si no, reporta "tz-naive" (no se puede afirmar horario UTC).
    - Devuelve dict con métricas principales (y lo imprime bonito si se desea).
    """
    if not isinstance(df.index, pd.DatetimeIndex):
        raise TypeError(f"{name}: se requiere DatetimeIndex, recibido: {type(df.index)}")

    idx = df.index

    # --- timezone / UTC info ---
    tzinfo = idx.tz
    if tzinfo is None:
        tz_status = "tz-naive (sin zona horaria)"
        if tz_assume_if_naive:
            idx = idx.tz_localize(tz_assume_if_naive)
            tzinfo = idx.tz
```

```

        tz_status = f"localizado como {tzinfo}"
    else:
        tz_status = f"{tzinfo}"

    # --- rango temporal ---
    ts_min = idx.min()
    ts_max = idx.max()

    first_day = ts_min.date()
    last_day = ts_max.date()

    # --- días ---
    if day_def == "calendar":
        total_days = (pd.Timestamp(last_day) - pd.Timestamp(first_day)).days + 1
    elif day_def == "trading":
        total_days = idx.normalize().nunique()
    else:
        raise ValueError("day_def debe ser 'calendar' o 'trading'")

    # --- columnas ---
    columns = list(df.columns)

    # --- checks útiles ---
    n_rows = len(df)
    n_cols = df.shape[1]
    n_missing = int(df.isna().sum().sum())
    missing_by_col = df.isna().sum().to_dict()
    dup_index = int(idx.duplicated().sum())
    is_monotonic = bool(idx.is_monotonic_increasing)

    # Frecuencia estimada (puede fallar si hay huecos grandes)
    freq = pd.infer_freq(idx[: min(50000, len(idx))]) # muestra grande pero aco

    # Cobertura por día (min/max de hora del día, en tz del índice)
    # (Útil para ver si es 24/7 o horario de sesión)
    tod = pd.Series(idx.time)
    # Convertimos time a minutos del día para resumen robusto
    tod_minutes = pd.Series([t.hour * 60 + t.minute for t in tod])
    typical_minute_min = int(tod_minutes.min())
    typical_minute_max = int(tod_minutes.max())

    # Rango promedio de filas por día (sólo días con datos)
    rows_per_day = df.groupby(idx.normalize()).size()
    rows_per_day_stats = {
        "days_with_data": int(rows_per_day.shape[0]),
        "rows_per_day_min": int(rows_per_day.min()),
        "rows_per_day_p50": float(rows_per_day.median()),
        "rows_per_day_max": int(rows_per_day.max()),
    }

    # Si hay tz, también mostramos rango en UTC
    if tzinfo is not None:
        ts_min_utc = ts_min.tz_convert("UTC")
        ts_max_utc = ts_max.tz_convert("UTC")
        utc_range = (str(ts_min_utc), str(ts_max_utc))
        utc_note = "El índice está tz-aware; el horario UTC es inequívoco."
    else:
        utc_range = None
        utc_note = "El índice es tz-naive; no se puede asegurar si está en UTC s

```

```

info: Dict[str, Any] = {
    "name": name,
    "shape": (n_rows, n_cols),
    "columns": columns,
    "index_type": type(df.index).__name__,
    "index_tz": tz_status,
    "utc_note": utc_note,
    "datetime_min": str(ts_min),
    "datetime_max": str(ts_max),
    "first_day": str(first_day),
    "last_day": str(last_day),
    "total_days": int(total_days),
    "day_definition": day_def,
    "utc_range_if_applicable": utc_range,
    "is_index_monotonic_increasing": is_monotonic,
    "duplicated_timestamps_in_index": dup_index,
    "inferred_freq_sample": freq,
    "missing_total_cells": n_missing,
    "missing_by_col": missing_by_col,
    "rows_per_day_stats": rows_per_day_stats,
    "time_of_day_minutes_range": {
        "min_minute_of_day": typical_minute_min,
        "max_minute_of_day": typical_minute_max,
    },
}
return info

def print_mnq_dataset_info(info: Dict[str, Any]) -> None:
    """Imprime el dict de mnq_dataset_info de forma ordenada."""
    print(f"Dataset: {info['name']}")
    print(f"Shape: {info['shape']}")
    print(f"Columns: {info['columns']}")
    print(f"Index: {info['index_type']} | TZ: {info['index_tz']}")
    print(f"Datetime min/max: {info['datetime_min']} -> {info['datetime_max']}")
    print(f"First/Last day: {info['first_day']} -> {info['last_day']}")
    print(f"Total days ({info['day_definition']}): {info['total_days']}")
    #print(f"Inferred freq (sample): {info['inferred_freq_sample']}")
    #print(f"Index monotonic increasing: {info['is_index_monotonic_increasing']}")
    #print(f"Duplicated timestamps in index: {info['duplicated_timestamps_in_index']}")
    #print(f"Missing total cells: {info['missing_total_cells']}")
    #print(f"Missing by col: {info['missing_by_col']}")
    #print(f"Rows/day stats: {info['rows_per_day_stats']}")
    print(f"Time-of-day range (minutes): {info['time_of_day_minutes_range']}")
    print(f"UTC note: {info['utc_note']}")
    if info["utc_range_if_applicable"] is not None:
        print(f"UTC range: {info['utc_range_if_applicable'][0]} -> {info['utc_

```

```

In [7]: info = mnq_dataset_info(mnq_intraday, name="mnq_intraday", tz_assume_if_naive=False)
print_mnq_dataset_info(info)

```

```

Dataset: mnq_intraday
Shape: (1024062, 9)
Columns: ['date', 'minute_of_day', 'regime_id', 'open', 'high', 'low', 'close',
'volume', 'contract']
Index: DatetimeIndex | TZ: America/New_York
Datetime min/max: 2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00
First/Last day: 2020-01-02 -> 2026-04-17
Total days (trading): 1482
Time-of-day range (minutes): {'min_minute_of_day': 270, 'max_minute_of_day': 960}
UTC note: El índice está tz-aware; el horario UTC es inequívoco.
UTC range: 2020-01-02 09:30:00+00:00 -> 2026-04-17 20:00:00+00:00

```

Validación temporal de dataset `mnq_intraday`

Valida lo esencial para series temporales antes de construir targets: orden cronológico, duplicados, consistencia por día, monotonicidad de `minute_of_day`, gaps temporales y saltos sospechosos. Esto es importante porque en trading hay que validar cuidadosamente los timestamps para evitar look-ahead bias, y porque en series temporales el orden secuencial es parte central del problema.

```

In [8]: import pandas as pd
import numpy as np

# =====
# Validación temporal de mnq_intraday
# Ejecutar inmediatamente después de:
# mnq_intraday = Load_mnq_parquet()
# =====

def validate_mnq_intraday(df: pd.DataFrame, verbose: bool = True) -> dict:
    """
    Valida consistencia temporal básica para un dataset intradía.

    Chequeos:
    1) Índice datetime válido
    2) Orden cronológico global
    3) Duplicados de timestamp
    4) Consistencia de columna `date`
    5) Monotonía de `minute_of_day` dentro de cada día
    6) Duplicados de `minute_of_day` dentro de cada día
    7) Saltos temporales negativos o nulos
    8) Gaps intradía distintos de 1 minuto
    """

    result = {
        "ok": True,
        "checks": {},
        "summary": {},
        "artifacts": {}
    }

    df = df.copy()

    # -----
    # 0) Verificaciones básicas de estructura
    # -----
    required_cols = ["date", "minute_of_day", "open", "high", "low", "close", "v

```

```

missing_cols = [c for c in required_cols if c not in df.columns]
if missing_cols:
    raise ValueError(f"Faltan columnas requeridas: {missing_cols}")

if not isinstance(df.index, pd.DatetimeIndex):
    raise TypeError("El índice del DataFrame debe ser un pd.DatetimeIndex")

if df.empty:
    raise ValueError("El DataFrame está vacío")

# -----
# 1) Orden global del índice
# -----
is_monotonic = df.index.is_monotonic_increasing
has_unique_index = df.index.is_unique
duplicated_index = df.index[df.index.duplicated()].unique()

result["checks"]["index_is_monotonic_increasing"] = bool(is_monotonic)
result["checks"]["index_is_unique"] = bool(has_unique_index)
result["summary"]["n_duplicated_timestamps"] = int(len(duplicated_index))
result["artifacts"]["duplicated_timestamps"] = duplicated_index

# Si no está ordenado, mostramos evidencia pero no reordenamos silenciosamen
if not is_monotonic:
    diffs_ns = pd.Series(df.index.view("i8")).diff()
    bad_order_pos = np.where(diffs_ns <= 0)[0]
    result["artifacts"]["bad_global_order_positions"] = bad_order_pos[:20]
    result["ok"] = False

if not has_unique_index:
    result["ok"] = False

# -----
# 2) Consistencia entre index.date y columna `date`
# -----
# Normalizamos ambos a fecha sin hora
index_dates = pd.Index(df.index.tz_localize(None).date if df.index.tz is not
col_dates = pd.to_datetime(df["date"]).dt.date

date_match = (index_dates == col_dates).all()
result["checks"]["date_column_matches_index_date"] = bool(date_match)

if not date_match:
    mismatch_mask = index_dates != col_dates
    mismatches = df.loc[mismatch_mask, ["date", "minute_of_day", "close"]].h
    result["artifacts"]["date_mismatches_head"] = mismatches
    result["summary"]["n_date_mismatches"] = int(mismatch_mask.sum())
    result["ok"] = False
else:
    result["summary"]["n_date_mismatches"] = 0

# -----
# 3) Diferencias temporales globales
# -----
# Trabajamos en segundos
diffs_sec = pd.Series(df.index).diff().dt.total_seconds()

n_non_positive_diffs = int((diffs_sec.iloc[1:] <= 0).sum())
result["checks"]["all_global_time_diffs_positive"] = (n_non_positive_diffs == 0)
result["summary"]["n_non_positive_global_diffs"] = n_non_positive_diffs

```

```

if n_non_positive_diffs > 0:
    bad_diff_rows = df.iloc[np.where((diffs_sec <= 0).fillna(False))[0][:20]]
    result["artifacts"]["non_positive_global_diffs_head"] = bad_diff_rows
    result["ok"] = False

# -----
# 4) Validación por día
# -----

daily_stats = []
bad_minute_order_days = []
duplicate_minute_days = []
intraday_gap_rows = []

grouped = df.groupby("date", sort=False)

for day, g in grouped:
    g = g.copy()

    # 4.1 orden del índice dentro del día
    idx_mono = g.index.is_monotonic_increasing

    # 4.2 minute_of_day creciente dentro del día
    mod_diff = g["minute_of_day"].diff()
    minute_order_ok = bool((mod_diff.iloc[1:] > 0).all())

    # 4.3 duplicados de minute_of_day dentro del día
    dup_mod = g["minute_of_day"].duplicated().sum()
    has_dup_mod = dup_mod > 0

    # 4.4 gaps intradía del índice
    idx_diff_sec = pd.Series(g.index).diff().dt.total_seconds()
    gap_mask = (~idx_diff_sec.isna()) & (idx_diff_sec != 60)

    n_intraday_gaps = int(gap_mask.sum())

    if not minute_order_ok:
        bad_minute_order_days.append(day)

    if has_dup_mod:
        duplicate_minute_days.append(day)

    if n_intraday_gaps > 0:
        gap_info = g.loc[gap_mask, ["date", "minute_of_day", "open", "high",
                                     "gap_seconds"]] = idx_diff_sec[gap_mask].values
        intraday_gap_rows.append(gap_info)

    daily_stats.append({
        "date": day,
        "n_rows": len(g),
        "index_monotonic": bool(idx_mono),
        "minute_of_day_monotonic": minute_order_ok,
        "n_duplicate_minute_of_day": int(dup_mod),
        "n_intraday_gaps_not_60s": n_intraday_gaps,
        "minute_min": int(g["minute_of_day"].min()),
        "minute_max": int(g["minute_of_day"].max()),
    })

    if not idx_mono or not minute_order_ok or has_dup_mod:
        result["ok"] = False

```

```

daily_stats_df = pd.DataFrame(daily_stats)

result["artifacts"]["daily_stats"] = daily_stats_df
result["summary"]["n_days"] = int(daily_stats_df.shape[0])
result["summary"]["days_with_bad_minute_order"] = int(len(bad_minute_order_d
result["summary"]["days_with_duplicate_minute_of_day"] = int(len(duplicate_m
result["summary"]["days_with_intraday_gaps_not_60s"] = int((daily_stats_df["

result["checks"]["all_days_have_monotonic_minute_of_day"] = (len(bad_minute_
result["checks"]["no_duplicate_minute_of_day_within_day"] = (len(duplicate_m
result["checks"]["all_intraday_steps_are_60s_within_day"] = bool(
    (daily_stats_df["n_intraday_gaps_not_60s"] == 0).all()
)

result["artifacts"]["bad_minute_order_days"] = bad_minute_order_days
result["artifacts"]["duplicate_minute_days"] = duplicate_minute_days

if intraday_gap_rows:
    result["artifacts"]["intraday_gaps_head"] = pd.concat(intraday_gap_rows,
else:
    result["artifacts"]["intraday_gaps_head"] = pd.DataFrame()

# -----
# 5) Resumen global
# -----
result["summary"]["n_rows"] = int(len(df))
result["summary"]["start"] = df.index.min()
result["summary"]["end"] = df.index.max()

# -----
# 6) Reporte por pantalla
# -----
if verbose:
    print("=" * 70)
    print("VALIDACIÓN TEMPORAL DE mnq_intraday")
    print("=" * 70)
    print(f"Rows                : {result['summary']['n_rows']}")
    print(f"Days                  : {result['summary']['n_days']}")
    print(f"Start                 : {result['summary']['start']}")
    print(f"End                   : {result['summary']['end']}")
    print("-" * 70)
    print(f"Index monotonic       : {result['checks']['index_is_monotonic']}")
    print(f"Index unique          : {result['checks']['index_is_unique']}")
    print(f>Date == index.date    : {result['checks']['date_column_matche']}")
    print(f"Global diffs > 0      : {result['checks']['all_global_time_di']}")
    print(f"minute_of_day monotonic : {result['checks']['all_days_have_mono']}")
    print(f"No dup minute_of_day  : {result['checks']['no_duplicate_minut']}")
    print(f>Intraday steps = 60s   : {result['checks']['all_intraday_steps']}")
    print("-" * 70)
    print(f"Duplicated timestamps : {result['summary']['n_duplicated_time']}")
    print(f>Date mismatches       : {result['summary']['n_date_mismatches']}")
    print(f"Non-positive global diffs: {result['summary']['n_non_positive_gl']}")
    print(f"Bad minute order days  : {result['summary']['days_with_bad_min']}")
    print(f"Dup minute_of_day days : {result['summary']['days_with_duplica']}")
    print(f"Days with !=60s gaps   : {result['summary']['days_with_intrada']}")
    print("-" * 70)
    print(f"DATASET OK            : {result['ok']}")
    print("=" * 70)

```

```

    if result["summary"]["n_duplicated_timestamps"] > 0:
        print("\nDuplicated timestamps (head):")
        print(pd.Index(result["artifacts"]["duplicated_timestamps"][:10]))

    if result["summary"]["n_date_mismatches"] > 0:
        print("\nDate mismatches (head):")
        print(result["artifacts"]["date_mismatches_head"])

    if result["summary"]["days_with_bad_minute_order"] > 0:
        print("\nDays with bad minute_of_day order (head):")
        print(result["artifacts"]["bad_minute_order_days"][:10])

    if result["summary"]["days_with_duplicate_minute_of_day"] > 0:
        print("\nDays with duplicate minute_of_day (head):")
        print(result["artifacts"]["duplicate_minute_days"][:10])

    if not result["artifacts"]["intraday_gaps_head"].empty:
        print("\nIntraday gaps != 60 seconds (head):")
        print(result["artifacts"]["intraday_gaps_head"])

    return result

# =====
# Ejecución inmediata
# =====
validation = validate_mnq_intraday(mnq_intraday, verbose=True)

# Si quiere abortar automáticamente cuando haya problemas críticos:
critical_checks = [
    "index_is_monotonic_increasing",
    "index_is_unique",
    "date_column_matches_index_date",
    "all_global_time_diffs_positive",
    "all_days_have_monotonic_minute_of_day",
    "no_duplicate_minute_of_day_within_day",
]

failed_critical = [k for k in critical_checks if not validation["checks"].get(k,

if failed_critical:
    raise ValueError(
        "Validación temporal fallida. Checks críticos con error: "
        + ", ".join(failed_critical)
    )

```

```

=====
VALIDACIÓN TEMPORAL DE mnq_intraday
=====
Rows                : 1024062
Days                : 1482
Start               : 2020-01-02 04:30:00-05:00
End                 : 2026-04-17 16:00:00-04:00
-----
Index monotonic     : True
Index unique        : True
Date == index.date  : True
Global diffs > 0    : True
minute_of_day monotonic : True
No dup minute_of_day : True
Intraday steps = 60s : True
-----
Duplicated timestamps : 0
Date mismatches       : 0
Non-positive global diffs: 0
Bad minute order days : 0
Dup minute_of_day days : 0
Days with !=60s gaps  : 0
-----
DATASET OK          : True
=====

```

El dataset `mnq_intraday` se encuentra correctamente estructurado para la construcción de targets temporales.

En particular, se verificó que:

- el índice `datetime` está ordenado cronológicamente en forma ascendente,
- no existen timestamps duplicados,
- la columna `date` coincide con la fecha derivada del índice,
- no hay saltos temporales negativos ni diferencias no positivas,
- dentro de cada jornada, `minute_of_day` es estrictamente creciente,
- no existen duplicados de `minute_of_day` dentro de un mismo día,
- y todos los pasos intradía son consistentes con una frecuencia de 1 minuto.

Por lo tanto, el dataset queda validado como temporalmente consistente y apto para construir targets forward del tipo `delta_h` y `ret_h`, siempre que dichos horizontes se calculen respetando los límites de cada jornada y evitando cruces entre días.

1. Introducción

El objetivo de este stage es estudiar, calcular y seleccionar thresholds operativos para el dataset intradiario de MNQ.

En etapas anteriores se identificó que el mercado no presenta una estructura homogénea durante toda la sesión. Las magnitudes de movimiento cambian de forma importante según el régimen intradiario, por ejemplo entre `Overnight`, `Pre-market`, `Opening`, `Regular` y `Closing`. Por este motivo, antes de construir targets

operativos, es necesario definir thresholds que representen de manera razonable la escala real de movimiento de cada contexto.

Este stage no tiene como objetivo construir todavía los targets finales de trading. En cambio, se concentra exclusivamente en responder una pregunta previa:

¿Qué magnitud mínima de movimiento debe considerarse significativa para cada horizonte y régimen intradiario?

Para responder esta pregunta se analizarán las excursiones futuras del precio dentro de distintos horizontes temporales, principalmente 30 , 60 y 90 minutos. Estas excursiones permiten medir cuánto se movió el precio hacia arriba o hacia abajo después de cada instante de decisión, sin cruzar el límite diario.

El análisis se realizará únicamente sobre el período de desarrollo 2020-2024 . El período 2025-2026 quedará reservado para validaciones posteriores, evitando que la selección de thresholds incorpore información del futuro.

Durante este stage se evaluarán distintas formas de calcular thresholds:

- thresholds globales por horizonte;
- thresholds por régimen intradiario;
- thresholds por régimen y año;
- thresholds por régimen y trimestre;
- thresholds por régimen y contrato;
- thresholds robustos calculados a partir de la estabilidad histórica.

La finalidad no es elegir el threshold más alto ni el más bajo, sino encontrar una definición estadísticamente razonable y operativamente interpretable. Un threshold demasiado bajo puede generar señales excesivamente ruidosas, mientras que un threshold demasiado alto puede eliminar demasiadas oportunidades válidas.

Como resultado final de este stage se espera obtener una tabla de thresholds candidatos, especialmente por horizon , percentile y regime_id , que será utilizada en el siguiente stage para construir los targets operativos.

En resumen, este stage funciona como una etapa intermedia entre el análisis exploratorio del mercado y la construcción formal de targets. Primero se define qué magnitud de movimiento será considerada significativa; recién después se utilizarán esos valores para etiquetar eventos operativos.

2. Marco metodológico para la calibración de thresholds

Antes de construir targets operativos, es necesario definir qué magnitud mínima de movimiento será considerada significativa dentro de cada horizonte temporal.

En una formulación inicial, los thresholds podrían calcularse a partir del desplazamiento neto entre el precio actual y el precio final del horizonte:

$$\text{forward_return_pts} = \text{close}(t+H) - \text{close}(t)$$

Este enfoque mide únicamente la diferencia entre el precio en el instante actual t y el cierre final de la ventana futura $t+H$.

Sin embargo, para este proyecto se utilizará una lógica más adecuada al comportamiento intradiario del precio. En lugar de observar solamente dónde termina el precio al final del horizonte, se analizará la máxima excursión alcanzada dentro de toda la ventana futura.

La ventana futura para cada observación será:

$t+1, t+2, \dots, t+H$

respetando siempre la restricción de no cruzar días.

2.1. Excursiones futuras

Para cada instante t y horizonte H , se calcularán los siguientes valores:

$$\text{close}_t = \text{close}(t)$$

future_close_max = máximo close futuro entre $t+1$ y $t+H$

future_close_min = mínimo close futuro entre $t+1$ y $t+H$

A partir de esos valores se obtienen dos excursiones:

$$\text{up_excursion_pts} = \text{future_close_max} - \text{close}_t$$

y:

$$\text{down_excursion_pts} = \text{close}_t - \text{future_close_min}$$

La primera mide cuánto logró subir el precio, como máximo, dentro del horizonte futuro.

La segunda mide cuánto logró bajar el precio, como máximo, dentro del mismo horizonte.

De esta forma, el análisis ya no depende únicamente del cierre final de la ventana, sino del movimiento máximo que realmente apareció dentro de ella.

2.2. Motivo para no usar solamente $\text{close}(t+H) - \text{close}(t)$

El problema de usar únicamente el precio final del horizonte es que puede ocultar movimientos relevantes ocurridos dentro de la ventana.

Por ejemplo:

$$\text{close}(t) = 100$$

Dentro de los próximos 30 minutos:

máximo close futuro = 120

mínimo close futuro = 98

close(t+30) = 101

Si se mira solo el cierre final:

$\text{close}(t+30) - \text{close}(t) = 101 - 100 = 1$ punto

parecería que no hubo un movimiento relevante.

Pero si se observa la excursión máxima:

$\text{up_excursion_pts} = 120 - 100 = 20$ puntos

se detecta que sí hubo un movimiento alcista importante dentro de la ventana.

Por este motivo, para calibrar thresholds operativos, el enfoque basado en excursiones futuras es más informativo que el enfoque basado únicamente en el cierre final del horizonte.

2.3. Redefinición de thresholds

Dado que los targets operativos posteriores se apoyarán en movimientos relevantes dentro de una ventana futura, los thresholds también deberán calcularse a partir de esas excursiones.

Por lo tanto, en lugar de calcular thresholds sobre:

$\text{close}(t+H) - \text{close}(t)$

se calcularán sobre:

up_excursion_pts

down_excursion_pts

max_excursion_pts

donde:

$\text{max_excursion_pts} = \max(\text{up_excursion_pts}, \text{down_excursion_pts})$

La idea central es:

El threshold debe medir el mismo fenómeno que luego se utilizará para construir los targets.

Si los targets operativos se basan en excursiones máximas, los thresholds también deben salir de la distribución histórica de esas excursiones máximas.

2.4. Thresholds por horizonte

Los thresholds se calcularán inicialmente por horizonte temporal.

Los horizontes principales serán:

30 minutos

60 minutos

90 minutos

Para cada horizonte se analizarán percentiles de las excursiones futuras.

Por ejemplo:

```
threshold_up_h30_p50 = percentile(up_excursion_pts_30m, 50)
threshold_down_h30_p50 = percentile(down_excursion_pts_30m, 50)
threshold_common_h30_p50 = percentile(max_excursion_pts_30m, 50)
```

También se calcularán varios percentiles para evaluar sensibilidad:

p40
p50
p60
p75
p90
p95

Un threshold bajo generará más eventos significativos, pero probablemente incluirá más ruido.

Un threshold alto generará menos eventos, pero exigirá movimientos de mayor magnitud.

2.5. Thresholds simétricos y separados por dirección

Existen dos alternativas principales.

La primera es usar un threshold común para ambas direcciones:

```
threshold_common_h = percentile(max(up_excursion_pts,
down_excursion_pts), p)
```

En este caso, el mismo umbral se usa para movimientos alcistas y bajistas.

La segunda alternativa es usar thresholds separados por dirección:

```
threshold_up_h = percentile(up_excursion_pts, p)
threshold_down_h = percentile(down_excursion_pts, p)
```

Este enfoque permite capturar posibles diferencias entre la magnitud típica de los movimientos alcistas y bajistas.

En este stage se calcularán ambos enfoques:

threshold común
threshold alcista
threshold bajista

La decisión final sobre cuál utilizar quedará documentada al cierre del análisis de thresholds.

2.6. Separación temporal para evitar contaminación

La calibración de thresholds no debe realizarse usando todo el dataset completo.

Para evitar contaminación del período final, la selección de thresholds se realizará únicamente con el período de desarrollo:

Desarrollo de thresholds: 2020-2024

Evaluación posterior: 2025-2026

Esto permite definir los thresholds con información histórica previa y luego evaluar si esos mismos thresholds mantienen un comportamiento razonable en el período más reciente.

El período 2025-2026 no participará en la selección de thresholds.

2.7. Niveles de análisis

El análisis de thresholds se realizará en distintos niveles:

- global
- por año
- por régimen intradiario
- por trimestre
- por contrato
- por régimen + año
- por régimen + trimestre
- por régimen + contrato

La finalidad es responder preguntas como:

- ¿Las excursiones típicas son parecidas entre años?
- ¿El Opening tiene excursiones mayores que Overnight?
- ¿Un mismo threshold global representa correctamente todos los regímenes?
- ¿Los thresholds por régimen son estables año a año?
- ¿Algunos contratos o trimestres concentran movimientos extremos?

Para cada horizonte y contexto se calcularán estadísticas como:

- media
- mediana
- percentiles
- mínimo
- máximo
- ratio top / low
- coeficiente de variación
- cantidad de observaciones válidas

2.8. Criterio metodológico

El threshold no se define al inicio.

Primero se calculan las excursiones futuras, luego se estudian sus distribuciones históricas y recién después se seleccionan thresholds candidatos.

La metodología general será:

1. Calcular future_close_max y future_close_min por horizonte.
2. Calcular up_excursion_pts, down_excursion_pts y max_excursion_pts.
3. Separar el período de desarrollo: 2020-2024.
4. Analizar distribuciones globales por horizonte.
5. Analizar distribuciones por régimen, año, trimestre, contrato y combinaciones.
6. Evaluar estabilidad de percentiles entre contextos.
7. Comparar thresholds globales contra thresholds por régimen.
8. Seleccionar thresholds candidatos.
9. Guardar los thresholds para utilizarlos en el Stage 04.

En síntesis, este stage no construye todavía los targets operativos. Su objetivo es definir, justificar y guardar los thresholds que serán utilizados posteriormente para construirlos.

1. Introducción
2. Marco metodológico para la calibración de thresholds
3. Cálculo de excursiones futuras
4. Preparación del período de desarrollo
5. Cálculo de thresholds globales
6. Cálculo de thresholds por régimen
7. Análisis de estabilidad
8. Selección de thresholds candidatos
9. Guardado de thresholds

3. Cálculo de excursiones futuras

```
In [34]: # 3.1. Configuración general
# =====
#
# Objetivo:
# Definir los horizontes futuros, la separación temporal del estudio
# y las rutas donde se guardarán los resultados intermedios.
#
# En este punto todavía NO se calculan thresholds.
# Solo se prepara el cálculo de excursiones futuras.
# =====

import numpy as np
import pandas as pd
from pathlib import Path
import json

# -----
# Horizontes futuros a investigar
# -----
#
# Estos horizontes se usarán para calcular excursiones futuras
# y posteriormente calibrar thresholds operativos.

THRESHOLD_HORIZONS = [30, 60, 90]

# -----
# Período de desarrollo
```

```

# -----
#
# Este período se usará para estudiar excursiones y definir
# thresholds candidatos.
#
# Importante:
# 2020-2024 será el período de investigación/diseño.

THRESHOLD_DEVELOPMENT_START_YEAR = 2020
THRESHOLD_DEVELOPMENT_END_YEAR = 2024

# -----
# Período reservado como evaluación posterior
# -----
#
# Este período no debe participar en el diseño de thresholds.
# Más adelante servirá para evaluar si los thresholds definidos
# con 2020-2024 mantienen un comportamiento razonable.

THRESHOLD_FINAL_TEST_START_YEAR = 2025
THRESHOLD_FINAL_TEST_END_YEAR = 2026

# -----
# Rutas de guardado
# -----

if "PROJECT_ROOT" in globals():
    MNQ_THRESHOLDS_PATH = PROJECT_ROOT / "data" / "03_mnq_thresholds"
else:
    MNQ_THRESHOLDS_PATH = Path("data") / "03_mnq_thresholds"

MNQ_THRESHOLDS_PATH.mkdir(parents=True, exist_ok=True)

MNQ_THRESHOLD_EXCURSIONS_PARQUET = MNQ_THRESHOLDS_PATH / "mnq_threshold_excursio
MNQ_THRESHOLD_INVALID_SUMMARY_PARQUET = MNQ_THRESHOLDS_PATH / "mnq_threshold_exc
MNQ_THRESHOLD_EXCURSIONS_SUMMARY_JSON = MNQ_THRESHOLDS_PATH / "mnq_threshold_exc

print("Carpeta donde se guardarán las excursiones futuras para calibración de th
print(MNQ_THRESHOLDS_PATH)

```

Carpeta donde se guardarán las excursiones futuras para calibración de thresholds:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds

```

In [35]: # 3.2. Funciones auxiliares
# =====
#
# Objetivo:
# Definir funciones para calcular máximos y mínimos futuros
# dentro de una ventana forward:
#
#     t+1, t+2, ..., t+H
#
# Reglas importantes:
# - La barra actual t queda excluida.
# - La ventana futura no cruza días.
# - Si no hay H barras futuras dentro del mismo día, devuelve NaN.
# =====

def forward_rolling_max_without_crossing_days(

```

```

df,
value_col,
horizon,
date_col="date"
):
    """
    Calcula el máximo futuro de una columna dentro de la ventana:

        t+1, t+2, ..., t+H

    sin cruzar días.
    """

    return (
        df
        .groupby(date_col, observed=True)[value_col]
        .transform(
            lambda s: (
                s
                .shift(-1)                # Excluye la barra act
                .iloc[::-1]                # Invierte la serie
                .rolling(window=horizon, min_periods=horizon)
                .max()
                .iloc[::-1]                # Vuelve al orden orig
            )
        )
    )

def forward_rolling_min_without_crossing_days(
df,
value_col,
horizon,
date_col="date"
):
    """
    Calcula el mínimo futuro de una columna dentro de la ventana:

        t+1, t+2, ..., t+H

    sin cruzar días.
    """

    return (
        df
        .groupby(date_col, observed=True)[value_col]
        .transform(
            lambda s: (
                s
                .shift(-1)                # Excluye la barra act
                .iloc[::-1]                # Invierte la serie
                .rolling(window=horizon, min_periods=horizon)
                .min()
                .iloc[::-1]                # Vuelve al orden orig
            )
        )
    )

def add_future_close_excursions(

```

```

df,
horizons,
price_col="close",
date_col="date"
):
    """
    Agrega las excursiones futuras necesarias para calibrar thresholds.

    Para cada horizonte H calcula:

    - future_close_max_Hm
    - future_close_min_Hm
    - up_excursion_pts_Hm
    - down_excursion_pts_Hm
    - max_excursion_pts_Hm

    Estas columnas miran hacia el futuro.
    Por lo tanto, se usan para investigar thresholds y construir targets
    en etapas posteriores, nunca como features predictivas.
    """

    out = df.copy()

    if not isinstance(out.index, pd.DatetimeIndex):
        raise TypeError("El DataFrame debe tener un DatetimeIndex.")

    if date_col not in out.columns:
        out[date_col] = out.index.date

    out = out.sort_index()

    for h in horizons:

        future_max_col = f"future_close_max_{h}m"
        future_min_col = f"future_close_min_{h}m"
        up_col = f"up_excursion_pts_{h}m"
        down_col = f"down_excursion_pts_{h}m"
        max_col = f"max_excursion_pts_{h}m"

        # Máximo close futuro dentro de t+1 ... t+H
        out[future_max_col] = forward_rolling_max_without_crossing_days(
            out,
            value_col=price_col,
            horizon=h,
            date_col=date_col
        )

        # Mínimo close futuro dentro de t+1 ... t+H
        out[future_min_col] = forward_rolling_min_without_crossing_days(
            out,
            value_col=price_col,
            horizon=h,
            date_col=date_col
        )

        # Excursión alcista máxima dentro del horizonte
        out[up_col] = out[future_max_col] - out[price_col]

        # Excursión bajista máxima dentro del horizonte
        out[down_col] = out[price_col] - out[future_min_col]

```

```

        # Excursión dominante, sin dirección
        out[max_col] = out[[up_col, down_col]].max(axis=1)

    return out

print("Funciones auxiliares cargadas para calcular excursiones futuras sin cruza

```

Funciones auxiliares cargadas para calcular excursiones futuras sin cruzar días.

```

In [36]: # 3.3. Cálculo de excursiones futuras
# =====
#
# Objetivo:
# Calcular, para cada fila y cada horizonte:
#
# - máximo close futuro
# - mínimo close futuro
# - excursión alcista
# - excursión bajista
# - excursión máxima dominante
#
# En este bloque todavía NO se calculan thresholds.
# Tampoco se construyen targets.
# =====

df_threshold_excursions = add_future_close_excursions(
    mnq_intraday,
    horizons=THRESHOLD_HORIZONS,
    price_col="close",
    date_col="date"
)

print("DataFrame creado con las excursiones futuras necesarias para calibrar thr
print("Todavía no contiene thresholds ni targets construidos.")
print(f"Filas: {df_threshold_excursions.shape[0]:,}")
print(f"Columnas: {df_threshold_excursions.shape[1]:,}")

excursion_preview_cols = [
    "date",
    "close",
]

for h in THRESHOLD_HORIZONS:
    excursion_preview_cols.extend([
        f"future_close_max_{h}m",
        f"future_close_min_{h}m",
        f"up_excursion_pts_{h}m",
        f"down_excursion_pts_{h}m",
        f"max_excursion_pts_{h}m",
    ])

print("\nVista previa de las excursiones futuras calculadas para cada horizonte.
print("Cada fila muestra el máximo y mínimo close futuro dentro de la ventana, y

display(df_threshold_excursions[excursion_preview_cols].head(10))

```

DataFrame creado con las excursiones futuras necesarias para calibrar thresholds.
Todavía no contiene thresholds ni targets contruidos.

Filas: 1,024,062

Columnas: 24

Vista previa de las excursiones futuras calculadas para cada horizonte.

Cada fila muestra el máximo y mínimo close futuro dentro de la ventana, y cuánto pudo subir o bajar el precio desde el close actual.

	date	close	future_close_max_30m	future_close_min_30m	up_excursion_pts_
datetime					
2020-01-02 04:30:00-05:00	2020-01-02	8813.25	8816.5	8809.5	
2020-01-02 04:31:00-05:00	2020-01-02	8812.50	8816.5	8809.5	
2020-01-02 04:32:00-05:00	2020-01-02	8811.75	8817.5	8809.5	
2020-01-02 04:33:00-05:00	2020-01-02	8810.50	8817.5	8809.5	
2020-01-02 04:34:00-05:00	2020-01-02	8812.00	8817.5	8809.5	
2020-01-02 04:35:00-05:00	2020-01-02	8813.00	8817.5	8809.5	
2020-01-02 04:36:00-05:00	2020-01-02	8813.25	8817.5	8809.5	
2020-01-02 04:37:00-05:00	2020-01-02	8813.25	8817.5	8809.5	
2020-01-02 04:38:00-05:00	2020-01-02	8814.00	8817.5	8809.5	
2020-01-02 04:39:00-05:00	2020-01-02	8813.50	8817.5	8809.5	

4. Preparación del período de desarrollo

4.1. Separación temporal

```
In [37]: # 4.1. Separación temporal development / final_test
# =====
#
# Objetivo:
# Crear una columna que indique si cada fila pertenece a:
#
# - development_2020_2024
# - final_test_2025_2026
#
# Esta separación será usada más adelante para calibrar thresholds
# únicamente con 2020-2024.
#
# En este bloque todavía NO se calculan thresholds.
# =====

# -----
# Validación del DataFrame base
# -----

if "df_threshold_excursions" not in globals():
    raise NameError(
        "No existe df_threshold_excursions en memoria. "
        "Primero debes ejecutar el punto 3.3."
    )

if df_threshold_excursions.empty:
    raise ValueError(
        "df_threshold_excursions está vacío. "
        "No hay datos disponibles para preparar el período de desarrollo."
    )

# -----
# Crear variables temporales
# -----

df_threshold_excursions = df_threshold_excursions.copy()

if "date" not in df_threshold_excursions.columns:
    if isinstance(df_threshold_excursions.index, pd.DatetimeIndex):
        df_threshold_excursions["date"] = df_threshold_excursions.index.date
    else:
        raise ValueError(
            "No existe la columna date y el índice no es DatetimeIndex."
        )

date_ts = pd.to_datetime(df_threshold_excursions["date"])

df_threshold_excursions["year"] = date_ts.dt.year.astype("int16")
df_threshold_excursions["quarter"] = date_ts.dt.quarter.astype("int8")
df_threshold_excursions["year_quarter"] = date_ts.dt.to_period("Q").astype(str)

# -----
# Separación temporal
# -----

conditions = [
```

```

df_threshold_excursions["year"].between(
    THRESHOLD_DEVELOPMENT_START_YEAR,
    THRESHOLD_DEVELOPMENT_END_YEAR
),
df_threshold_excursions["year"].between(
    THRESHOLD_FINAL_TEST_START_YEAR,
    THRESHOLD_FINAL_TEST_END_YEAR
),
]

choices = [
    "development_2020_2024",
    "final_test_2025_2026",
]

df_threshold_excursions["threshold_calibration_split"] = np.select(
    conditions,
    choices,
    default="outside_scope"
)

# -----
# Crear familia de contrato si existe la columna contract
# -----

if "contract" in df_threshold_excursions.columns:
    df_threshold_excursions["contract_code"] = (
        df_threshold_excursions["contract"]
        .astype(str)
        .str.extract(r"([HMUZ])", expand=False)
    )

# -----
# Resumen de separación temporal
# -----

df_threshold_split_summary = (
    df_threshold_excursions["threshold_calibration_split"]
    .value_counts(dropna=False)
    .rename_axis("threshold_calibration_split")
    .reset_index(name="rows")
)

df_threshold_split_summary["rows_ratio"] = (
    df_threshold_split_summary["rows"]
    / df_threshold_split_summary["rows"].sum()
).round(4)

print("Distribución de filas según la separación temporal del estudio.")
print("development_2020_2024 será usado para estudiar y seleccionar thresholds.")
print("final_test_2025_2026 queda reservado para evaluación posterior.")
print("outside_scope corresponde a filas fuera del rango definido, si existieran")

display(df_threshold_split_summary)

```

Distribución de filas según la separación temporal del estudio.
 development_2020_2024 será usado para estudiar y seleccionar thresholds.
 final_test_2025_2026 queda reservado para evaluación posterior.
 outside_scope corresponde a filas fuera del rango definido, si existieran.

	threshold_calibration_split	rows	rows_ratio
0	development_2020_2024	818835	0.7996
1	final_test_2025_2026	205227	0.2004

4.2. Validación de filas válidas

```
In [38]: # 4.2. Validación básica de filas válidas e inválidas
# =====
#
# Objetivo:
# Verificar cuántas filas tienen excursiones válidas por horizonte.
#
# Las filas inválidas aparecen principalmente al final de cada día,
# porque no existen suficientes barras futuras dentro del mismo trading day.
#
# Esto no es un error.
# Es una consecuencia esperada de calcular excursiones futuras
# sin cruzar días.
# =====

# -----
# Validaciones previas
# -----

if "df_threshold_excursions" not in globals():
    raise NameError(
        "No existe df_threshold_excursions en memoria. "
        "Primero debes ejecutar el punto 3.3."
    )

if "THRESHOLD_HORIZONS" not in globals():
    raise NameError(
        "No existe THRESHOLD_HORIZONS en memoria. "
        "Primero debes ejecutar el punto 3.1."
    )

# -----
# Cálculo de filas válidas e inválidas
# -----

invalid_rows_summary = []

for h in THRESHOLD_HORIZONS:

    future_max_col = f"future_close_max_{h}m"
    future_min_col = f"future_close_min_{h}m"
    up_col = f"up_excursion_pts_{h}m"
    down_col = f"down_excursion_pts_{h}m"
    max_col = f"max_excursion_pts_{h}m"

    required_cols = [
        future_max_col,
        future_min_col,
        up_col,
        down_col,
```

```

        max_col,
    ]

    missing_cols = [
        col for col in required_cols
        if col not in df_threshold_excursions.columns
    ]

    if missing_cols:
        raise ValueError(
            f"Faltan columnas necesarias para H={h}: {missing_cols}"
        )

    invalid_mask = (
        df_threshold_excursions[required_cols]
        .isna()
        .any(axis=1)
    )

    invalid_rows_summary.append({
        "horizon": h,
        "total_rows": int(len(df_threshold_excursions)),
        "valid_rows": int((~invalid_mask).sum()),
        "invalid_rows": int(invalid_mask.sum()),
        "valid_ratio": float((~invalid_mask).mean()),
        "invalid_ratio": float(invalid_mask.mean()),
    })

df_threshold_invalid_summary = pd.DataFrame(invalid_rows_summary)

print("Resumen de filas válidas e inválidas por horizonte.")
print("Las filas inválidas son esperables en las últimas barras de cada día, por")
print("A mayor horizonte, mayor cantidad de filas inválidas al cierre de cada jo")

display(df_threshold_invalid_summary)

```

Resumen de filas válidas e inválidas por horizonte.

Las filas inválidas son esperables en las últimas barras de cada día, porque no tienen suficiente ventana futura.

A mayor horizonte, mayor cantidad de filas inválidas al cierre de cada jornada.

	horizon	total_rows	valid_rows	invalid_rows	valid_ratio	invalid_ratio
0	30	1024062	979602	44460	0.956585	0.043415
1	60	1024062	935142	88920	0.913169	0.086831
2	90	1024062	890682	133380	0.869754	0.130246

4.3. Guardado intermedio

```

In [39]: # 4.3. Guardado de resultados intermedios
# =====
#
# Objetivo:
# Guardar los resultados necesarios para continuar con la calibración
# de thresholds en los siguientes puntos.
#
# Se guarda:

```

```

# - df_threshold_excursions
# - df_threshold_invalid_summary
# - resumen metodológico en JSON
#
# Todavía NO se guardan thresholds porque aún no fueron calculados.
# =====

# -----
# Validaciones previas
# -----

if "df_threshold_excursions" not in globals():
    raise NameError(
        "No existe df_threshold_excursions en memoria."
    )

if "df_threshold_invalid_summary" not in globals():
    raise NameError(
        "No existe df_threshold_invalid_summary en memoria. "
        "Primero debes ejecutar el punto 4.2."
    )

# -----
# Guardar archivos principales
# -----

df_threshold_excursions.to_parquet(
    MNQ_THRESHOLD_EXCURSIONS_PARQUET,
    index=True
)

df_threshold_invalid_summary.to_parquet(
    MNQ_THRESHOLD_INVALID_SUMMARY_PARQUET,
    index=False
)

# -----
# Resumen metodológico
# -----

threshold_excursions_summary = {
    "stage": "S03_threshold_calibration",
    "section": "4_development_period_preparation",
    "description": (
        "Cálculo y preparación de excursiones futuras necesarias "
        "para calibrar thresholds operativos."
    ),
    "horizons": THRESHOLD_HORIZONS,
    "development_period": (
        f"{THRESHOLD_DEVELOPMENT_START_YEAR}-"
        f"{THRESHOLD_DEVELOPMENT_END_YEAR}"
    ),
    "final_test_period": (
        f"{THRESHOLD_FINAL_TEST_START_YEAR}-"
        f"{THRESHOLD_FINAL_TEST_END_YEAR}"
    ),
    "split_column": "threshold_calibration_split",
    "methodological_rule": (
        "Las excursiones se calculan para todo el dataset, "
        "pero los thresholds se calibrarán usando solamente el período "

```

```

        "development_2020_2024. "
        "El período final_test_2025_2026 queda reservado para evaluación posteri
    ),
    "important_note": (
        "Las columnas de excursiones futuras miran hacia adelante. "
        "Por lo tanto, no deben usarse como features predictivas."
    ),
    "saved_files": {
        "threshold_excursions": str(MNQ_THRESHOLD_EXCURSIONS_PARQUET),
        "invalid_summary": str(MNQ_THRESHOLD_INVALID_SUMMARY_PARQUET),
        "methodological_summary": str(MNQ_THRESHOLD_EXCURSIONS_SUMMARY_JSON),
    }
}

with MNQ_THRESHOLD_EXCURSIONS_SUMMARY_JSON.open("w", encoding="utf-8") as f:
    json.dump(
        threshold_excursions_summary,
        f,
        indent=2,
        ensure_ascii=False
    )

# -----
# Confirmación de guardado
# -----

print("Archivos guardados para continuar la calibración de thresholds.")
print("El primer archivo contiene las excursiones futuras calculadas.")
print("El segundo archivo contiene el resumen de filas válidas e inválidas por h
print("El tercer archivo contiene un resumen metodológico del proceso.")

print("\nExcursiones futuras:")
print(MNQ_THRESHOLD_EXCURSIONS_PARQUET)

print("\nResumen de filas inválidas:")
print(MNQ_THRESHOLD_INVALID_SUMMARY_PARQUET)

print("\nResumen metodológico:")
print(MNQ_THRESHOLD_EXCURSIONS_SUMMARY_JSON)

```

Archivos guardados para continuar la calibración de thresholds.

El primer archivo contiene las excursiones futuras calculadas.

El segundo archivo contiene el resumen de filas válidas e inválidas por horizont

El tercer archivo contiene un resumen metodológico del proceso.

Excursiones futuras:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_threshold_excursions.parquet

Resumen de filas inválidas:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_threshold_excursions_invalid_summary.parquet

Resumen metodológico:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_threshold_excursions_summary.json

4.4. Plan de análisis para la calibración de thresholds

Una vez calculadas las excursiones futuras y preparado el período de desarrollo, el siguiente paso consiste en analizar qué magnitudes de movimiento pueden considerarse relevantes para calibrar thresholds operativos.

En este stage, los thresholds se estudiarán a partir de las excursiones máximas observadas dentro de cada ventana futura. Para cada observación y horizonte se utilizarán principalmente las siguientes métricas:

up_excursion_pts_Hm
down_excursion_pts_Hm
max_excursion_pts_Hm
donde:

up_excursion_pts_Hm = future_close_max_Hm - close(t)
down_excursion_pts_Hm = close(t) - future_close_min_Hm
max_excursion_pts_Hm = mayor valor entre la excursión alcista y la bajista

El objetivo no es construir todavía targets operativos, sino estudiar la distribución histórica de estas excursiones para determinar qué valores pueden funcionar como thresholds candidatos.

La calibración de thresholds se realizará usando únicamente el período de desarrollo:

development_2020_2024

El período:

final_test_2025_2026

quedará reservado para evaluar posteriormente si los thresholds ya definidos mantienen un comportamiento razonable en datos más recientes.

La regla metodológica será:

Los thresholds se estudian y seleccionan usando 2020-2024.

El período 2025-2026 no participa en la calibración.

El análisis se realizará en varios niveles. Primero se calcularán thresholds globales por horizonte, con el objetivo de obtener una referencia general de las magnitudes típicas del movimiento futuro.

Luego se estudiarán thresholds por régimen intradiario, dado que el comportamiento del precio no es homogéneo durante toda la sesión. En particular, se compararán los regímenes:

Overnight
Pre-market
Opening
Regular
Closing

Este análisis permitirá evaluar si un único threshold global representa correctamente todos los contextos o si resulta demasiado exigente para regímenes tranquilos y demasiado flexible para regímenes más activos.

Después se analizarán combinaciones adicionales de contexto:

régimen + año
régimen + trimestre
régimen + contrato
régimen + familia de contrato

El objetivo será responder preguntas como:

¿Los thresholds son estables dentro de un mismo régimen?
¿El año 2022 se comporta como un año extremo?
¿El Opening mantiene thresholds mayores que Overnight en todos los años?
¿Un threshold global por horizonte es representativo?
¿Conviene usar thresholds por regime_id como metodología principal?

Para comparar los thresholds entre contextos se utilizarán métricas de estabilidad como:

mediana
mínimo
máximo
ratio top / low
coeficiente de variación
cantidad de observaciones válidas

La interpretación general será la siguiente:

Si las diferencias entre contextos son moderadas, un threshold global por horizonte puede ser suficiente como benchmark simple.

Si las diferencias entre regímenes son grandes, pero dentro de cada régimen los thresholds son razonablemente estables año a año, entonces los thresholds por `regime_id` serán una alternativa más cercana al comportamiento real del mercado.

En este análisis, los thresholds por año, trimestre o contrato se utilizarán principalmente como diagnóstico de estabilidad, no necesariamente como definición principal. La prioridad será encontrar una metodología suficientemente robusta, interpretable y no excesivamente ajustada al pasado.

En síntesis, este bloque establece el plan de análisis que se aplicará en los siguientes puntos: primero thresholds globales, luego thresholds por régimen, después estabilidad temporal y finalmente selección de thresholds candidatos.

5. Cálculo y diagnóstico de thresholds candidatos

Una vez calculadas las excursiones futuras del precio, el siguiente paso consiste en calcular thresholds candidatos para construir versiones preliminares del target `DIR`.

En esta etapa todavía no se construirá el target. El objetivo será únicamente preparar y calcular las magnitudes de referencia que podrían utilizarse luego como umbrales de movimiento significativo.

La regla metodológica principal será:

Los thresholds se calculan usando únicamente el período 2020-2024.

El período 2025-2026 queda reservado para evaluar posteriormente cómo se comportan esos thresholds ya definidos.

Por lo tanto, el primer paso será preparar el dataframe de trabajo filtrando solamente el período de desarrollo.

5.1. Preparación del dataset para cálculo de thresholds

```
In [59]: # 5.1. Preparación del dataset para cálculo de thresholds
# =====
#
# Objetivo:
# Preparar el dataset que se utilizará para calcular thresholds candidatos.
#
# En este bloque:
# - Verificamos que exista df_threshold_excursions.
# - Validamos que estén las columnas necesarias.
# - Filtramos únicamente el período development_2020_2024.
# - Revisamos la cantidad de filas válidas por horizonte.
#
# Importante:
# En este bloque todavía NO se calculan thresholds.
# =====

import numpy as np
import pandas as pd

# -----
# Verificación de existencia del DataFrame base
# -----

if "df_threshold_excursions" not in globals():
    raise NameError(
        "No existe df_threshold_excursions en memoria. "
        "Primero debes ejecutar el punto 3.3, donde se calculan las excusiones
    )

if df_threshold_excursions.empty:
    raise ValueError(
        "df_threshold_excursions está vacío. "
        "No hay datos disponibles para preparar el cálculo de thresholds."
    )

# -----
# Validación de configuración previa
# -----

if "THRESHOLD_HORIZONS" not in globals():
```

```

    raise NameError(
        "No existe THRESHOLD_HORIZONS en memoria. "
        "Primero debes ejecutar el punto 3.1."
    )

# -----
# Columnas mínimas requeridas
# -----

required_base_cols = [
    "date",
    "close",
    "threshold_calibration_split",
]

missing_base_cols = [
    col for col in required_base_cols
    if col not in df_threshold_excursions.columns
]

if missing_base_cols:
    raise ValueError(
        "Faltan columnas base necesarias en df_threshold_excursions: "
        f"{missing_base_cols}"
    )

# -----
# Columnas de excursiones requeridas por horizonte
# -----

required_excursion_cols = []

for h in THRESHOLD_HORIZONS:
    required_excursion_cols.extend([
        f"up_excursion_pts_{h}m",
        f"down_excursion_pts_{h}m",
        f"max_excursion_pts_{h}m",
    ])

missing_excursion_cols = [
    col for col in required_excursion_cols
    if col not in df_threshold_excursions.columns
]

if missing_excursion_cols:
    raise ValueError(
        "Faltan columnas de excursiones futuras necesarias para calcular thresho"
        f"{missing_excursion_cols}"
    )

print("Validación inicial completada correctamente.")
print("El DataFrame df_threshold_excursions existe y contiene las columnas neces

# -----
# Filtro del período de desarrollo
# -----
#
# Este será el único período usado para calcular thresholds.
# No usamos 2025-2026 en este cálculo.

```

```

df_threshold_base = (
    df_threshold_excursions[
        df_threshold_excursions["threshold_calibration_split"] == "development_2
    ]
    .copy()
)

if df_threshold_base.empty:
    raise ValueError(
        "df_threshold_base está vacío después de filtrar development_2020_2024."
    )

print("\nDataset filtrado para cálculo de thresholds.")
print("Este DataFrame contiene únicamente observaciones del período 2020-2024.")
print("Estas filas serán la base para calcular los thresholds candidatos.")
print(f"Filas disponibles: {df_threshold_base.shape[0]:,}")
print(f"Columnas disponibles: {df_threshold_base.shape[1]:,}")

# -----
# Validación de rango temporal del período filtrado
# -----

start_date = pd.to_datetime(df_threshold_base["date"]).min()
end_date = pd.to_datetime(df_threshold_base["date"]).max()

print("\nRango temporal del dataset usado para calcular thresholds.")
print("Este rango debe estar dentro del período de desarrollo 2020-2024.")
print(f"Fecha inicial: {start_date.date()}")
print(f"Fecha final: {end_date.date()}")

# -----
# Crear columnas auxiliares si no existen
# -----

date_ts = pd.to_datetime(df_threshold_base["date"])

if "year" not in df_threshold_base.columns:
    df_threshold_base["year"] = date_ts.dt.year.astype("int16")

if "quarter" not in df_threshold_base.columns:
    df_threshold_base["quarter"] = date_ts.dt.quarter.astype("int8")

if "year_quarter" not in df_threshold_base.columns:
    df_threshold_base["year_quarter"] = date_ts.dt.to_period("Q").astype(str)

# -----
# Crear regime_name si no existe
# -----

REGIME_NAMES = {
    0: "Overnight",
    1: "Pre-market",
    2: "Opening",
    3: "Regular",
    4: "Closing",
}

if "regime_id" in df_threshold_base.columns and "regime_name" not in df_threshol
    df_threshold_base["regime_name"] = (
        df_threshold_base["regime_id"]

```

```

        .map(REGIME_NAMES)
        .fillna("Unknown")
    )

# -----
# Crear contract_code si existe contract y no existe contract_code
# -----

if "contract" in df_threshold_base.columns and "contract_code" not in df_thresho
    df_threshold_base["contract_code"] = (
        df_threshold_base["contract"]
        .astype(str)
        .str.extract(r"([HMUZ])", expand=False)
    )

# -----
# Revisión de filas válidas por horizonte
# -----

valid_rows_summary = []

for h in THRESHOLD_HORIZONS:

    up_col = f"up_excursion_pts_{h}m"
    down_col = f"down_excursion_pts_{h}m"
    max_col = f"max_excursion_pts_{h}m"

    valid_mask = (
        df_threshold_base[[up_col, down_col, max_col]]
        .notna()
        .all(axis=1)
    )

    valid_rows_summary.append({
        "horizon": h,
        "valid_rows": int(valid_mask.sum()),
        "invalid_rows": int((~valid_mask).sum()),
        "valid_ratio": float(valid_mask.mean()),
        "invalid_ratio": float((~valid_mask).mean()),
    })

df_threshold_base_validity = pd.DataFrame(valid_rows_summary)

print("\nResumen de filas válidas para calcular thresholds por horizonte.")
print("Las filas inválidas corresponden principalmente al final de cada día, don
display(df_threshold_base_validity)

# -----
# Vista previa del dataset base
# -----

preview_cols = [
    "date",
    "close",
    "threshold_calibration_split",
]

optional_preview_cols = [
    "year",
    "quarter",

```

```

        "year_quarter",
        "regime_id",
        "regime_name",
        "contract",
        "contract_code",
    ]

    preview_cols.extend([
        col for col in optional_preview_cols
        if col in df_threshold_base.columns
    ])

    for h in THRESHOLD_HORIZONS:
        preview_cols.extend([
            f"up_excursion_pts_{h}m",
            f"down_excursion_pts_{h}m",
            f"max_excursion_pts_{h}m",
        ])

    print("\nVista previa del dataset que se usará para calcular thresholds.")
    print("Cada fila pertenece al período 2020-2024 y contiene las excursiones futuras")
    display(df_threshold_base[preview_cols].head(10))

```

Validación inicial completada correctamente.

El DataFrame `df_threshold_excursions` existe y contiene las columnas necesarias para continuar.

Dataset filtrado para cálculo de thresholds.

Este DataFrame contiene únicamente observaciones del período 2020-2024.

Estas filas serán la base para calcular los thresholds candidatos.

Filas disponibles: 818,835

Columnas disponibles: 29

Rango temporal del dataset usado para calcular thresholds.

Este rango debe estar dentro del período de desarrollo 2020-2024.

Fecha inicial: 2020-01-02

Fecha final: 2024-12-31

Resumen de filas válidas para calcular thresholds por horizonte.

Las filas inválidas corresponden principalmente al final de cada día, donde no hay suficiente ventana futura.

	horizon	valid_rows	invalid_rows	valid_ratio	invalid_ratio
0	30	783285	35550	0.956585	0.043415
1	60	747735	71100	0.913169	0.086831
2	90	712185	106650	0.869754	0.130246

Vista previa del dataset que se usará para calcular thresholds.

Cada fila pertenece al período 2020-2024 y contiene las excursiones futuras calculadas por horizonte.

	date	close	threshold_calibration_split	year	quarter	year_quarter	regime
datetime							
2020-01-02 04:30:00-05:00	2020-01-02	8813.25	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:31:00-05:00	2020-01-02	8812.50	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:32:00-05:00	2020-01-02	8811.75	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:33:00-05:00	2020-01-02	8810.50	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:34:00-05:00	2020-01-02	8812.00	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:35:00-05:00	2020-01-02	8813.00	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:36:00-05:00	2020-01-02	8813.25	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:37:00-05:00	2020-01-02	8813.25	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:38:00-05:00	2020-01-02	8814.00	development_2020_2024	2020	1	2020Q1	
2020-01-02 04:39:00-05:00	2020-01-02	8813.50	development_2020_2024	2020	1	2020Q1	

5.2. Percentiles globales por horizonte

```
In [60]: # 5.2. Percentiles globales por horizonte
# =====
#
```

```

# Objetivo:
# Calcular percentiles globales de las excursiones futuras usando
# únicamente el período development_2020_2024.
#
# En este bloque:
# - NO se construyen targets.
# - NO se elige todavía un threshold definitivo.
# - Solo se calculan valores candidatos para análisis posterior.
#
# Columnas calculadas:
# - threshold_up_pts
# - threshold_down_pts
# - threshold_common_pts
#
# Regla metodológica:
# 2025-2026 NO participa en este cálculo.
# =====

# -----
# Percentiles a calcular
# -----
#
# p40, p50 y p60 serán candidatos principales.
# p25, p75, p90 y p95 se calculan como referencia estadística adicional.

THRESHOLD_PERCENTILES_MAIN = [40, 50, 60]
THRESHOLD_PERCENTILES_ALL = [25, 40, 50, 60, 75, 90, 95]

# -----
# Validación del dataset base
# -----

if "df_threshold_base" not in globals():
    raise NameError(
        "No existe df_threshold_base en memoria. "
        "Primero debes ejecutar el punto 5.1."
    )

if df_threshold_base.empty:
    raise ValueError(
        "df_threshold_base está vacío. "
        "No hay datos disponibles para calcular thresholds."
    )

# -----
# Función general para calcular percentiles de thresholds
# -----

def compute_threshold_percentiles(
    df,
    horizons,
    percentiles_all,
    percentiles_main,
    threshold_scope,
    group_cols=None,
    selection_period="development_2020_2024"
):
    """
    Calcula percentiles de excursiones futuras para distintos niveles de agrupac

```

Para cada horizonte H calcula:

- threshold_up_pts
- threshold_down_pts
- threshold_common_pts

Si group_cols es None o lista vacía, calcula thresholds globales.

Si group_cols contiene columnas, calcula thresholds por grupo.

"""

```
if group_cols is None:
    group_cols = []
```

```
rows = []
```

```
for h in horizons:
```

```
    up_col = f"up_excursion_pts_{h}m"
    down_col = f"down_excursion_pts_{h}m"
    max_col = f"max_excursion_pts_{h}m"
```

```
    cols_needed = group_cols + [
        up_col,
        down_col,
        max_col,
    ]
```

```
    missing_cols = [
        col for col in cols_needed
        if col not in df.columns
    ]
```

```
    if missing_cols:
        raise ValueError(
            f"Faltan columnas necesarias para H={h}: {missing_cols}"
        )
```

```
    valid_h = (
        df[cols_needed]
        .dropna(subset=[up_col, down_col, max_col])
        .copy()
    )
```

```
    if valid_h.empty:
        raise ValueError(
            f"No hay filas válidas para calcular thresholds en H={h}."
        )
```

```
    if len(group_cols) == 0:
        grouped_items = [(), valid_h]
    else:
        grouped_items = valid_h.groupby(
            group_cols,
            observed=True,
            dropna=False
        )
```

```
    for keys, group in grouped_items:
```

```
        if len(group_cols) == 0:
```

```

        context_values = {}
    else:
        if not isinstance(keys, tuple):
            keys = (keys,)

        context_values = {
            col: value
            for col, value in zip(group_cols, keys)
        }

    for p in percentiles_all:

        row = {
            "threshold_scope": threshold_scope,
            "selection_period": selection_period,
            "horizon": h,
            "percentile": p,
            "threshold_up_pts": float(np.nanpercentile(group[up_col], p))
            "threshold_down_pts": float(np.nanpercentile(group[down_col], p))
            "threshold_common_pts": float(np.nanpercentile(group[max_col], p))
            "n_valid": int(len(group)),
            "candidate_for_threshold": p in percentiles_main,
        }

        row.update(context_values)
        rows.append(row)

    return pd.DataFrame(rows)

# -----
# Cálculo de percentiles globales
# -----

df_thresholds_global = compute_threshold_percentiles(
    df=df_threshold_base,
    horizons=THRESHOLD_HORIZONS,
    percentiles_all=THRESHOLD_PERCENTILES_ALL,
    percentiles_main=THRESHOLD_PERCENTILES_MAIN,
    threshold_scope="global",
    group_cols=None,
)

# -----
# Vista de resultados
# -----

print("Percentiles globales de excursiones futuras calculados con datos de 2020-2024.")
print("Cada fila representa un horizonte y un percentil.")
print("Estos valores son candidatos o referencias para definir thresholds, pero")

display(df_thresholds_global)

```

Percentiles globales de excursiones futuras calculados con datos de 2020-2024.
 Cada fila representa un horizonte y un percentil.
 Estos valores son candidatos o referencias para definir thresholds, pero todavía
 no se selecciona ninguno como definitivo.

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thresh
0	global	development_2020_2024	30	25	4.75	
1	global	development_2020_2024	30	40	9.25	
2	global	development_2020_2024	30	50	13.00	
3	global	development_2020_2024	30	60	17.25	
4	global	development_2020_2024	30	75	26.75	
5	global	development_2020_2024	30	90	47.25	
6	global	development_2020_2024	30	95	64.75	
7	global	development_2020_2024	60	25	7.75	
8	global	development_2020_2024	60	40	14.50	
9	global	development_2020_2024	60	50	19.75	
10	global	development_2020_2024	60	60	26.25	
11	global	development_2020_2024	60	75	40.50	
12	global	development_2020_2024	60	90	69.75	
13	global	development_2020_2024	60	95	94.25	
14	global	development_2020_2024	90	25	10.50	
15	global	development_2020_2024	90	40	18.75	
16	global	development_2020_2024	90	50	25.50	
17	global	development_2020_2024	90	60	33.75	
18	global	development_2020_2024	90	75	51.50	
19	global	development_2020_2024	90	90	88.00	
20	global	development_2020_2024	90	95	117.75	

5.3. Percentiles por régimen intradiario

```
In [61]: # 5.3. Percentiles por régimen intradiario
# =====
#
# Objetivo:
# Calcular percentiles de excursiones futuras por régimen intradiario,
# usando únicamente el período development_2020_2024.
#
# Regla metodológica:
# 2025-2026 NO participa en este cálculo.
# =====

required_cols = [
    "regime_id",
    "regime_name",
]
```

```

missing_cols = [
    col for col in required_cols
    if col not in df_threshold_base.columns
]

if missing_cols:
    raise ValueError(
        "Faltan columnas necesarias para calcular thresholds por régimen: "
        f"{missing_cols}"
    )

df_thresholds_by_regime = compute_threshold_percentiles(
    df=df_threshold_base,
    horizons=THRESHOLD_HORIZONS,
    percentiles_all=THRESHOLD_PERCENTILES_ALL,
    percentiles_main=THRESHOLD_PERCENTILES_MAIN,
    threshold_scope="regime",
    group_cols=[
        "regime_id",
        "regime_name",
    ],
)

df_thresholds_by_regime = (
    df_thresholds_by_regime
    .sort_values(
        [
            "horizon",
            "regime_id",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

print("Percentiles de excursiones futuras por régimen intradiario calculados con")
print("Cada fila representa una combinación de horizonte, régimen y percentil.")
print("Todavía no se selecciona ningún threshold definitivo.")

display(df_thresholds_by_regime)

df_regime_valid_counts = (
    df_thresholds_by_regime
    [["horizon", "regime_id", "regime_name", "n_valid"]]
    .drop_duplicates()
    .sort_values(["horizon", "regime_id"])
    .reset_index(drop=True)
)

print("\nCantidad de observaciones válidas usadas por régimen y horizonte.")
display(df_regime_valid_counts)

```

Percentiles de excursiones futuras por régimen intradiario calculados con datos de 2020-2024.

Cada fila representa una combinación de horizonte, régimen y percentil.

Todavía no se selecciona ningún threshold definitivo.

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thresh
0	regime	development_2020_2024	30	25	3.25	
1	regime	development_2020_2024	30	40	6.25	
2	regime	development_2020_2024	30	50	8.50	
3	regime	development_2020_2024	30	60	11.25	
4	regime	development_2020_2024	30	75	16.75	
...	...	...	...	...	...	...
86	regime	development_2020_2024	90	50	29.75	
87	regime	development_2020_2024	90	60	38.25	
88	regime	development_2020_2024	90	75	55.00	
89	regime	development_2020_2024	90	90	87.50	
90	regime	development_2020_2024	90	95	113.75	

91 rows × 11 columns



Cantidad de observaciones válidas usadas por régimen y horizonte.

	horizon	regime_id	regime_name	n_valid
0	30	0	Overnight	284400
1	30	1	Pre-market	71100
2	30	2	Opening	71100
3	30	3	Regular	355500
4	30	4	Closing	1185
5	60	0	Overnight	284400
6	60	1	Pre-market	71100
7	60	2	Opening	71100
8	60	3	Regular	321135
9	90	0	Overnight	284400
10	90	1	Pre-market	71100
11	90	2	Opening	71100
12	90	3	Regular	285585

5.4. Percentiles por régimen y año

```
In [62]: # 5.4. Percentiles por régimen y año
# =====
#
# Objetivo:
```

```

# Calcular percentiles de excursiones futuras cruzando:
#
# régimen intradiario + año
#
# usando únicamente el período development_2020_2024.
#
# Pregunta:
# ¿Dentro de un mismo régimen, las excursiones cambian mucho según el año?
# =====

required_cols = [
    "year",
    "regime_id",
    "regime_name",
]

missing_cols = [
    col for col in required_cols
    if col not in df_threshold_base.columns
]

if missing_cols:
    raise ValueError(
        "Faltan columnas necesarias para calcular thresholds por régimen y año: "
        f"{missing_cols}"
    )

df_thresholds_by_regime_year = compute_threshold_percentiles(
    df=df_threshold_base,
    horizons=THRESHOLD_HORIZONS,
    percentiles_all=THRESHOLD_PERCENTILES_ALL,
    percentiles_main=THRESHOLD_PERCENTILES_MAIN,
    threshold_scope="regime_year",
    group_cols=[
        "year",
        "regime_id",
        "regime_name",
    ],
)

df_thresholds_by_regime_year = (
    df_thresholds_by_regime_year
    .sort_values(
        [
            "horizon",
            "regime_id",
            "year",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

print("Percentiles de excursiones futuras por régimen intradiario y año calculados")
print("Cada fila representa una combinación de horizonte, régimen, año y percentil")
print("Esta tabla servirá para evaluar estabilidad año a año.")
print("Todavía no se selecciona ningún threshold definitivo.")

display(df_thresholds_by_regime_year)

```

```

df_regime_year_valid_counts = (
    df_thresholds_by_regime_year
    [
        [
            "horizon",
            "year",
            "regime_id",
            "regime_name",
            "n_valid",
        ]
    ]
    .drop_duplicates()
    .sort_values(
        [
            "horizon",
            "regime_id",
            "year",
        ]
    )
    .reset_index(drop=True)
)

print("\nCantidad de observaciones válidas usadas por régimen, año y horizonte.")
display(df_regime_year_valid_counts)

```

Percentiles de excursiones futuras por régimen intradiario y año calculados con datos de 2020-2024.

Cada fila representa una combinación de horizonte, régimen, año y percentil.

Esta tabla servirá para evaluar estabilidad año a año.

Todavía no se selecciona ningún threshold definitivo.

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thres
0	regime_year	development_2020_2024	30	25	2.75	
1	regime_year	development_2020_2024	30	40	5.50	
2	regime_year	development_2020_2024	30	50	7.50	
3	regime_year	development_2020_2024	30	60	10.25	
4	regime_year	development_2020_2024	30	75	15.75	
...	...	...	...	...	...	
450	regime_year	development_2020_2024	90	50	31.25	
451	regime_year	development_2020_2024	90	60	40.00	
452	regime_year	development_2020_2024	90	75	58.00	
453	regime_year	development_2020_2024	90	90	91.00	
454	regime_year	development_2020_2024	90	95	115.25	

455 rows × 12 columns

◀ Cantidad de observaciones válidas usadas por régimen, año y horizonte. ▶

	horizon	year	regime_id	regime_name	n_valid
0	30	2020	0	Overnight	54000
1	30	2021	0	Overnight	58560
2	30	2022	0	Overnight	57600
3	30	2023	0	Overnight	56880
4	30	2024	0	Overnight	57360
...	...	...	...	...	...
60	90	2020	3	Regular	54225
61	90	2021	3	Regular	58804
62	90	2022	3	Regular	57840
63	90	2023	3	Regular	57117
64	90	2024	3	Regular	57599

65 rows × 5 columns

5.5. Percentiles por régimen y trimestre

```
In [63]: # 5.5. Percentiles por régimen y trimestre
# =====
#
# Objetivo:
# Calcular percentiles de excursiones futuras cruzando:
#
# régimen intradiario + trimestre
#
# usando únicamente el período development_2020_2024.
#
# Pregunta:
# ¿Dentro de un mismo régimen, las excursiones cambian mucho
# según el trimestre o período del año?
# =====

required_cols = [
    "year",
    "quarter",
    "year_quarter",
    "regime_id",
    "regime_name",
]

missing_cols = [
    col for col in required_cols
    if col not in df_threshold_base.columns
]

if missing_cols:
    raise ValueError(
        "Faltan columnas necesarias para calcular thresholds por régimen y trimestre"
```

```

        f"{missing_cols}"
    )

df_thresholds_by_regime_quarter = compute_threshold_percentiles(
    df=df_threshold_base,
    horizons=THRESHOLD_HORIZONS,
    percentiles_all=THRESHOLD_PERCENTILES_ALL,
    percentiles_main=THRESHOLD_PERCENTILES_MAIN,
    threshold_scope="regime_year_quarter",
    group_cols=[
        "year",
        "quarter",
        "year_quarter",
        "regime_id",
        "regime_name",
    ],
)

df_thresholds_by_regime_quarter = (
    df_thresholds_by_regime_quarter
    .sort_values(
        [
            "horizon",
            "regime_id",
            "year",
            "quarter",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

print("Percentiles de excursiones futuras por régimen intradiario y trimestre ca
print("Cada fila representa una combinación de horizonte, régimen, trimestre y p
print("Esta tabla servirá para evaluar estabilidad trimestral.")
print("Todavía no se selecciona ningún threshold definitivo.")

print(f"\nFilas generadas: {df_thresholds_by_regime_quarter.shape[0]:,}")
print(f"Columnas generadas: {df_thresholds_by_regime_quarter.shape[1]:,}")

display(df_thresholds_by_regime_quarter.head(30))

df_regime_quarter_valid_counts = (
    df_thresholds_by_regime_quarter
    [
        [
            "horizon",
            "year",
            "quarter",
            "year_quarter",
            "regime_id",
            "regime_name",
            "n_valid",
        ]
    ]
    .drop_duplicates()
    .sort_values(
        [
            "horizon",
            "regime_id",

```

```
        "year",  
        "quarter",  
    ]  
    )  
    .reset_index(drop=True)  
)  
  
print("\nCantidad de observaciones válidas usadas por régimen, trimestre y horizonte")  
print(f"Filas generadas: {df_regime_quarter_valid_counts.shape[0]:,}")  
  
display(df_regime_quarter_valid_counts.head(30))
```

Percentiles de excursiones futuras por régimen intradiario y trimestre calculados con datos de 2020-2024.

Cada fila representa una combinación de horizonte, régimen, trimestre y percentil.

Esta tabla servirá para evaluar estabilidad trimestral.

Todavía no se selecciona ningún threshold definitivo.

Filas generadas: 1,820

Columnas generadas: 14

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thr
0	regime_year_quarter	development_2020_2024	30	25	2.50	
1	regime_year_quarter	development_2020_2024	30	40	4.75	
2	regime_year_quarter	development_2020_2024	30	50	6.50	
3	regime_year_quarter	development_2020_2024	30	60	8.75	
4	regime_year_quarter	development_2020_2024	30	75	14.75	
5	regime_year_quarter	development_2020_2024	30	90	31.00	
6	regime_year_quarter	development_2020_2024	30	95	44.75	
7	regime_year_quarter	development_2020_2024	30	25	3.50	
8	regime_year_quarter	development_2020_2024	30	40	6.50	
9	regime_year_quarter	development_2020_2024	30	50	9.00	
10	regime_year_quarter	development_2020_2024	30	60	11.75	
11	regime_year_quarter	development_2020_2024	30	75	17.00	
12	regime_year_quarter	development_2020_2024	30	90	25.75	
13	regime_year_quarter	development_2020_2024	30	95	33.25	
14	regime_year_quarter	development_2020_2024	30	25	3.00	
15	regime_year_quarter	development_2020_2024	30	40	5.75	
16	regime_year_quarter	development_2020_2024	30	50	8.00	
17	regime_year_quarter	development_2020_2024	30	60	10.75	
18	regime_year_quarter	development_2020_2024	30	75	16.00	
19	regime_year_quarter	development_2020_2024	30	90	28.00	
20	regime_year_quarter	development_2020_2024	30	95	37.25	
21	regime_year_quarter	development_2020_2024	30	25	2.50	
22	regime_year_quarter	development_2020_2024	30	40	5.25	
23	regime_year_quarter	development_2020_2024	30	50	7.25	
24	regime_year_quarter	development_2020_2024	30	60	9.50	
25	regime_year_quarter	development_2020_2024	30	75	15.00	
26	regime_year_quarter	development_2020_2024	30	90	28.25	
27	regime_year_quarter	development_2020_2024	30	95	37.25	
28	regime_year_quarter	development_2020_2024	30	25	3.00	
29	regime_year_quarter	development_2020_2024	30	40	6.00	



Cantidad de observaciones válidas usadas por régimen, trimestre y horizonte.
Filas generadas: 260

	horizon	year	quarter	year_quarter	regime_id	regime_name	n_valid
0	30	2020	1	2020Q1	0	Overnight	12960
1	30	2020	2	2020Q2	0	Overnight	13680
2	30	2020	3	2020Q3	0	Overnight	14640
3	30	2020	4	2020Q4	0	Overnight	12720
4	30	2021	1	2021Q1	0	Overnight	14400
5	30	2021	2	2021Q2	0	Overnight	14640
6	30	2021	3	2021Q3	0	Overnight	14640
7	30	2021	4	2021Q4	0	Overnight	14880
8	30	2022	1	2022Q1	0	Overnight	14640
9	30	2022	2	2022Q2	0	Overnight	14400
10	30	2022	3	2022Q3	0	Overnight	14400
11	30	2022	4	2022Q4	0	Overnight	14160
12	30	2023	1	2023Q1	0	Overnight	14640
13	30	2023	2	2023Q2	0	Overnight	12960
14	30	2023	3	2023Q3	0	Overnight	14640
15	30	2023	4	2023Q4	0	Overnight	14640
16	30	2024	1	2024Q1	0	Overnight	14400
17	30	2024	2	2024Q2	0	Overnight	14880
18	30	2024	3	2024Q3	0	Overnight	13680
19	30	2024	4	2024Q4	0	Overnight	14400
20	30	2020	1	2020Q1	1	Pre-market	3240
21	30	2020	2	2020Q2	1	Pre-market	3420
22	30	2020	3	2020Q3	1	Pre-market	3660
23	30	2020	4	2020Q4	1	Pre-market	3180
24	30	2021	1	2021Q1	1	Pre-market	3600
25	30	2021	2	2021Q2	1	Pre-market	3660
26	30	2021	3	2021Q3	1	Pre-market	3660
27	30	2021	4	2021Q4	1	Pre-market	3720
28	30	2022	1	2022Q1	1	Pre-market	3660
29	30	2022	2	2022Q2	1	Pre-market	3600

5.6. Percentiles por régimen y contrato

```

In [64]: # 5.6. Percentiles por régimen y contrato
# =====
#
# Objetivo:
# Calcular percentiles de excursiones futuras cruzando:
#
# régimen intradiario + contrato
#
# usando únicamente el período development_2020_2024.
#
# Pregunta:
# ¿Dentro de un mismo régimen, las excursiones cambian mucho
# según el contrato activo?
# =====

required_cols = [
    "contract",
    "regime_id",
    "regime_name",
]

missing_cols = [
    col for col in required_cols
    if col not in df_threshold_base.columns
]

if missing_cols:
    raise ValueError(
        "Faltan columnas necesarias para calcular thresholds por régimen y contrato"
        f"{missing_cols}"
    )

df_thresholds_by_regime_contract = compute_threshold_percentiles(
    df=df_threshold_base,
    horizons=THRESHOLD_HORIZONS,
    percentiles_all=THRESHOLD_PERCENTILES_ALL,
    percentiles_main=THRESHOLD_PERCENTILES_MAIN,
    threshold_scope="regime_contract",
    group_cols=[
        "contract",
        "regime_id",
        "regime_name",
    ],
)

df_thresholds_by_regime_contract = (
    df_thresholds_by_regime_contract
    .sort_values(
        [
            "horizon",
            "regime_id",
            "contract",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

print("Percentiles de excursiones futuras por régimen intradiario y contrato calculados")

```

```

print("Cada fila representa una combinación de horizonte, régimen, contrato y pe
print("Esta tabla servirá para evaluar estabilidad entre contratos.")
print("Todavía no se selecciona ningún threshold definitivo.")

print(f"\nFilas generadas: {df_thresholds_by_regime_contract.shape[0]:,}")
print(f"Columnas generadas: {df_thresholds_by_regime_contract.shape[1]:,}")

display(df_thresholds_by_regime_contract.head(30))

df_regime_contract_valid_counts = (
    df_thresholds_by_regime_contract
    [
        [
            "horizon",
            "contract",
            "regime_id",
            "regime_name",
            "n_valid",
        ]
    ]
    .drop_duplicates()
    .sort_values(
        [
            "horizon",
            "regime_id",
            "contract",
        ]
    )
    .reset_index(drop=True)
)

print("\nCantidad de observaciones válidas usadas por régimen, contrato y horizo
print(f"Filas generadas: {df_regime_contract_valid_counts.shape[0]:,}")

display(df_regime_contract_valid_counts.head(30))

```

Percentiles de excursiones futuras por régimen intradiario y contrato calculados con datos de 2020-2024.

Cada fila representa una combinación de horizonte, régimen, contrato y percentil. Esta tabla servirá para evaluar estabilidad entre contratos.

Todavía no se selecciona ningún threshold definitivo.

Filas generadas: 1,911

Columnas generadas: 12

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thresh
0	regime_contract	development_2020_2024	30	25	2.2500	
1	regime_contract	development_2020_2024	30	40	4.2500	
2	regime_contract	development_2020_2024	30	50	5.7500	
3	regime_contract	development_2020_2024	30	60	7.7500	
4	regime_contract	development_2020_2024	30	75	12.7500	
5	regime_contract	development_2020_2024	30	90	25.2500	
6	regime_contract	development_2020_2024	30	95	36.7500	
7	regime_contract	development_2020_2024	30	25	2.5000	
8	regime_contract	development_2020_2024	30	40	5.5000	
9	regime_contract	development_2020_2024	30	50	7.5000	
10	regime_contract	development_2020_2024	30	60	10.0000	
11	regime_contract	development_2020_2024	30	75	16.0000	
12	regime_contract	development_2020_2024	30	90	28.7500	
13	regime_contract	development_2020_2024	30	95	40.7500	
14	regime_contract	development_2020_2024	30	25	4.7500	
15	regime_contract	development_2020_2024	30	40	9.7500	
16	regime_contract	development_2020_2024	30	50	13.5000	
17	regime_contract	development_2020_2024	30	60	18.1000	
18	regime_contract	development_2020_2024	30	75	27.2500	
19	regime_contract	development_2020_2024	30	90	44.5000	
20	regime_contract	development_2020_2024	30	95	60.7625	
21	regime_contract	development_2020_2024	30	25	3.2500	
22	regime_contract	development_2020_2024	30	40	6.7500	
23	regime_contract	development_2020_2024	30	50	9.0000	
24	regime_contract	development_2020_2024	30	60	11.7500	
25	regime_contract	development_2020_2024	30	75	17.2500	
26	regime_contract	development_2020_2024	30	90	27.7500	
27	regime_contract	development_2020_2024	30	95	37.0000	
28	regime_contract	development_2020_2024	30	25	2.7500	
29	regime_contract	development_2020_2024	30	40	5.5000	



Cantidad de observaciones válidas usadas por régimen, contrato y horizonte.
Filas generadas: 273

	horizon	contract	regime_id	regime_name	n_valid
0	30	H20	0	Overnight	11520
1	30	H21	0	Overnight	14160
2	30	H22	0	Overnight	14640
3	30	H23	0	Overnight	14400
4	30	H24	0	Overnight	14400
5	30	H25	0	Overnight	960
6	30	M20	0	Overnight	13440
7	30	M21	0	Overnight	14640
8	30	M22	0	Overnight	14640
9	30	M23	0	Overnight	13200
10	30	M24	0	Overnight	15840
11	30	U20	0	Overnight	14640
12	30	U21	0	Overnight	14400
13	30	U22	0	Overnight	14160
14	30	U23	0	Overnight	14400
15	30	U24	0	Overnight	13440
16	30	Z20	0	Overnight	12720
17	30	Z21	0	Overnight	14880
18	30	Z22	0	Overnight	14160
19	30	Z23	0	Overnight	14880
20	30	Z24	0	Overnight	14880
21	30	H20	1	Pre-market	2880
22	30	H21	1	Pre-market	3540
23	30	H22	1	Pre-market	3660
24	30	H23	1	Pre-market	3600
25	30	H24	1	Pre-market	3600
26	30	H25	1	Pre-market	240
27	30	M20	1	Pre-market	3360
28	30	M21	1	Pre-market	3660
29	30	M22	1	Pre-market	3660

5.7. Percentiles por régimen y familia de contrato

```

In [65]: # 5.7. Percentiles por régimen y familia de contrato
# =====
#
# Objetivo:
# Calcular percentiles de excursiones futuras cruzando:
#
# régimen intradiario + familia de contrato
#
# usando únicamente el período development_2020_2024.
#
# Pregunta:
# ¿Dentro de un mismo régimen, las excursiones cambian mucho según
# la familia del contrato: H, M, U o Z?
# =====

required_cols = [
    "contract_code",
    "regime_id",
    "regime_name",
]

missing_cols = [
    col for col in required_cols
    if col not in df_threshold_base.columns
]

if missing_cols:
    raise ValueError(
        "Faltan columnas necesarias para calcular thresholds por régimen y familia de contrato: "
        f"{missing_cols}"
    )

if df_threshold_base["contract_code"].isna().all():
    raise ValueError(
        "contract_code existe, pero todos sus valores son NaN. "
        "Revisar el formato de la columna contract."
    )

df_thresholds_by_regime_contract_family = compute_threshold_percentiles(
    df=df_threshold_base.dropna(subset=["contract_code"]),
    horizons=THRESHOLD_HORIZONS,
    percentiles_all=THRESHOLD_PERCENTILES_ALL,
    percentiles_main=THRESHOLD_PERCENTILES_MAIN,
    threshold_scope="regime_contract_family",
    group_cols=[
        "contract_code",
        "regime_id",
        "regime_name",
    ],
)

df_thresholds_by_regime_contract_family = (
    df_thresholds_by_regime_contract_family
    .sort_values(
        [
            "horizon",
            "regime_id",
            "contract_code",
            "percentile",
        ]
    )
)

```

```

    ]
    )
    .reset_index(drop=True)
)

print("Percentiles de excursiones futuras por régimen intradiario y familia de c
print("Cada fila representa una combinación de horizonte, régimen, familia de co
print("Esta tabla se usará para comparar la estabilidad de los thresholds entre
print("No se selecciona ningún threshold definitivo en este bloque.")

print(f"\nTabla principal generada: df_thresholds_by_regime_contract_family")
print(f"Filas: {df_thresholds_by_regime_contract_family.shape[0]:,}")
print(f"Columnas: {df_thresholds_by_regime_contract_family.shape[1]:,}")

display(df_thresholds_by_regime_contract_family.head(30))

df_regime_contract_family_valid_counts = (
    df_thresholds_by_regime_contract_family
    [
        [
            "horizon",
            "contract_code",
            "regime_id",
            "regime_name",
            "n_valid",
        ]
    ]
    .drop_duplicates()
    .sort_values(
        [
            "horizon",
            "regime_id",
            "contract_code",
        ]
    )
    .reset_index(drop=True)
)

print("\nResumen de observaciones válidas por horizonte, régimen y familia de co
print(f"Filas: {df_regime_contract_family_valid_counts.shape[0]:,}")
print(f"Columnas: {df_regime_contract_family_valid_counts.shape[1]:,}")

display(df_regime_contract_family_valid_counts.head(30))

```

Percentiles de excursiones futuras por régimen intradiario y familia de contrato calculados con datos de 2020-2024.

Cada fila representa una combinación de horizonte, régimen, familia de contrato y percentil.

Esta tabla se usará para comparar la estabilidad de los thresholds entre familias de contrato.

No se selecciona ningún threshold definitivo en este bloque.

Tabla principal generada: df_thresholds_by_regime_contract_family

Filas: 364

Columnas: 12

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	1
0	regime_contract_family	development_2020_2024	30	25	3.00	
1	regime_contract_family	development_2020_2024	30	40	6.00	
2	regime_contract_family	development_2020_2024	30	50	8.50	
3	regime_contract_family	development_2020_2024	30	60	11.25	
4	regime_contract_family	development_2020_2024	30	75	17.50	
5	regime_contract_family	development_2020_2024	30	90	30.75	
6	regime_contract_family	development_2020_2024	30	95	42.25	
7	regime_contract_family	development_2020_2024	30	25	3.25	
8	regime_contract_family	development_2020_2024	30	40	6.25	
9	regime_contract_family	development_2020_2024	30	50	8.75	
10	regime_contract_family	development_2020_2024	30	60	11.25	
11	regime_contract_family	development_2020_2024	30	75	16.75	
12	regime_contract_family	development_2020_2024	30	90	27.75	
13	regime_contract_family	development_2020_2024	30	95	37.75	
14	regime_contract_family	development_2020_2024	30	25	3.00	
15	regime_contract_family	development_2020_2024	30	40	6.00	
16	regime_contract_family	development_2020_2024	30	50	8.00	
17	regime_contract_family	development_2020_2024	30	60	10.75	
18	regime_contract_family	development_2020_2024	30	75	15.75	
19	regime_contract_family	development_2020_2024	30	90	25.50	
20	regime_contract_family	development_2020_2024	30	95	33.25	
21	regime_contract_family	development_2020_2024	30	25	3.50	
22	regime_contract_family	development_2020_2024	30	40	6.75	
23	regime_contract_family	development_2020_2024	30	50	9.25	
24	regime_contract_family	development_2020_2024	30	60	12.00	
25	regime_contract_family	development_2020_2024	30	75	17.50	
26	regime_contract_family	development_2020_2024	30	90	29.00	
27	regime_contract_family	development_2020_2024	30	95	38.50	
28	regime_contract_family	development_2020_2024	30	25	5.00	
29	regime_contract_family	development_2020_2024	30	40	10.50	

◀  ▶

Resumen de observaciones válidas por horizonte, régimen y familia de contrato.

Filas: 52

Columnas: 5

	horizon	contract_code	regime_id	regime_name	n_valid
0	30	H	0	Overnight	70080
1	30	M	0	Overnight	71760
2	30	U	0	Overnight	71040
3	30	Z	0	Overnight	71520
4	30	H	1	Pre-market	17520
5	30	M	1	Pre-market	17940
6	30	U	1	Pre-market	17760
7	30	Z	1	Pre-market	17880
8	30	H	2	Opening	17520
9	30	M	2	Opening	17940
10	30	U	2	Opening	17760
11	30	Z	2	Opening	17880
12	30	H	3	Regular	87600
13	30	M	3	Regular	89700
14	30	U	3	Regular	88800
15	30	Z	3	Regular	89400
16	30	H	4	Closing	292
17	30	M	4	Closing	299
18	30	U	4	Closing	296
19	30	Z	4	Closing	298
20	60	H	0	Overnight	70080
21	60	M	0	Overnight	71760
22	60	U	0	Overnight	71040
23	60	Z	0	Overnight	71520
24	60	H	1	Pre-market	17520
25	60	M	1	Pre-market	17940
26	60	U	1	Pre-market	17760
27	60	Z	1	Pre-market	17880
28	60	H	2	Opening	17520
29	60	M	2	Opening	17940

5.8. Consolidación de tablas de thresholds

```

In [66]: # 5.8. Consolidación de tablas de thresholds
# =====
#
# Objetivo:
# Unificar todas las tablas de percentiles calculadas en una sola tabla.
#
# Tablas que se consolidan:
# - df_thresholds_global
# - df_thresholds_by_regime
# - df_thresholds_by_regime_year
# - df_thresholds_by_regime_quarter
# - df_thresholds_by_regime_contract
# - df_thresholds_by_regime_contract_family
#
# En este bloque:
# - No se construyen targets.
# - No se selecciona todavía un threshold definitivo.
# - No se interpreta todavía si un threshold es bueno o malo.
# - Solo se organiza la información para el análisis posterior.
# =====

# -----
# Definir las tablas que deben existir
# -----

threshold_tables = {
    "global": "df_thresholds_global",
    "regime": "df_thresholds_by_regime",
    "regime_year": "df_thresholds_by_regime_year",
    "regime_year_quarter": "df_thresholds_by_regime_quarter",
    "regime_contract": "df_thresholds_by_regime_contract",
    "regime_contract_family": "df_thresholds_by_regime_contract_family",
}

# -----
# Validar que todas las tablas estén creadas
# -----

missing_tables = [
    table_name
    for table_name in threshold_tables.values()
    if table_name not in globals()
]

if missing_tables:
    raise NameError(
        "Faltan tablas de thresholds para consolidar. "
        f"Tablas faltantes: {missing_tables}"
    )

# -----
# Columnas estándar de consolidación
# -----

standard_cols = [
    "threshold_scope",
    "selection_period",
    "horizon",
    "percentile",

```

```

        "threshold_up_pts",
        "threshold_down_pts",
        "threshold_common_pts",
        "n_valid",
        "candidate_for_threshold",
        "year",
        "quarter",
        "year_quarter",
        "contract",
        "contract_code",
        "regime_id",
        "regime_name",
    ]

# -----
# Consolidar tablas
# -----

consolidated_tables = []

for scope_name, table_var_name in threshold_tables.items():

    df_tmp = globals()[table_var_name].copy()

    # Guardamos el nombre de la tabla original.
    df_tmp["source_table"] = table_var_name

    # Si alguna columna estándar no existe, se crea como NaN.
    for col in standard_cols:
        if col not in df_tmp.columns:
            df_tmp[col] = np.nan

    # Reordenamos columnas principales.
    df_tmp = df_tmp[standard_cols + ["source_table"]]

    consolidated_tables.append(df_tmp)

df_thresholds_all = pd.concat(
    consolidated_tables,
    ignore_index=True
)

# -----
# Ajuste de tipos de datos
# -----

numeric_cols = [
    "horizon",
    "percentile",
    "threshold_up_pts",
    "threshold_down_pts",
    "threshold_common_pts",
    "n_valid",
    "year",
    "quarter",
    "regime_id",
]

for col in numeric_cols:
    df_thresholds_all[col] = pd.to_numeric(

```

```

        df_thresholds_all[col],
        errors="coerce"
    )

df_thresholds_all["candidate_for_threshold"] = (
    df_thresholds_all["candidate_for_threshold"]
    .fillna(False)
    .astype(bool)
)

# -----
# Crear etiqueta de contexto
# -----

def build_threshold_context_label(row):
    """
    Construye una etiqueta legible del contexto de cada threshold.
    """

    scope = row["threshold_scope"]

    if scope == "global":
        return "global"

    if scope == "regime":
        return str(row["regime_name"])

    if scope == "regime_year":
        return f"{row['regime_name']} | {int(row['year'])}"

    if scope == "regime_year_quarter":
        return f"{row['regime_name']} | {row['year_quarter']}"

    if scope == "regime_contract":
        return f"{row['regime_name']} | {row['contract']}"

    if scope == "regime_contract_family":
        return f"{row['regime_name']} | {row['contract_code']}"

    return str(scope)

df_thresholds_all["context_label"] = (
    df_thresholds_all
    .apply(build_threshold_context_label, axis=1)
)

# -----
# Ordenar tabla consolidada
# -----

threshold_scope_order = {
    "global": 0,
    "regime": 1,
    "regime_year": 2,
    "regime_year_quarter": 3,
    "regime_contract": 4,
    "regime_contract_family": 5,
}

```

```

df_thresholds_all["threshold_scope_order"] = (
    df_thresholds_all["threshold_scope"]
    .map(threshold_scope_order)
)

df_thresholds_all = (
    df_thresholds_all
    .sort_values(
        [
            "threshold_scope_order",
            "horizon",
            "regime_id",
            "year",
            "quarter",
            "contract",
            "contract_code",
            "percentile",
        ],
        na_position="last"
    )
    .reset_index(drop=True)
)

# -----
# Crear versión larga de thresholds
# -----
#
# La tabla original tiene tres columnas:
# - threshold_up_pts
# - threshold_down_pts
# - threshold_common_pts
#
# Para el análisis de estabilidad será más cómodo tener:
#
# threshold_type | threshold_pts
# up             | ...
# down           | ...
# common         | ...

df_thresholds_all_long = (
    df_thresholds_all
    .melt(
        id_vars=[
            "threshold_scope",
            "threshold_scope_order",
            "selection_period",
            "horizon",
            "percentile",
            "candidate_for_threshold",
            "year",
            "quarter",
            "year_quarter",
            "contract",
            "contract_code",
            "regime_id",
            "regime_name",
            "context_label",
            "n_valid",
            "source_table",
        ],
    )

```

```

        value_vars=[
            "threshold_up_pts",
            "threshold_down_pts",
            "threshold_common_pts",
        ],
        var_name="threshold_type",
        value_name="threshold_pts"
    )
)

threshold_type_map = {
    "threshold_up_pts": "up",
    "threshold_down_pts": "down",
    "threshold_common_pts": "common",
}

df_thresholds_all_long["threshold_type"] = (
    df_thresholds_all_long["threshold_type"]
    .map(threshold_type_map)
)

# -----
# Tabla filtrada con percentiles candidatos principales
# -----

df_thresholds_candidates_main = (
    df_thresholds_all[
        df_thresholds_all["candidate_for_threshold"]
    ]
    .copy()
    .reset_index(drop=True)
)

df_thresholds_candidates_main_long = (
    df_thresholds_all_long[
        df_thresholds_all_long["candidate_for_threshold"]
    ]
    .copy()
    .reset_index(drop=True)
)

# -----
# Resumen técnico por tipo de tabla
# -----

df_thresholds_consolidation_summary = (
    df_thresholds_all
    .groupby(
        [
            "threshold_scope",
            "source_table",
        ],
        observed=True
    )
    .agg(
        rows=("threshold_scope", "size"),
        horizons=("horizon", "nunique"),
        percentiles=("percentile", "nunique"),
        min_n_valid=("n_valid", "min"),
        max_n_valid=("n_valid", "max"),
    )
)

```

```

    )
    .reset_index()
    .sort_values("threshold_scope")
    .reset_index(drop=True)
)

# -----
# Vista técnica de resultados
# -----

print("Tablas de thresholds consolidadas correctamente.")
print("La tabla principal contiene todos los percentiles calculados en los disti")
print("La versión larga reorganiza los thresholds en formato threshold_type / th")
print("No se interpreta ni se selecciona ningún threshold definitivo en este blo")

print(f"\nTabla consolidada: df_thresholds_all")
print(f"Filas: {df_thresholds_all.shape[0]:,}")
print(f"Columnas: {df_thresholds_all.shape[1]:,}")

display(df_thresholds_all.head(30))

print(f"\nTabla consolidada en formato largo: df_thresholds_all_long")
print(f"Filas: {df_thresholds_all_long.shape[0]:,}")
print(f"Columnas: {df_thresholds_all_long.shape[1]:,}")

display(df_thresholds_all_long.head(30))

print("\nResumen técnico de tablas consolidadas.")
display(df_thresholds_consolidation_summary)

```

Tablas de thresholds consolidadas correctamente.

La tabla principal contiene todos los percentiles calculados en los distintos niveles de análisis.

La versión larga reorganiza los thresholds en formato threshold_type / threshold_pts para facilitar el análisis de estabilidad posterior.

No se interpreta ni se selecciona ningún threshold definitivo en este bloque.

Tabla consolidada: df_thresholds_all

Filas: 4,662

Columnas: 19

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thresh
0	global	development_2020_2024	30	25	4.75	
1	global	development_2020_2024	30	40	9.25	
2	global	development_2020_2024	30	50	13.00	
3	global	development_2020_2024	30	60	17.25	
4	global	development_2020_2024	30	75	26.75	
5	global	development_2020_2024	30	90	47.25	
6	global	development_2020_2024	30	95	64.75	
7	global	development_2020_2024	60	25	7.75	
8	global	development_2020_2024	60	40	14.50	
9	global	development_2020_2024	60	50	19.75	
10	global	development_2020_2024	60	60	26.25	
11	global	development_2020_2024	60	75	40.50	
12	global	development_2020_2024	60	90	69.75	
13	global	development_2020_2024	60	95	94.25	
14	global	development_2020_2024	90	25	10.50	
15	global	development_2020_2024	90	40	18.75	
16	global	development_2020_2024	90	50	25.50	
17	global	development_2020_2024	90	60	33.75	
18	global	development_2020_2024	90	75	51.50	
19	global	development_2020_2024	90	90	88.00	
20	global	development_2020_2024	90	95	117.75	
21	regime	development_2020_2024	30	25	3.25	
22	regime	development_2020_2024	30	40	6.25	
23	regime	development_2020_2024	30	50	8.50	
24	regime	development_2020_2024	30	60	11.25	
25	regime	development_2020_2024	30	75	16.75	
26	regime	development_2020_2024	30	90	28.25	
27	regime	development_2020_2024	30	95	38.00	
28	regime	development_2020_2024	30	25	5.25	
29	regime	development_2020_2024	30	40	10.50	

◀  ▶

Tabla consolidada en formato largo: df_thresholds_all_long

Filas: 13,986

Columnas: 18

	threshold_scope	threshold_scope_order	selection_period	horizon	percentile
0	global	0	development_2020_2024	30	25
1	global	0	development_2020_2024	30	40
2	global	0	development_2020_2024	30	50
3	global	0	development_2020_2024	30	60
4	global	0	development_2020_2024	30	75
5	global	0	development_2020_2024	30	90
6	global	0	development_2020_2024	30	95
7	global	0	development_2020_2024	60	25
8	global	0	development_2020_2024	60	40
9	global	0	development_2020_2024	60	50
10	global	0	development_2020_2024	60	60
11	global	0	development_2020_2024	60	75
12	global	0	development_2020_2024	60	90
13	global	0	development_2020_2024	60	95
14	global	0	development_2020_2024	90	25
15	global	0	development_2020_2024	90	40
16	global	0	development_2020_2024	90	50
17	global	0	development_2020_2024	90	60
18	global	0	development_2020_2024	90	75
19	global	0	development_2020_2024	90	90
20	global	0	development_2020_2024	90	95
21	regime	1	development_2020_2024	30	25
22	regime	1	development_2020_2024	30	40
23	regime	1	development_2020_2024	30	50
24	regime	1	development_2020_2024	30	60
25	regime	1	development_2020_2024	30	75
26	regime	1	development_2020_2024	30	90
27	regime	1	development_2020_2024	30	95
28	regime	1	development_2020_2024	30	25
29	regime	1	development_2020_2024	30	40



Resumen técnico de tablas consolidadas.

	threshold_scope	source_table	rows	horizons	percentile
0	global	df_thresholds_global	21	3	
1	regime	df_thresholds_by_regime	91	3	
2	regime_contract	df_thresholds_by_regime_contract	1911	3	
3	regime_contract_family	df_thresholds_by_regime_contract_family	364	3	
4	regime_year	df_thresholds_by_regime_year	455	3	
5	regime_year_quarter	df_thresholds_by_regime_quarter	1820	3	

5.9. Análisis comparativo de estabilidad

```
In [77]: # 5.9. Análisis comparativo de estabilidad de thresholds
# =====
#
# Objetivo:
# Evaluar qué tan estables son los thresholds entre distintos contextos.
#
# A partir de la tabla consolidada en formato largo:
#
# df_thresholds_all_long
#
# se calcularán métricas como:
#
# - cantidad de contextos analizados
# - threshold mínimo
# - threshold mediano
# - threshold máximo
# - ratio top / low
# - coeficiente de variación
# - comparación contra threshold global
# - estabilidad año a año dentro de cada régimen
#
# En este bloque:
# - No se construyen targets.
# - No se selecciona todavía un threshold definitivo.
# - Solo se mide la estabilidad relativa de los thresholds candidatos.
#
# Regla metodológica:
# Todo el análisis sigue basado en thresholds calculados con 2020-2024.
# =====

import numpy as np
import pandas as pd

# -----
# Validación de tabla consolidada
# -----

if "df_thresholds_all_long" not in globals():
    raise NameError(
        "No existe df_thresholds_all_long en memoria. "
        "Primero debes ejecutar el punto 5.8 de consolidación."
```

```

    )

if df_thresholds_all_long.empty:
    raise ValueError(
        "df_thresholds_all_long está vacío. "
        "No hay datos disponibles para analizar estabilidad."
    )

required_cols = [
    "threshold_scope",
    "horizon",
    "percentile",
    "threshold_type",
    "threshold_pts",
    "context_label",
    "n_valid",
    "candidate_for_threshold",
]

missing_cols = [
    col for col in required_cols
    if col not in df_thresholds_all_long.columns
]

if missing_cols:
    raise ValueError(
        "Faltan columnas necesarias en df_thresholds_all_long: "
        f"{missing_cols}"
    )

# -----
# Preparación de base de análisis
# -----

df_threshold_stability_base = (
    df_thresholds_all_long
    .copy()
    .dropna(subset=["threshold_pts"])
)

df_threshold_stability_base["threshold_pts"] = pd.to_numeric(
    df_threshold_stability_base["threshold_pts"],
    errors="coerce"
)

df_threshold_stability_base["n_valid"] = pd.to_numeric(
    df_threshold_stability_base["n_valid"],
    errors="coerce"
)

df_threshold_stability_base = (
    df_threshold_stability_base
    .dropna(subset=["threshold_pts"])
    .reset_index(drop=True)
)

print("Base preparada para análisis comparativo de estabilidad de thresholds.")
print("La tabla contiene thresholds en formato largo, separados por tipo: up, do")
print("No se modifica ningún threshold en este bloque; solo se calculan métricas")

```

```
print(f"\nFilas disponibles: {df_threshold_stability_base.shape[0]:,}")
print(f"Columnas disponibles: {df_threshold_stability_base.shape[1]:,}")

display(df_threshold_stability_base.head(20))
```

Base preparada para análisis comparativo de estabilidad de thresholds.
La tabla contiene thresholds en formato largo, separados por tipo: up, down y com mon.
No se modifica ningún threshold en este bloque; solo se calculan métricas compara tivas.

Filas disponibles: 13,986
Columnas disponibles: 18

	threshold_scope	threshold_scope_order	selection_period	horizon	percentile
0	global	0	development_2020_2024	30	25
1	global	0	development_2020_2024	30	40
2	global	0	development_2020_2024	30	50
3	global	0	development_2020_2024	30	60
4	global	0	development_2020_2024	30	75
5	global	0	development_2020_2024	30	90
6	global	0	development_2020_2024	30	95
7	global	0	development_2020_2024	60	25
8	global	0	development_2020_2024	60	40
9	global	0	development_2020_2024	60	50
10	global	0	development_2020_2024	60	60
11	global	0	development_2020_2024	60	75
12	global	0	development_2020_2024	60	90
13	global	0	development_2020_2024	60	95
14	global	0	development_2020_2024	90	25
15	global	0	development_2020_2024	90	40
16	global	0	development_2020_2024	90	50
17	global	0	development_2020_2024	90	60
18	global	0	development_2020_2024	90	75
19	global	0	development_2020_2024	90	90

5.9.1. Métricas de estabilidad por tipo de contexto

```
In [78]: # 5.9.1. Métricas de estabilidad por tipo de contexto
# =====
#
# Objetivo:
# Calcular métricas de dispersión de thresholds dentro de cada
```

```

# tipo de contexto.
#
# Ejemplos de contexto:
# - regime
# - regime_year
# - regime_year_quarter
# - regime_contract
# - regime_contract_family
#
# La tabla global se excluye de este cálculo porque no tiene dispersión
# entre contextos: es un único valor por horizonte, percentil y tipo.
# =====

# -----
# Excluir thresholds globales para medir variabilidad contextual
# -----

df_context_thresholds = (
    df_threshold_stability_base[
        df_threshold_stability_base["threshold_scope"] != "global"
    ]
    .copy()
)

if df_context_thresholds.empty:
    raise ValueError(
        "No hay thresholds contextuales disponibles para analizar estabilidad."
    )

# -----
# Función auxiliar para coeficiente de variación
# -----

def safe_cv(series):
    """
    Calcula coeficiente de variación:

    std / mean

    Si la media es cero o no válida, devuelve NaN.
    """

    mean_value = series.mean()
    std_value = series.std(ddof=0)

    if pd.isna(mean_value) or mean_value == 0:
        return np.nan

    return std_value / mean_value

# -----
# Cálculo de estabilidad
# -----

df_thresholds_stability = (
    df_context_thresholds
    .groupby(
        [
            "threshold_scope",

```

```

        "horizon",
        "percentile",
        "threshold_type",
    ],
    observed=True
)
.agg(
    n_contexts=("context_label", "nunique"),
    n_rows=("threshold_pts", "size"),
    total_valid_obs=("n_valid", "sum"),
    min_threshold_pts=("threshold_pts", "min"),
    p25_threshold_pts=("threshold_pts", lambda x: np.nanpercentile(x, 25)),
    median_threshold_pts=("threshold_pts", "median"),
    mean_threshold_pts=("threshold_pts", "mean"),
    p75_threshold_pts=("threshold_pts", lambda x: np.nanpercentile(x, 75)),
    max_threshold_pts=("threshold_pts", "max"),
    std_threshold_pts=("threshold_pts", lambda x: x.std(ddof=0)),
    cv_threshold=("threshold_pts", safe_cv),
)
.reset_index()
)

# -----
# Ratio top / Low
# -----

df_thresholds_stability["has_zero_min_threshold"] = (
    df_thresholds_stability["min_threshold_pts"] <= 0
)

df_thresholds_stability["top_to_low_ratio"] = np.where(
    df_thresholds_stability["min_threshold_pts"] > 0,
    (
        df_thresholds_stability["max_threshold_pts"]
        / df_thresholds_stability["min_threshold_pts"]
    ),
    np.nan
)

df_thresholds_stability["range_threshold_pts"] = (
    df_thresholds_stability["max_threshold_pts"]
    - df_thresholds_stability["min_threshold_pts"]
)

# -----
# Etiqueta simple de estabilidad
# -----
#
# Esta clasificación es diagnóstica, no una regla definitiva.
#
# estable:
#   diferencias relativamente bajas entre contextos.
#
# moderada:
#   diferencias visibles, pero no extremas.
#
# inestable:
#   diferencias altas entre contextos.

def classify_threshold_stability(row):

```

```

cv = row["cv_threshold"]
ratio = row["top_to_low_ratio"]

if pd.isna(cv) or pd.isna(ratio):
    return "revisar"

if cv <= 0.15 and ratio <= 1.50:
    return "estable"

if cv <= 0.35 and ratio <= 2.50:
    return "moderada"

return "inestable"

df_thresholds_stability["stability_label"] = (
    df_thresholds_stability
    .apply(classify_threshold_stability, axis=1)
)

# -----
# Ordenar resultados
# -----

scope_order = {
    "regime": 1,
    "regime_year": 2,
    "regime_year_quarter": 3,
    "regime_contract": 4,
    "regime_contract_family": 5,
}

type_order = {
    "common": 0,
    "up": 1,
    "down": 2,
}

df_thresholds_stability["scope_order"] = (
    df_thresholds_stability["threshold_scope"]
    .map(scope_order)
)

df_thresholds_stability["threshold_type_order"] = (
    df_thresholds_stability["threshold_type"]
    .map(type_order)
)

df_thresholds_stability = (
    df_thresholds_stability
    .sort_values(
        [
            "scope_order",
            "horizon",
            "percentile",
            "threshold_type_order",
        ]
    )
    .reset_index(drop=True)
)

```

```
print("Métricas de estabilidad calculadas por tipo de contexto.")
print("Cada fila resume qué tan dispersos son los thresholds para una combinació
print("La etiqueta stability_label es diagnóstica y se revisará al final junto c

print(f"\nTabla generada: df_thresholds_stability")
print(f"Filas: {df_thresholds_stability.shape[0]:,}")
print(f"Columnas: {df_thresholds_stability.shape[1]:,}")

display(df_thresholds_stability.head(40))
```

Métricas de estabilidad calculadas por tipo de contexto.

Cada fila resume qué tan dispersos son los thresholds para una combinación de con
texto, horizonte, percentil y tipo de excursión.

La etiqueta stability_label es diagnóstica y se revisará al final junto con el re
sto del análisis.

Tabla generada: df_thresholds_stability

Filas: 315

Columnas: 21

	threshold_scope	horizon	percentile	threshold_type	n_contexts	n_rows	total_valid_c
0	regime	30	25	common	5	5	783%
1	regime	30	25	up	5	5	783%
2	regime	30	25	down	5	5	783%
3	regime	30	40	common	5	5	783%
4	regime	30	40	up	5	5	783%
5	regime	30	40	down	5	5	783%
6	regime	30	50	common	5	5	783%
7	regime	30	50	up	5	5	783%
8	regime	30	50	down	5	5	783%
9	regime	30	60	common	5	5	783%
10	regime	30	60	up	5	5	783%
11	regime	30	60	down	5	5	783%
12	regime	30	75	common	5	5	783%
13	regime	30	75	up	5	5	783%
14	regime	30	75	down	5	5	783%
15	regime	30	90	common	5	5	783%
16	regime	30	90	up	5	5	783%
17	regime	30	90	down	5	5	783%
18	regime	30	95	common	5	5	783%
19	regime	30	95	up	5	5	783%
20	regime	30	95	down	5	5	783%
21	regime	60	25	common	4	4	747%
22	regime	60	25	up	4	4	747%
23	regime	60	25	down	4	4	747%
24	regime	60	40	common	4	4	747%
25	regime	60	40	up	4	4	747%
26	regime	60	40	down	4	4	747%
27	regime	60	50	common	4	4	747%
28	regime	60	50	up	4	4	747%
29	regime	60	50	down	4	4	747%
30	regime	60	60	common	4	4	747%
31	regime	60	60	up	4	4	747%
32	regime	60	60	down	4	4	747%

	threshold_scope	horizon	percentile	threshold_type	n_contexts	n_rows	total_valid_c
33	regime	60	75	common	4	4	747
34	regime	60	75	up	4	4	747
35	regime	60	75	down	4	4	747
36	regime	60	90	common	4	4	747
37	regime	60	90	up	4	4	747
38	regime	60	90	down	4	4	747
39	regime	60	95	common	4	4	747

40 rows × 21 columns

5.9.2. Comparación de thresholds contextuales contra globales

```
In [79]: # 5.9.2. Comparación de thresholds contextuales contra globales
# =====
#
# Objetivo:
# Comparar cada threshold contextual contra su equivalente global.
#
# Esto permite ver si los thresholds por régimen, año, contrato, etc.
# están cerca o lejos del threshold global calculado para el mismo:
#
# - horizonte
# - percentil
# - tipo de threshold
#
# No se decide todavía si el global es válido.
# Solo se calcula la diferencia.
# =====
# -----
# Extraer thresholds globales
# -----

df_global_thresholds_long = (
    df_threshold_stability_base[
        df_threshold_stability_base["threshold_scope"] == "global"
    ]
    [
        [
            "horizon",
            "percentile",
            "threshold_type",
            "threshold_pts",
        ]
    ]
    .rename(
        columns={
            "threshold_pts": "global_threshold_pts"
        }
    )
)
```

```

    )
    .copy()
)

if df_global_thresholds_long.empty:
    raise ValueError(
        "No se encontraron thresholds globales para comparar."
    )

# -----
# Unir thresholds contextuales con su global equivalente
# -----

df_thresholds_vs_global = (
    df_context_thresholds
    .merge(
        df_global_thresholds_long,
        on=[
            "horizon",
            "percentile",
            "threshold_type",
        ],
        how="left"
    )
)

# -----
# Calcular diferencias absolutas y relativas
# -----

df_thresholds_vs_global["diff_vs_global_pts"] = (
    df_thresholds_vs_global["threshold_pts"]
    - df_thresholds_vs_global["global_threshold_pts"]
)

df_thresholds_vs_global["abs_diff_vs_global_pts"] = (
    df_thresholds_vs_global["diff_vs_global_pts"]
    .abs()
)

df_thresholds_vs_global["diff_vs_global_ratio"] = np.where(
    df_thresholds_vs_global["global_threshold_pts"] > 0,
    (
        df_thresholds_vs_global["threshold_pts"]
        / df_thresholds_vs_global["global_threshold_pts"]
    ),
    np.nan
)

df_thresholds_vs_global["diff_vs_global_pct"] = np.where(
    df_thresholds_vs_global["global_threshold_pts"] > 0,
    (
        (
            df_thresholds_vs_global["threshold_pts"]
            / df_thresholds_vs_global["global_threshold_pts"]
        )
        - 1
    )
    * 100,
    np.nan
)

```

```

)

df_thresholds_vs_global = (
    df_thresholds_vs_global
    .replace([np.inf, -np.inf], np.nan)
)

# -----
# Resumen de diferencias contra global
# -----

df_thresholds_vs_global_summary = (
    df_thresholds_vs_global
    .groupby(
        [
            "threshold_scope",
            "horizon",
            "percentile",
            "threshold_type",
        ],
        observed=True
    )
    .agg(
        n_contexts=("context_label", "nunique"),
        global_threshold_pts=("global_threshold_pts", "first"),
        median_context_threshold_pts=("threshold_pts", "median"),
        mean_context_threshold_pts=("threshold_pts", "mean"),
        min_context_threshold_pts=("threshold_pts", "min"),
        max_context_threshold_pts=("threshold_pts", "max"),
        median_diff_vs_global_pct=("diff_vs_global_pct", "median"),
        mean_abs_diff_vs_global_pct=("diff_vs_global_pct", lambda x: x.abs().mean()),
        max_abs_diff_vs_global_pct=("diff_vs_global_pct", lambda x: x.abs().max())
    )
    .reset_index()
)

df_thresholds_vs_global_summary["scope_order"] = (
    df_thresholds_vs_global_summary["threshold_scope"]
    .map(scope_order)
)

df_thresholds_vs_global_summary["threshold_type_order"] = (
    df_thresholds_vs_global_summary["threshold_type"]
    .map(type_order)
)

df_thresholds_vs_global_summary = (
    df_thresholds_vs_global_summary
    .sort_values(
        [
            "scope_order",
            "horizon",
            "percentile",
            "threshold_type_order",
        ]
    )
    .reset_index(drop=True)
)

print("Comparación de thresholds contextuales contra thresholds globales calcula

```

```
print("La tabla permite ver cuánto se aleja cada contexto del threshold global e  
print("Todavía no se concluye si el threshold global es válido o no; esta tabla  
  
print(f"\nTabla generada: df_thresholds_vs_global")  
print(f"Filas: {df_thresholds_vs_global.shape[0]:,}")  
print(f"Columnas: {df_thresholds_vs_global.shape[1]:,}")  
  
display(df_thresholds_vs_global.head(30))  
  
print("\nResumen de diferencias contra thresholds globales.")  
print("Cada fila resume las diferencias entre thresholds contextuales y globales  
  
print(f"\nTabla generada: df_thresholds_vs_global_summary")  
print(f"Filas: {df_thresholds_vs_global_summary.shape[0]:,}")  
print(f"Columnas: {df_thresholds_vs_global_summary.shape[1]:,}")  
  
display(df_thresholds_vs_global_summary.head(40))
```

Comparación de thresholds contextuales contra thresholds globales calculada.

La tabla permite ver cuánto se aleja cada contexto del threshold global equivalente.

Todavía no se concluye si el threshold global es válido o no; esta tabla es insumo para esa decisión.

Tabla generada: df_thresholds_vs_global
Filas: 13,923
Columnas: 23

	threshold_scope	threshold_scope_order	selection_period	horizon	percentile
0	regime	1	development_2020_2024	30	25
1	regime	1	development_2020_2024	30	40
2	regime	1	development_2020_2024	30	50
3	regime	1	development_2020_2024	30	60
4	regime	1	development_2020_2024	30	75
5	regime	1	development_2020_2024	30	90
6	regime	1	development_2020_2024	30	95
7	regime	1	development_2020_2024	30	25
8	regime	1	development_2020_2024	30	40
9	regime	1	development_2020_2024	30	50
10	regime	1	development_2020_2024	30	60
11	regime	1	development_2020_2024	30	75
12	regime	1	development_2020_2024	30	90
13	regime	1	development_2020_2024	30	95
14	regime	1	development_2020_2024	30	25
15	regime	1	development_2020_2024	30	40
16	regime	1	development_2020_2024	30	50
17	regime	1	development_2020_2024	30	60
18	regime	1	development_2020_2024	30	75
19	regime	1	development_2020_2024	30	90
20	regime	1	development_2020_2024	30	95
21	regime	1	development_2020_2024	30	25
22	regime	1	development_2020_2024	30	40
23	regime	1	development_2020_2024	30	50
24	regime	1	development_2020_2024	30	60
25	regime	1	development_2020_2024	30	75
26	regime	1	development_2020_2024	30	90
27	regime	1	development_2020_2024	30	95
28	regime	1	development_2020_2024	30	25
29	regime	1	development_2020_2024	30	40

30 rows × 23 columns



Resumen de diferencias contra thresholds globales.

Cada fila resume las diferencias entre thresholds contextuales y globales para una combinación de horizonte, percentil y tipo.

Tabla generada: df_thresholds_vs_global_summary

Filas: 315

Columnas: 15

	threshold_scope	horizon	percentile	threshold_type	n_contexts	global_threshold_pts
0	regime	30	25	common	5	15.25
1	regime	30	25	up	5	4.75
2	regime	30	25	down	5	4.00
3	regime	30	40	common	5	21.00
4	regime	30	40	up	5	9.25
5	regime	30	40	down	5	8.50
6	regime	30	50	common	5	25.50
7	regime	30	50	up	5	13.00
8	regime	30	50	down	5	12.25
9	regime	30	60	common	5	31.00
10	regime	30	60	up	5	17.25
11	regime	30	60	down	5	17.00
12	regime	30	75	common	5	42.75
13	regime	30	75	up	5	26.75
14	regime	30	75	down	5	27.50
15	regime	30	90	common	5	67.75
16	regime	30	90	up	5	47.25
17	regime	30	90	down	5	51.25
18	regime	30	95	common	5	88.25
19	regime	30	95	up	5	64.75
20	regime	30	95	down	5	71.25
21	regime	60	25	common	4	23.25
22	regime	60	25	up	4	7.75
23	regime	60	25	down	4	6.75
24	regime	60	40	common	4	31.75
25	regime	60	40	up	4	14.50
26	regime	60	40	down	4	13.25
27	regime	60	50	common	4	38.50
28	regime	60	50	up	4	19.75
29	regime	60	50	down	4	18.75
30	regime	60	60	common	4	46.50
31	regime	60	60	up	4	26.25
32	regime	60	60	down	4	25.75

	threshold_scope	horizon	percentile	threshold_type	n_contexts	global_threshold_pts
33	regime	60	75	common	4	63.75
34	regime	60	75	up	4	40.50
35	regime	60	75	down	4	41.75
36	regime	60	90	common	4	99.50
37	regime	60	90	up	4	69.75
38	regime	60	90	down	4	76.50
39	regime	60	95	common	4	128.25

5.9.3. Identificación de contextos extremos

```
In [80]: # 5.9.3. Identificación de contextos extremos
# =====
#
# Objetivo:
# Identificar, para cada combinación de:
#
# - tipo de contexto
# - horizonte
# - percentil
# - tipo de threshold
#
# cuál fue el contexto con threshold más bajo y cuál fue el contexto
# con threshold más alto.
#
# Esto ayuda a ver qué contextos explican las diferencias extremas.
# =====

extreme_context_rows = []

group_cols = [
    "threshold_scope",
    "horizon",
    "percentile",
    "threshold_type",
]

for keys, group in df_context_thresholds.groupby(group_cols, observed=True):

    threshold_scope, horizon, percentile, threshold_type = keys

    group_valid = group.dropna(subset=["threshold_pts"]).copy()

    if group_valid.empty:
        continue

    low_row = group_valid.loc[group_valid["threshold_pts"].idxmin()]
    high_row = group_valid.loc[group_valid["threshold_pts"].idxmax()]

    extreme_context_rows.append({
        "threshold_scope": threshold_scope,
```

```

        "horizon": horizon,
        "percentile": percentile,
        "threshold_type": threshold_type,

        "low_context": low_row["context_label"],
        "low_threshold_pts": float(low_row["threshold_pts"]),
        "low_n_valid": int(low_row["n_valid"]),

        "high_context": high_row["context_label"],
        "high_threshold_pts": float(high_row["threshold_pts"]),
        "high_n_valid": int(high_row["n_valid"]),

        "top_to_low_ratio": (
            float(high_row["threshold_pts"] / low_row["threshold_pts"])
            if low_row["threshold_pts"] > 0
            else np.nan
        ),

        "range_threshold_pts": float(
            high_row["threshold_pts"] - low_row["threshold_pts"]
        ),
    })

df_thresholds_extreme_contexts = pd.DataFrame(extreme_context_rows)

df_thresholds_extreme_contexts["scope_order"] = (
    df_thresholds_extreme_contexts["threshold_scope"]
    .map(scope_order)
)

df_thresholds_extreme_contexts["threshold_type_order"] = (
    df_thresholds_extreme_contexts["threshold_type"]
    .map(type_order)
)

df_thresholds_extreme_contexts = (
    df_thresholds_extreme_contexts
    .sort_values(
        [
            "scope_order",
            "horizon",
            "percentile",
            "threshold_type_order",
        ]
    )
    .reset_index(drop=True)
)

print("Contextos extremos identificados.")
print("La tabla muestra, para cada combinación analizada, qué contexto tuvo el t
print("Este resultado ayudará a interpretar las diferencias en la sección de con

print(f"\nTabla generada: df_thresholds_extreme_contexts")
print(f"Filas: {df_thresholds_extreme_contexts.shape[0]:,}")
print(f"Columnas: {df_thresholds_extreme_contexts.shape[1]:,}")

display(df_thresholds_extreme_contexts.head(40))

```

Contextos extremos identificados.

La tabla muestra, para cada combinación analizada, qué contexto tuvo el threshold más bajo y cuál tuvo el threshold más alto.

Este resultado ayudará a interpretar las diferencias en la sección de conclusiones.

Tabla generada: df_thresholds_extreme_contexts

Filas: 315

Columnas: 14

	threshold_scope	horizon	percentile	threshold_type	low_context	low_threshold_pts
0	regime	30	25	common	Overnight	10.50
1	regime	30	25	up	Overnight	3.25
2	regime	30	25	down	Overnight	2.75
3	regime	30	40	common	Overnight	14.00
4	regime	30	40	up	Overnight	6.25
5	regime	30	40	down	Overnight	5.75
6	regime	30	50	common	Overnight	16.50
7	regime	30	50	up	Overnight	8.50
8	regime	30	50	down	Overnight	8.25
9	regime	30	60	common	Overnight	19.25
10	regime	30	60	up	Overnight	11.25
11	regime	30	60	down	Overnight	11.00
12	regime	30	75	common	Overnight	25.75
13	regime	30	75	up	Overnight	16.75
14	regime	30	75	down	Overnight	17.25
15	regime	30	90	common	Overnight	39.25
16	regime	30	90	up	Overnight	28.25
17	regime	30	90	down	Overnight	30.25
18	regime	30	95	common	Overnight	51.25
19	regime	30	95	up	Overnight	38.00
20	regime	30	95	down	Overnight	41.25
21	regime	60	25	common	Overnight	16.00
22	regime	60	25	up	Overnight	5.25
23	regime	60	25	down	Overnight	4.75
24	regime	60	40	common	Overnight	21.00
25	regime	60	40	up	Overnight	9.75
26	regime	60	40	down	Overnight	9.25
27	regime	60	50	common	Overnight	24.75
28	regime	60	50	up	Overnight	13.25
29	regime	60	50	down	Overnight	12.75
30	regime	60	60	common	Overnight	29.25
31	regime	60	60	up	Overnight	17.00
32	regime	60	60	down	Overnight	17.00

	threshold_scope	horizon	percentile	threshold_type	low_context	low_threshold_pts
33	regime	60	75	common	Overnight	38.75
34	regime	60	75	up	Overnight	25.50
35	regime	60	75	down	Overnight	26.00
36	regime	60	90	common	Overnight	59.25
37	regime	60	90	up	Overnight	42.50
38	regime	60	90	down	Overnight	45.50
39	regime	60	95	common	Overnight	78.25

5.9.4. Resumen compacto para candidatos principales

```
In [81]: # 5.9.4. Resumen compacto para candidatos principales
# =====
#
# Objetivo:
# Crear una tabla compacta con los percentiles principales:
#
# p40, p50, p60
#
# y priorizando el threshold_type = common.
#
# Esta tabla facilita revisar la estabilidad de los candidatos
# que probablemente se usen primero como thresholds operativos.
# =====

df_thresholds_stability_main = (
    df_thresholds_stability[
        df_thresholds_stability["percentile"].isin(THRESHOLD_PERCENTILES_MAIN)
    ]
    .copy()
)

df_thresholds_stability_main_common = (
    df_thresholds_stability_main[
        df_thresholds_stability_main["threshold_type"] == "common"
    ]
    .copy()
    .reset_index(drop=True)
)

compact_cols = [
    "threshold_scope",
    "horizon",
    "percentile",
    "threshold_type",
    "n_contexts",
    "min_threshold_pts",
    "median_threshold_pts",
    "max_threshold_pts",
    "top_to_low_ratio",
    "cv_threshold",
]
```

```
    "stability_label",
]

df_thresholds_stability_compact = (
    df_thresholds_stability_main_common[compact_cols]
    .sort_values(
        [
            "threshold_scope",
            "horizon",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

print("Resumen compacto de estabilidad para percentiles candidatos principales.")
print("Esta tabla se enfoca en p40, p50 y p60 usando threshold_common_pts.")
print("Se usará como una de las tablas principales para decidir si conviene usar")

print(f"\nTabla generada: df_thresholds_stability_compact")
print(f"Filas: {df_thresholds_stability_compact.shape[0]:,}")
print(f"Columnas: {df_thresholds_stability_compact.shape[1]:,}")

display(df_thresholds_stability_compact)
```

Resumen compacto de estabilidad para percentiles candidatos principales.

Esta tabla se enfoca en p40, p50 y p60 usando threshold_common_pts.

Se usará como una de las tablas principales para decidir si conviene usar thresholds globales o segmentados.

Tabla generada: df_thresholds_stability_compact

Filas: 45

Columnas: 11

	threshold_scope	horizon	percentile	threshold_type	n_contexts	min_threshold_
0	regime	30	40	common	5	14
1	regime	30	50	common	5	16
2	regime	30	60	common	5	19
3	regime	60	40	common	4	21
4	regime	60	50	common	4	24
5	regime	60	60	common	4	29
6	regime	90	40	common	4	28
7	regime	90	50	common	4	33
8	regime	90	60	common	4	39
9	regime_contract	30	40	common	105	8
10	regime_contract	30	50	common	105	10
11	regime_contract	30	60	common	105	12
12	regime_contract	60	40	common	84	12
13	regime_contract	60	50	common	84	15
14	regime_contract	60	60	common	84	19
15	regime_contract	90	40	common	84	16
16	regime_contract	90	50	common	84	21
17	regime_contract	90	60	common	84	26
18	regime_contract_family	30	40	common	20	13
19	regime_contract_family	30	50	common	20	15
20	regime_contract_family	30	60	common	20	17
21	regime_contract_family	60	40	common	16	19
22	regime_contract_family	60	50	common	16	23
23	regime_contract_family	60	60	common	16	26
24	regime_contract_family	90	40	common	16	26
25	regime_contract_family	90	50	common	16	31
26	regime_contract_family	90	60	common	16	36
27	regime_year	30	40	common	25	11
28	regime_year	30	50	common	25	13
29	regime_year	30	60	common	25	16
30	regime_year	60	40	common	20	17
31	regime_year	60	50	common	20	20
32	regime_year	60	60	common	20	24

	threshold_scope	horizon	percentile	threshold_type	n_contexts	min_threshold_
33	regime_year	90	40	common	20	23
34	regime_year	90	50	common	20	28
35	regime_year	90	60	common	20	33
36	regime_year_quarter	30	40	common	100	9
37	regime_year_quarter	30	50	common	100	11
38	regime_year_quarter	30	60	common	100	13
39	regime_year_quarter	60	40	common	80	13
40	regime_year_quarter	60	50	common	80	17
41	regime_year_quarter	60	60	common	80	21
42	regime_year_quarter	90	40	common	80	18
43	regime_year_quarter	90	50	common	80	24
44	regime_year_quarter	90	60	common	80	29

5.9.5. Ranking de mayor inestabilidad

```
In [82]: # 5.9.5. Ranking de mayor inestabilidad
# =====
#
# Objetivo:
# Identificar las combinaciones donde los thresholds muestran
# mayor variación entre contextos.
#
# Esto no implica todavía una decisión final.
# Solo ayuda a detectar qué análisis requieren más atención.
# =====

df_thresholds_most_unstable = (
    df_thresholds_stability
    .copy()
    .sort_values(
        [
            "top_to_low_ratio",
            "cv_threshold",
        ],
        ascending=[
            False,
            False,
        ],
        na_position="last"
    )
    .reset_index(drop=True)
)

print("Ranking de combinaciones con mayor variación de thresholds entre contexto")
print("Esta tabla ayuda a identificar dónde los thresholds son más inestables.")
print("La interpretación final se realizará después de revisar los resultados co
```

```
display(  
    df_thresholds_most_unstable[  
        [  
            "threshold_scope",  
            "horizon",  
            "percentile",  
            "threshold_type",  
            "n_contexts",  
            "min_threshold_pts",  
            "median_threshold_pts",  
            "max_threshold_pts",  
            "top_to_low_ratio",  
            "cv_threshold",  
            "stability_label",  
        ]  
    ].head(30)  
)
```

Ranking de combinaciones con mayor variación de thresholds entre contextos.
Esta tabla ayuda a identificar dónde los thresholds son más inestables.
La interpretación final se realizará después de revisar los resultados completos.

	threshold_scope	horizon	percentile	threshold_type	n_contexts	min_threshold_pts
0	regime_contract	60	25	down	84	2.7500
1	regime_contract	30	25	down	105	1.5000
2	regime_contract	90	25	down	84	3.7500
3	regime_contract	90	40	down	84	7.2500
4	regime_contract	60	40	down	84	5.2500
5	regime_contract	30	40	down	105	3.5000
6	regime_contract	30	50	down	105	5.0000
7	regime_contract	90	50	down	84	10.7500
8	regime_contract	60	50	down	84	7.7500
9	regime_contract	60	60	down	84	10.7500
10	regime_contract	30	60	down	105	7.0000
11	regime_year_quarter	30	25	down	100	1.7500
12	regime_year_quarter	90	25	down	80	4.0000
13	regime_contract	30	25	common	105	6.0000
14	regime_year_quarter	60	25	down	80	3.0000
15	regime_contract	90	60	down	84	14.7500
16	regime_contract	90	90	down	84	39.5000
17	regime_year_quarter	30	40	down	100	4.0000
18	regime_contract	30	75	down	105	10.7500
19	regime_contract	60	90	down	84	30.0000
20	regime_year_quarter	30	25	common	100	6.5000
21	regime_contract	90	75	down	84	24.0000
22	regime_contract	90	25	common	84	12.0000
23	regime_contract	90	25	up	84	3.4375
24	regime_contract	60	95	down	84	37.2500
25	regime_year_quarter	90	25	common	80	12.7500
26	regime_contract	60	75	down	84	16.5000
27	regime_contract	60	25	common	84	9.0000
28	regime_year_quarter	30	50	down	100	5.7500
29	regime_contract	30	40	common	105	8.5000

5.9.6. Estabilidad año a año de thresholds por régimen

```

In [83]: # 5.9.6. Estabilidad año a año de thresholds por régimen
# =====
#
# Objetivo:
# Analizar cómo se comportan los thresholds por régimen año a año.
#
# Pregunta central:
# Si definimos thresholds por regime_id, ¿son estables entre 2020,
# 2021, 2022, 2023 y 2024?
#
# En este bloque:
# - No se construyen targets.
# - No se selecciona todavía un threshold definitivo.
# - Se analiza la estabilidad temporal de los thresholds por régimen.
#
# Regla metodológica:
# Solo se analiza development_2020_2024.
# =====

# -----
# Validación de tabla consolidada
# -----

required_cols = [
    "threshold_scope",
    "horizon",
    "percentile",
    "threshold_type",
    "threshold_pts",
    "year",
    "regime_id",
    "regime_name",
    "n_valid",
    "candidate_for_threshold",
]

missing_cols = [
    col for col in required_cols
    if col not in df_thresholds_all_long.columns
]

if missing_cols:
    raise ValueError(
        "Faltan columnas necesarias en df_thresholds_all_long: "
        f"{missing_cols}"
    )

# -----
# Base: thresholds por régimen y año
# -----

df_regime_year_thresholds = (
    df_thresholds_all_long[
        df_thresholds_all_long["threshold_scope"] == "regime_year"
    ]
    .copy()
    .dropna(subset=["threshold_pts", "year", "regime_id"])
)

```

```
df_regime_year_thresholds["year"] = (  
    df_regime_year_thresholds["year"]  
    .astype(int)  
)  
  
df_regime_year_thresholds["threshold_pts"] = pd.to_numeric(  
    df_regime_year_thresholds["threshold_pts"],  
    errors="coerce"  
)  
  
df_regime_year_thresholds["n_valid"] = pd.to_numeric(  
    df_regime_year_thresholds["n_valid"],  
    errors="coerce"  
)  
  
df_regime_year_thresholds = (  
    df_regime_year_thresholds  
    .dropna(subset=["threshold_pts"])  
    .reset_index(drop=True)  
)  
  
print("Base de thresholds por régimen y año preparada.")  
print("Cada fila representa un threshold calculado para una combinación de régimen,  
print("No se selecciona todavía ningún threshold definitivo.")  
  
print(f"\nFilas disponibles: {df_regime_year_thresholds.shape[0]:,}")  
print(f"Columnas disponibles: {df_regime_year_thresholds.shape[1]:,}")  
  
display(df_regime_year_thresholds.head(30))
```

Base de thresholds por régimen y año preparada.

Cada fila representa un threshold calculado para una combinación de régimen, año, horizonte, percentil y tipo.

No se selecciona todavía ningún threshold definitivo.

Filas disponibles: 1,365

Columnas disponibles: 18

	threshold_scope	threshold_scope_order	selection_period	horizon	percentile	
0	regime_year	2	development_2020_2024	30	25	
1	regime_year	2	development_2020_2024	30	40	
2	regime_year	2	development_2020_2024	30	50	
3	regime_year	2	development_2020_2024	30	60	
4	regime_year	2	development_2020_2024	30	75	
5	regime_year	2	development_2020_2024	30	90	
6	regime_year	2	development_2020_2024	30	95	
7	regime_year	2	development_2020_2024	30	25	
8	regime_year	2	development_2020_2024	30	40	
9	regime_year	2	development_2020_2024	30	50	
10	regime_year	2	development_2020_2024	30	60	
11	regime_year	2	development_2020_2024	30	75	
12	regime_year	2	development_2020_2024	30	90	
13	regime_year	2	development_2020_2024	30	95	
14	regime_year	2	development_2020_2024	30	25	
15	regime_year	2	development_2020_2024	30	40	
16	regime_year	2	development_2020_2024	30	50	
17	regime_year	2	development_2020_2024	30	60	
18	regime_year	2	development_2020_2024	30	75	
19	regime_year	2	development_2020_2024	30	90	
20	regime_year	2	development_2020_2024	30	95	

	threshold_scope	threshold_scope_order	selection_period	horizon	percentile
21	regime_year	2	development_2020_2024	30	25
22	regime_year	2	development_2020_2024	30	40
23	regime_year	2	development_2020_2024	30	50
24	regime_year	2	development_2020_2024	30	60
25	regime_year	2	development_2020_2024	30	75
26	regime_year	2	development_2020_2024	30	90
27	regime_year	2	development_2020_2024	30	95
28	regime_year	2	development_2020_2024	30	25
29	regime_year	2	development_2020_2024	30	40

5.9.7. Resumen de estabilidad año a año por régimen

```
In [84]: # 5.9.7. Resumen de estabilidad año a año por régimen
# =====

df_regime_year_stability = (
    df_regime_year_thresholds
    .groupby(
        [
            "horizon",
            "percentile",
            "threshold_type",
            "regime_id",
            "regime_name",
        ],
        observed=True
    )
    .agg(
        n_years=("year", "nunique"),
        n_rows=("threshold_pts", "size"),
        total_valid_obs=("n_valid", "sum"),
        min_threshold_pts=("threshold_pts", "min"),
        median_threshold_pts=("threshold_pts", "median"),
        mean_threshold_pts=("threshold_pts", "mean"),
        max_threshold_pts=("threshold_pts", "max"),
        std_threshold_pts=("threshold_pts", lambda x: x.std(ddof=0)),
        cv_threshold=("threshold_pts", safe_cv),
    )
    .reset_index()
)
```

```

df_regime_year_stability["top_to_low_ratio"] = np.where(
    df_regime_year_stability["min_threshold_pts"] > 0,
    (
        df_regime_year_stability["max_threshold_pts"]
        / df_regime_year_stability["min_threshold_pts"]
    ),
    np.nan
)

df_regime_year_stability["range_threshold_pts"] = (
    df_regime_year_stability["max_threshold_pts"]
    - df_regime_year_stability["min_threshold_pts"]
)

# -----
# Clasificación diagn stica de estabilidad a o a a o
# -----

def classify_yearly_stability(row):
    cv = row["cv_threshold"]
    ratio = row["top_to_low_ratio"]

    if pd.isna(cv) or pd.isna(ratio):
        return "revisar"

    if cv <= 0.15 and ratio <= 1.50:
        return "estable"

    if cv <= 0.30 and ratio <= 2.25:
        return "moderada"

    return "inestable"

df_regime_year_stability["yearly_stability_label"] = (
    df_regime_year_stability
    .apply(classify_yearly_stability, axis=1)
)

df_regime_year_stability["threshold_type_order"] = (
    df_regime_year_stability["threshold_type"]
    .map(type_order)
)

df_regime_year_stability = (
    df_regime_year_stability
    .sort_values(
        [
            "horizon",
            "regime_id",
            "percentile",
            "threshold_type_order",
        ]
    )
    .reset_index(drop=True)
)

print("Estabilidad a o a a o de thresholds por r gimen calculada.")
print("Cada fila resume c mo var a un threshold dentro de un mismo r gimen entre

```

```
print(f"\nTabla generada: df_regime_year_stability")
print(f"Filas: {df_regime_year_stability.shape[0]:,}")
print(f"Columnas: {df_regime_year_stability.shape[1]:,}")

display(df_regime_year_stability.head(40))
```

Estabilidad año a año de thresholds por régimen calculada.
Cada fila resume cómo varía un threshold dentro de un mismo régimen entre los años 2020-2024.

Tabla generada: df_regime_year_stability
Filas: 273
Columnas: 18

	horizon	percentile	threshold_type	regime_id	regime_name	n_years	n_rows	total_
0	30	25	common	0.0	Overnight	5	5	
1	30	25	up	0.0	Overnight	5	5	
2	30	25	down	0.0	Overnight	5	5	
3	30	40	common	0.0	Overnight	5	5	
4	30	40	up	0.0	Overnight	5	5	
5	30	40	down	0.0	Overnight	5	5	
6	30	50	common	0.0	Overnight	5	5	
7	30	50	up	0.0	Overnight	5	5	
8	30	50	down	0.0	Overnight	5	5	
9	30	60	common	0.0	Overnight	5	5	
10	30	60	up	0.0	Overnight	5	5	
11	30	60	down	0.0	Overnight	5	5	
12	30	75	common	0.0	Overnight	5	5	
13	30	75	up	0.0	Overnight	5	5	
14	30	75	down	0.0	Overnight	5	5	
15	30	90	common	0.0	Overnight	5	5	
16	30	90	up	0.0	Overnight	5	5	
17	30	90	down	0.0	Overnight	5	5	
18	30	95	common	0.0	Overnight	5	5	
19	30	95	up	0.0	Overnight	5	5	
20	30	95	down	0.0	Overnight	5	5	
21	30	25	common	1.0	Pre-market	5	5	
22	30	25	up	1.0	Pre-market	5	5	
23	30	25	down	1.0	Pre-market	5	5	
24	30	40	common	1.0	Pre-market	5	5	
25	30	40	up	1.0	Pre-market	5	5	
26	30	40	down	1.0	Pre-market	5	5	
27	30	50	common	1.0	Pre-market	5	5	
28	30	50	up	1.0	Pre-market	5	5	
29	30	50	down	1.0	Pre-market	5	5	
30	30	60	common	1.0	Pre-market	5	5	
31	30	60	up	1.0	Pre-market	5	5	
32	30	60	down	1.0	Pre-market	5	5	

	horizon	percentile	threshold_type	regime_id	regime_name	n_years	n_rows	total_
33	30	75	common	1.0	Pre-market	5	5	
34	30	75	up	1.0	Pre-market	5	5	
35	30	75	down	1.0	Pre-market	5	5	
36	30	90	common	1.0	Pre-market	5	5	
37	30	90	up	1.0	Pre-market	5	5	
38	30	90	down	1.0	Pre-market	5	5	
39	30	95	common	1.0	Pre-market	5	5	

5.9.8. Vista compacta año a año por régimen

```
In [85]: # 5.9.8. Vista compacta año a año por régimen
# =====
#
# Objetivo:
# Enfocar la revisión en:
#
# - threshold_type = common
# - percentiles principales p40, p50, p60
#
# Esta tabla será clave para decidir si thresholds por regime_id
# son razonablemente estables año a año.
# =====

df_regime_year_stability_compact = (
    df_regime_year_stability[
        (df_regime_year_stability["threshold_type"] == "common")
        & (df_regime_year_stability["percentile"].isin(THRESHOLD_PERCENTILES_MAI
    ]
    [
        [
            "horizon",
            "percentile",
            "regime_id",
            "regime_name",
            "n_years",
            "min_threshold_pts",
            "median_threshold_pts",
            "max_threshold_pts",
            "top_to_low_ratio",
            "cv_threshold",
            "yearly_stability_label",
        ]
    ]
    .copy()
    .sort_values(
        [
            "horizon",
            "regime_id",
            "percentile",
        ]
    )
)
```

```
)  
    .reset_index(drop=True)  
)  
  
print("Resumen compacto de estabilidad año a año por régimen.")  
print("Se muestran solo thresholds common para percentiles candidatos p40, p50 y  
  
display(df_regime_year_stability_compact)
```

Resumen compacto de estabilidad año a año por régimen.

Se muestran solo thresholds common para percentiles candidatos p40, p50 y p60.

	horizon	percentile	regime_id	regime_name	n_years	min_threshold_pts	median_thr
0	30	40	0.0	Overnight	5	11.500	
1	30	50	0.0	Overnight	5	13.750	
2	30	60	0.0	Overnight	5	16.500	
3	30	40	1.0	Pre-market	5	20.000	
4	30	50	1.0	Pre-market	5	25.250	
5	30	60	1.0	Pre-market	5	31.750	
6	30	40	2.0	Opening	5	33.650	
7	30	50	2.0	Opening	5	39.500	
8	30	60	2.0	Opening	5	46.500	
9	30	40	3.0	Regular	5	20.500	
10	30	50	3.0	Regular	5	24.250	
11	30	60	3.0	Regular	5	29.000	
12	30	40	4.0	Closing	5	25.000	
13	30	50	4.0	Closing	5	30.000	
14	30	60	4.0	Closing	5	34.500	
15	60	40	0.0	Overnight	5	17.500	
16	60	50	0.0	Overnight	5	20.750	
17	60	60	0.0	Overnight	5	24.750	
18	60	40	1.0	Pre-market	5	44.000	
19	60	50	1.0	Pre-market	5	52.000	
20	60	60	1.0	Pre-market	5	60.750	
21	60	40	2.0	Opening	5	46.500	
22	60	50	2.0	Opening	5	54.375	
23	60	60	2.0	Opening	5	63.250	
24	60	40	3.0	Regular	5	29.750	
25	60	50	3.0	Regular	5	35.250	
26	60	60	3.0	Regular	5	42.250	
27	90	40	0.0	Overnight	5	23.750	
28	90	50	0.0	Overnight	5	28.250	
29	90	60	0.0	Overnight	5	33.500	
30	90	40	1.0	Pre-market	5	59.750	
31	90	50	1.0	Pre-market	5	69.000	
32	90	60	1.0	Pre-market	5	78.500	

	horizon	percentile	regime_id	regime_name	n_years	min_threshold_pts	median_thr
33	90	40	2.0	Opening	5	55.250	
34	90	50	2.0	Opening	5	64.500	
35	90	60	2.0	Opening	5	75.500	
36	90	40	3.0	Regular	5	36.500	
37	90	50	3.0	Regular	5	43.500	
38	90	60	3.0	Regular	5	52.000	

5.9.9. Matriz de thresholds por régimen y año

```
In [86]: # 5.9.9. Matriz de thresholds por régimen y año
# =====
#
# Objetivo:
# Mostrar Los thresholds de cada año lado a lado.
#
# Esta tabla permite detectar visualmente años extremos dentro de
# cada régimen, horizonte y percentil.
# =====

df_regime_year_threshold_matrix = (
    df_regime_year_thresholds[
        (df_regime_year_thresholds["threshold_type"] == "common")
        & (df_regime_year_thresholds["percentile"].isin(THRESHOLD_PERCENTILES_MA
    )
    .pivot_table(
        index=[
            "horizon",
            "percentile",
            "regime_id",
            "regime_name",
        ],
        columns="year",
        values="threshold_pts",
        aggfunc="median",
        observed=True
    )
    .reset_index()
)

df_regime_year_threshold_matrix.columns.name = None

print("Matriz de thresholds common por régimen y año.")
print("Cada fila permite comparar 2020-2024 para un mismo horizonte, percentil y régimen.")

display(df_regime_year_threshold_matrix)
```

Matriz de thresholds common por régimen y año.
Cada fila permite comparar 2020-2024 para un mismo horizonte, percentil y régimen.

	horizon	percentile	regime_id	regime_name	2020	2021	2022	2023	2024
0	30	40	0.0	Overnight	12.500	11.50	19.500	12.50	14.50
1	30	40	1.0	Pre-market	20.000	22.25	36.750	23.50	27.50
2	30	40	2.0	Opening	33.650	36.75	60.750	41.25	46.50
3	30	40	3.0	Regular	21.500	20.50	36.250	24.25	27.50
4	30	40	4.0	Closing	27.650	25.00	43.400	27.00	32.05
5	30	50	0.0	Overnight	15.000	13.75	22.750	14.50	16.75
6	30	50	1.0	Pre-market	25.250	27.00	45.250	28.25	33.25
7	30	50	2.0	Opening	39.500	42.50	70.000	47.25	54.00
8	30	50	3.0	Regular	25.750	24.25	41.750	28.00	32.00
9	30	50	4.0	Closing	31.250	30.00	48.625	31.00	37.50
10	30	60	0.0	Overnight	18.000	16.50	26.500	16.75	19.50
11	30	60	1.0	Pre-market	31.750	33.25	55.250	33.75	39.25
12	30	60	2.0	Opening	46.500	49.75	79.750	54.50	62.50
13	30	60	3.0	Regular	30.750	29.00	48.500	32.25	37.25
14	30	60	4.0	Closing	39.650	35.70	58.600	34.50	42.50
15	60	40	0.0	Overnight	18.500	17.50	29.750	19.00	21.75
16	60	40	1.0	Pre-market	44.000	48.00	77.500	49.75	56.50
17	60	40	2.0	Opening	46.500	49.25	83.000	56.25	64.50
18	60	40	3.0	Regular	31.000	29.75	52.250	35.75	40.50
19	60	50	0.0	Overnight	22.250	20.75	34.500	21.75	25.50
20	60	50	1.0	Pre-market	52.000	56.25	88.750	57.00	67.00
21	60	50	2.0	Opening	54.375	58.00	95.000	63.75	75.00
22	60	50	3.0	Regular	37.000	35.25	60.250	41.00	47.25
23	60	60	0.0	Overnight	26.750	24.75	40.000	25.25	29.50
24	60	60	1.0	Pre-market	60.750	66.25	100.750	65.25	79.00
25	60	60	2.0	Opening	63.250	67.75	109.500	73.25	86.50
26	60	60	3.0	Regular	44.000	42.25	70.250	47.00	54.75
27	90	40	0.0	Overnight	25.000	23.75	39.500	24.75	29.25
28	90	40	1.0	Pre-market	59.750	65.50	106.750	67.50	79.15
29	90	40	2.0	Opening	55.250	59.00	98.000	65.00	74.25
30	90	40	3.0	Regular	38.750	36.50	64.750	44.75	50.50
31	90	50	0.0	Overnight	29.750	28.25	45.750	29.25	34.25
32	90	50	1.0	Pre-market	69.000	75.50	120.750	78.00	93.50

	horizon	percentile	regime_id	regime_name	2020	2021	2022	2023	2024
33	90	50	2.0	Opening	64.500	69.50	111.750	74.50	86.50
34	90	50	3.0	Regular	45.500	43.50	74.750	51.00	58.75
35	90	60	0.0	Overnight	35.500	33.50	54.250	34.25	40.00
36	90	60	1.0	Pre-market	78.500	86.50	135.250	88.25	108.75
37	90	60	2.0	Opening	75.500	81.00	127.750	85.10	100.50
38	90	60	3.0	Regular	53.750	52.00	87.250	58.25	68.50

5.10. Observaciones y conclusiones del diagnóstico de thresholds

El análisis comparativo muestra que las magnitudes de movimiento futuro no son homogéneas en todo el dataset. Los thresholds cambian de manera importante según el contexto analizado, especialmente cuando se comparan distintos regímenes intradiarios.

En primer lugar, los thresholds globales por horizonte funcionan como una referencia inicial, pero no representan correctamente todos los contextos del mercado. Al comparar los thresholds por régimen contra los valores globales, se observa que algunos regímenes quedan sistemáticamente por debajo del threshold global, mientras que otros quedan muy por encima.

El caso más claro es la comparación entre **Overnight** y **Opening**. En prácticamente todos los horizontes y percentiles analizados, **Overnight** presenta los thresholds más bajos, mientras que **Opening** presenta los thresholds más altos. Por ejemplo, para thresholds **common**, en el horizonte de **30m**, los ratios **top / low** entre regímenes se ubican alrededor de **3x** para los percentiles principales **p40**, **p50** y **p60**.

Esto indica que un mismo movimiento no tiene el mismo significado operativo en todos los regímenes. Un movimiento que puede ser relevante durante **Overnight** puede ser normal durante **Opening**. Por lo tanto, utilizar un threshold global podría generar una definición poco equilibrada: demasiado exigente para regímenes tranquilos y demasiado flexible para regímenes más volátiles.

El análisis por régimen confirma que el mercado intradiario tiene una estructura diferenciada. La jerarquía observada es consistente:

Overnight → thresholds más bajos
 Regular → nivel intermedio
 Pre-market → nivel alto
 Opening → nivel alto / muy alto
 Closing → válido principalmente para horizontes cortos

En horizontes de **60m** y **90m**, el régimen **Closing** pierde relevancia porque no dispone de suficiente ventana futura sin cruzar el cierre diario. Esto es coherente con la metodología definida: si no existen suficientes barras futuras dentro del mismo día, la observación no debe usarse para calcular excursiones ni thresholds de ese horizonte.

Los análisis más segmentados, como `regime_contract`, `regime_year_quarter` y `regime_contract_family`, muestran niveles de inestabilidad mucho mayores. En particular, el cruce `regime_contract` presenta ratios `top / low` muy elevados, llegando a valores superiores a `7x` en thresholds `common` y aún mayores en thresholds direccionales, especialmente en `down`. Esto sugiere que segmentar por contrato introduce demasiada variabilidad y puede fragmentar excesivamente la muestra.

Por este motivo, los thresholds por contrato, trimestre o familia de contrato deben interpretarse como análisis de diagnóstico y sensibilidad, no como una metodología principal para definir thresholds operativos.

Un punto importante surge al analizar la estabilidad año a año dentro de cada régimen. Aunque el análisis global de `regime_year` aparece como inestable al mezclar todos los regímenes y años, cuando se observa cada régimen por separado la variación año a año se vuelve moderada. En la tabla compacta de estabilidad por régimen y año, los thresholds `common` para `p40`, `p50` y `p60` presentan una etiqueta de estabilidad principalmente `moderada`.

Esto significa que el régimen intradiario explica una parte importante de la variabilidad. Una vez separado el mercado por `regime_id`, los thresholds mantienen una estructura razonablemente estable entre `2020` y `2024`.

La matriz año a año muestra además que `2022` fue el año con thresholds más altos en casi todos los regímenes, horizontes y percentiles. Esto sugiere que `2022` fue un período de mayor amplitud o volatilidad relativa. Sin embargo, este comportamiento no invalida la segmentación por régimen, ya que la jerarquía entre regímenes se mantiene: `Overnight` continúa siendo el régimen de menor magnitud y `Opening / Pre-market` continúan mostrando thresholds superiores.

Por ejemplo, usando thresholds `common` y percentil `p50`, las medianas por régimen muestran diferencias claras:

H30:

Overnight	≈ 15.00 puntos
Pre-market	≈ 28.25 puntos
Opening	≈ 47.25 puntos
Regular	≈ 28.00 puntos
Closing	≈ 31.25 puntos

H60:

Overnight	≈ 22.25 puntos
Pre-market	≈ 57.00 puntos
Opening	≈ 63.75 puntos
Regular	≈ 41.00 puntos

H90:

Overnight	≈ 29.75 puntos
Pre-market	≈ 78.00 puntos
Opening	≈ 74.50 puntos
Regular	≈ 51.00 puntos

Estos valores confirman que el uso de thresholds por `regime_id` es más cercano al comportamiento real del mercado que el uso de un único threshold global por horizonte.

La conclusión metodológica es que los thresholds globales deben mantenerse como benchmark simple, pero no como definición principal. La metodología principal debería basarse en thresholds por `regime_id`, calculados con datos del período de desarrollo `2020-2024`.

No se recomienda utilizar thresholds por `regime_year`, `regime_year_quarter` o `regime_contract` como definición principal, porque aumentan el riesgo de sobreajuste y cambian demasiado la definición del threshold según contextos muy específicos. Estos cruces son útiles para diagnosticar estabilidad, detectar años extremos y evaluar sensibilidad, pero no para fijar la primera versión operativa.

La recomendación final para el siguiente stage es construir los targets operativos usando como candidato principal:

threshold por horizon + percentile + `regime_id`
y mantener como comparación:

threshold global por horizon + percentile

En particular, los percentiles `p40`, `p50` y `p60` deberían conservarse como escenarios candidatos:

`p40` → versión más flexible

`p50` → versión intermedia

`p60` → versión más exigente

El percentil `p50` puede considerarse el candidato base inicial, mientras que `p40` y `p60` permitirán evaluar sensibilidad en la construcción de targets del Stage 04.

En síntesis, el diagnóstico muestra que el mercado intradiario de MNQ requiere thresholds adaptados al régimen. El uso de `regime_id` permite capturar diferencias estructurales entre momentos del día sin llegar a una segmentación excesivamente específica. Por ello, los thresholds por régimen constituyen la alternativa más robusta, interpretable y operativamente razonable para continuar con la construcción de targets.

6. Selección y formalización de thresholds candidatos

Después de calcular y diagnosticar thresholds en distintos niveles de agregación, el siguiente paso consiste en transformar ese análisis en una decisión metodológica concreta.

El objetivo de este punto no es elegir un único threshold definitivo, sino definir qué metodología de thresholds será utilizada como base en el siguiente stage para construir targets operativos.

A partir del análisis anterior, se evaluarán tres alternativas principales:

1. Threshold global por horizonte
 2. Threshold por régimen intradiario
 3. Threshold por régimen con control robusto año a año
- Los thresholds globales se mantendrán como benchmark simple, ya que permiten comparar contra una regla uniforme para todo el dataset.

Los thresholds por `regime_id` serán evaluados como metodología principal, debido a que el análisis mostró diferencias estructurales importantes entre los distintos momentos de la sesión.

Finalmente, los thresholds por régimen y año se utilizarán como control de robustez, para verificar si los valores por régimen son razonablemente estables entre `2020` y `2024`.

La regla metodológica se mantiene:

Los thresholds se seleccionan usando únicamente 2020-2024.

El período 2025-2026 no participa en esta selección.

En este punto se formalizarán los thresholds candidatos que serán utilizados posteriormente en el Stage 04.

6.1. Comparación metodológica: threshold global vs threshold por régimen

```
In [87]: # 6.1. Comparación metodológica: threshold global vs threshold por régimen
# =====
#
# Objetivo:
# Comparar formalmente los thresholds globales contra los thresholds
# calculados por régimen intradiario.
#
# En este bloque:
# - No se construyen targets.
# - No se usan datos 2025-2026.
# - No se elige todavía un único threshold definitivo.
#
# La comparación se enfoca en los percentiles candidatos principales:
# p40, p50 y p60.
#
# Regla metodológica:
# - threshold global = benchmark
# - threshold por régimen = candidato principal
# =====

import numpy as np
import pandas as pd

# -----
# Validaciones previas
# -----

required_objects = [
    "df_thresholds_global",
```

```

    "df_thresholds_by_regime",
    "THRESHOLD_PERCENTILES_MAIN",
]

missing_objects = [
    obj for obj in required_objects
    if obj not in globals()
]

if missing_objects:
    raise NameError(
        "Faltan objetos necesarios para ejecutar el punto 6.1: "
        f"{missing_objects}"
    )

if df_thresholds_global.empty:
    raise ValueError("df_thresholds_global está vacío.")

if df_thresholds_by_regime.empty:
    raise ValueError("df_thresholds_by_regime está vacío.")

# -----
# Filtrar percentiles candidatos principales
# -----

df_global_candidates = (
    df_thresholds_global[
        df_thresholds_global["percentile"].isin(THRESHOLD_PERCENTILES_MAIN)
    ]
    .copy()
    .reset_index(drop=True)
)

df_regime_candidates = (
    df_thresholds_by_regime[
        df_thresholds_by_regime["percentile"].isin(THRESHOLD_PERCENTILES_MAIN)
    ]
    .copy()
    .reset_index(drop=True)
)

print("Percentiles candidatos principales filtrados.")
print(f"Percentiles considerados: {THRESHOLD_PERCENTILES_MAIN}")

print(f"\nFilas globales: {df_global_candidates.shape[0]:,}")
print(f"Filas por régimen: {df_regime_candidates.shape[0]:,}")

```

Percentiles candidatos principales filtrados.

Percentiles considerados: [40, 50, 60]

Filas globales: 9

Filas por régimen: 39

6.1.1. Unión entre thresholds globales y thresholds por régimen

```

In [88]: # 6.1.1. Unión entre thresholds globales y thresholds por régimen
# =====

```

```

# -----
# Preparar tabla global
# -----

df_global_for_compare = (
    df_global_candidates[
        [
            "horizon",
            "percentile",
            "threshold_up_pts",
            "threshold_down_pts",
            "threshold_common_pts",
            "n_valid",
        ]
    ]
    .rename(
        columns={
            "threshold_up_pts": "global_threshold_up_pts",
            "threshold_down_pts": "global_threshold_down_pts",
            "threshold_common_pts": "global_threshold_common_pts",
            "n_valid": "global_n_valid",
        }
    )
    .copy()
)

# -----
# Preparar tabla por régimen
# -----

df_regime_for_compare = (
    df_regime_candidates[
        [
            "horizon",
            "percentile",
            "regime_id",
            "regime_name",
            "threshold_up_pts",
            "threshold_down_pts",
            "threshold_common_pts",
            "n_valid",
        ]
    ]
    .rename(
        columns={
            "threshold_up_pts": "regime_threshold_up_pts",
            "threshold_down_pts": "regime_threshold_down_pts",
            "threshold_common_pts": "regime_threshold_common_pts",
            "n_valid": "regime_n_valid",
        }
    )
    .copy()
)

# -----
# Comparación global vs régimen
# -----

df_threshold_global_vs_regime = (
    df_regime_for_compare

```

```

        .merge(
            df_global_for_compare,
            on=[
                "horizon",
                "percentile",
            ],
            how="left"
        )
    )

# -----
# Calcular diferencias por tipo de threshold
# -----

for threshold_type in ["up", "down", "common"]:

    regime_col = f"regime_threshold_{threshold_type}_pts"
    global_col = f"global_threshold_{threshold_type}_pts"

    diff_col = f"diff_{threshold_type}_vs_global_pts"
    abs_diff_col = f"abs_diff_{threshold_type}_vs_global_pts"
    ratio_col = f"ratio_{threshold_type}_vs_global"
    pct_col = f"diff_{threshold_type}_vs_global_pct"

    df_threshold_global_vs_regime[diff_col] = (
        df_threshold_global_vs_regime[regime_col]
        - df_threshold_global_vs_regime[global_col]
    )

    df_threshold_global_vs_regime[abs_diff_col] = (
        df_threshold_global_vs_regime[diff_col]
        .abs()
    )

    df_threshold_global_vs_regime[ratio_col] = np.where(
        df_threshold_global_vs_regime[global_col] > 0,
        (
            df_threshold_global_vs_regime[regime_col]
            / df_threshold_global_vs_regime[global_col]
        ),
        np.nan
    )

    df_threshold_global_vs_regime[pct_col] = np.where(
        df_threshold_global_vs_regime[global_col] > 0,
        (
            (
                df_threshold_global_vs_regime[regime_col]
                / df_threshold_global_vs_regime[global_col]
            )
            - 1
        )
        * 100,
        np.nan
    )

df_threshold_global_vs_regime = (
    df_threshold_global_vs_regime
    .replace([np.inf, -np.inf], np.nan)
)

```

```
# -----  
# Vista de resultados  
# -----  
  
print("Comparación entre thresholds globales y thresholds por régimen creada.")  
print("Cada fila compara un régimen contra el threshold global equivalente para  
  
display(df_threshold_global_vs_regime)
```

Comparación entre thresholds globales y thresholds por régimen creada.
Cada fila compara un régimen contra el threshold global equivalente para el mismo
horizonte y percentil.

	horizon	percentile	regime_id	regime_name	regime_threshold_up_pts	regime_thresh
0	30	40	0	Overnight	6.25	
1	30	50	0	Overnight	8.50	
2	30	60	0	Overnight	11.25	
3	30	40	1	Pre-market	10.50	
4	30	50	1	Pre-market	15.00	
5	30	60	1	Pre-market	20.50	
6	30	40	2	Opening	19.00	
7	30	50	2	Opening	26.00	
8	30	60	2	Opening	34.00	
9	30	40	3	Regular	11.75	
10	30	50	3	Regular	16.00	
11	30	60	3	Regular	20.75	
12	30	40	4	Closing	13.00	
13	30	50	4	Closing	18.00	
14	30	60	4	Closing	23.75	
15	60	40	0	Overnight	9.75	
16	60	50	0	Overnight	13.25	
17	60	60	0	Overnight	17.00	
18	60	40	1	Pre-market	24.25	
19	60	50	1	Pre-market	32.75	
20	60	60	1	Pre-market	43.25	
21	60	40	2	Opening	27.00	
22	60	50	2	Opening	36.25	
23	60	60	2	Opening	46.75	
24	60	40	3	Regular	18.00	
25	60	50	3	Regular	23.75	
26	60	60	3	Regular	30.75	
27	90	40	0	Overnight	13.25	
28	90	50	0	Overnight	17.50	
29	90	60	0	Overnight	23.00	
30	90	40	1	Pre-market	33.50	
31	90	50	1	Pre-market	45.25	
32	90	60	1	Pre-market	59.50	

	horizon	percentile	regime_id	regime_name	regime_threshold_up_pts	regime_thresh
33	90	40	2	Opening	32.00	
34	90	50	2	Opening	42.75	
35	90	60	2	Opening	55.25	
36	90	40	3	Regular	22.50	
37	90	50	3	Regular	29.75	
38	90	60	3	Regular	38.25	

39 rows × 24 columns

6.1.2. Resumen enfocado en threshold_common_pts

```
In [89]: # 6.1.2. Resumen enfocado en threshold_common_pts
# =====
#
# Objetivo:
# Resumir la comparación usando threshold_common_pts, ya que este
# será el threshold principal para evaluar magnitud de movimiento
# sin separar inicialmente dirección alcista/bajista.
# =====

df_threshold_global_vs_regime_common_summary = (
    df_threshold_global_vs_regime
    .groupby(
        [
            "horizon",
            "percentile",
        ],
        observed=True
    )
    .agg(
        n_regimes=("regime_id", "nunique"),
        global_threshold_common_pts=("global_threshold_common_pts", "first"),
        min_regime_threshold_common_pts=("regime_threshold_common_pts", "min"),
        median_regime_threshold_common_pts=("regime_threshold_common_pts", "medi
max_regime_threshold_common_pts=("regime_threshold_common_pts", "max"),
        mean_abs_diff_common_vs_global_pct=("diff_common_vs_global_pct", lambda
max_abs_diff_common_vs_global_pct=("diff_common_vs_global_pct", lambda x
min_ratio_common_vs_global=("ratio_common_vs_global", "min"),
        max_ratio_common_vs_global=("ratio_common_vs_global", "max"),
    )
    .reset_index()
)

df_threshold_global_vs_regime_common_summary["regime_top_to_low_ratio"] = np.where
df_threshold_global_vs_regime_common_summary["min_regime_threshold_common_pt
(
    df_threshold_global_vs_regime_common_summary["max_regime_threshold_commo
    / df_threshold_global_vs_regime_common_summary["min_regime_threshold_com
),
    np.nan
)
```

```
df_threshold_global_vs_regime_common_summary = (
    df_threshold_global_vs_regime_common_summary
    .sort_values(
        [
            "horizon",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

print("Resumen de comparación global vs régimen usando threshold_common_pts.")
print("Esta tabla resume cuánto se alejan los thresholds por régimen del threshold global.")
print("Un ratio alto indica que el threshold global no representa de forma homogénea todos los regímenes.")

display(df_threshold_global_vs_regime_common_summary)
```

Resumen de comparación global vs régimen usando threshold_common_pts.
 Esta tabla resume cuánto se alejan los thresholds por régimen del threshold global.
 Un ratio alto indica que el threshold global no representa de forma homogénea todos los regímenes.

	horizon	percentile	n_regimes	global_threshold_common_pts	min_regime_threshold_co
0	30	40	5	21.00	
1	30	50	5	25.50	
2	30	60	5	31.00	
3	60	40	4	31.75	
4	60	50	4	38.50	
5	60	60	4	46.50	
6	90	40	4	40.50	
7	90	50	4	49.00	
8	90	60	4	59.50	

6.1.3. Vista compacta por régimen

```
In [90]: # 6.1.3. Vista compacta por régimen
# =====
#
# Objetivo:
# Ver directamente qué tan lejos queda cada régimen del threshold global.
# =====

compact_cols = [
    "horizon",
    "percentile",
    "regime_id",
    "regime_name",
    "global_threshold_common_pts",
    "regime_threshold_common_pts",
```

```
    "diff_common_vs_global_pts",
    "diff_common_vs_global_pct",
    "ratio_common_vs_global",
    "regime_n_valid",
]

df_threshold_global_vs_regime_common = (
    df_threshold_global_vs_regime[compact_cols]
    .copy()
    .sort_values(
        [
            "horizon",
            "percentile",
            "regime_id",
        ]
    )
    .reset_index(drop=True)
)

print("Vista compacta de threshold común global vs threshold común por régimen.")
print("Valores negativos indican que el régimen tiene un threshold menor al glob")
print("Valores positivos indican que el régimen tiene un threshold mayor al glob")

display(df_threshold_global_vs_regime_common)
```

Vista compacta de threshold común global vs threshold común por régimen.
Valores negativos indican que el régimen tiene un threshold menor al global.
Valores positivos indican que el régimen tiene un threshold mayor al global.

	horizon	percentile	regime_id	regime_name	global_threshold_common_pts	regime_t
0	30	40	0	Overnight	21.00	
1	30	40	1	Pre-market	21.00	
2	30	40	2	Opening	21.00	
3	30	40	3	Regular	21.00	
4	30	40	4	Closing	21.00	
5	30	50	0	Overnight	25.50	
6	30	50	1	Pre-market	25.50	
7	30	50	2	Opening	25.50	
8	30	50	3	Regular	25.50	
9	30	50	4	Closing	25.50	
10	30	60	0	Overnight	31.00	
11	30	60	1	Pre-market	31.00	
12	30	60	2	Opening	31.00	
13	30	60	3	Regular	31.00	
14	30	60	4	Closing	31.00	
15	60	40	0	Overnight	31.75	
16	60	40	1	Pre-market	31.75	
17	60	40	2	Opening	31.75	
18	60	40	3	Regular	31.75	
19	60	50	0	Overnight	38.50	
20	60	50	1	Pre-market	38.50	
21	60	50	2	Opening	38.50	
22	60	50	3	Regular	38.50	
23	60	60	0	Overnight	46.50	
24	60	60	1	Pre-market	46.50	
25	60	60	2	Opening	46.50	
26	60	60	3	Regular	46.50	
27	90	40	0	Overnight	40.50	
28	90	40	1	Pre-market	40.50	
29	90	40	2	Opening	40.50	
30	90	40	3	Regular	40.50	
31	90	50	0	Overnight	49.00	
32	90	50	1	Pre-market	49.00	

	horizon	percentile	regime_id	regime_name	global_threshold_common_pts	regime_t
33	90	50	2	Opening	49.00	
34	90	50	3	Regular	49.00	
35	90	60	0	Overnight	59.50	
36	90	60	1	Pre-market	59.50	
37	90	60	2	Opening	59.50	
38	90	60	3	Regular	59.50	

6.1.4. Observaciones parciales de la comparación global vs régimen

La comparación entre thresholds globales y thresholds por régimen confirma que un único threshold global por horizonte no representa de manera homogénea todos los regímenes intradiarios.

En todos los horizontes y percentiles principales, **Overnight** queda sistemáticamente por debajo del threshold global. Por ejemplo, usando **threshold_common_pts**, en **H30** el threshold global pasa de **21.00** a **31.00** puntos entre **p40** y **p60**, mientras que **Overnight** se ubica entre **14.00** y **19.25** puntos. Esto implica diferencias aproximadas de entre **-33%** y **-38%** respecto al global.

Esto significa que el threshold global sería demasiado exigente para **Overnight**. Muchos movimientos que son relativamente relevantes para ese régimen podrían quedar etiquetados como no significativos si se usa un único threshold global.

En el extremo opuesto, **Opening** queda muy por encima del threshold global, especialmente en horizontes cortos. En **H30**, para **threshold_common_pts**, **Opening** se ubica entre **42.75** y **58.50** puntos para **p40**, **p50** y **p60**, mientras que el global se ubica entre **21.00** y **31.00** puntos. Esto representa diferencias positivas de aproximadamente **+89%** a **+104%**.

Esto indica que el threshold global sería demasiado flexible para **Opening**. Movimientos que durante la apertura pueden ser normales o frecuentes podrían ser tratados como significativos bajo una regla global.

El régimen **Pre-market** también se ubica por encima del global, especialmente en horizontes de **60m** y **90m**. En **H90**, por ejemplo, los thresholds **common** de **Pre-market** están entre **73.75** y **99.00** puntos, frente a valores globales entre **40.50** y **59.50** puntos. La diferencia relativa se mantiene aproximadamente entre **+66%** y **+82%**.

El régimen **Regular** es el más cercano al threshold global. Sus diferencias son positivas, pero más moderadas. En **H90**, para **threshold_common_pts**, **Regular** se ubica apenas entre **+6.7%** y **+14.2%** por encima del global. Esto sugiere que el threshold

global está más influenciado por la gran cantidad de observaciones del régimen regular, pero no necesariamente representa bien a los demás regímenes.

El régimen `Closing` solo aparece en el horizonte de `30m`, lo cual es coherente con la restricción metodológica de no cruzar días. Para horizontes de `60m` y `90m`, no hay suficiente ventana futura dentro del mismo día para evaluar adecuadamente ese régimen.

La tabla resumen confirma esta heterogeneidad. Para `threshold_common_pts`, el ratio entre el régimen con threshold más alto y el más bajo se mantiene alrededor de `3x` en `H30`, entre `2.7x` y `2.8x` en `H60`, y entre `2.5x` y `2.6x` en `H90`.

Además, la diferencia absoluta media contra el threshold global es alta. Para los percentiles principales, la desviación media absoluta se ubica aproximadamente entre `40%` y `51%`, dependiendo del horizonte y percentil. Esto refuerza la idea de que el threshold global no es suficientemente representativo para todo el dataset.

Desde una perspectiva operativa, estos resultados indican que el significado de un movimiento futuro depende fuertemente del régimen intradiario. Un movimiento de cierta magnitud puede ser relevante durante `Overnight`, pero normal durante `Opening` o `Pre-market`.

Por lo tanto, el threshold global debe mantenerse como benchmark simple, pero no debería ser la metodología principal para construir targets operativos.

La alternativa por `regime_id` queda mejor justificada, porque permite adaptar el threshold a la escala típica de movimiento de cada tramo de la sesión, sin llegar todavía a una segmentación excesivamente específica por año, trimestre o contrato.

La conclusión parcial del punto 6.1 es:

threshold global por horizonte = benchmark de comparación
threshold por horizon + percentile + regime_id = candidato metodológico principal

Antes de formalizar esta decisión, todavía falta evaluar en el punto 6.2 si los thresholds por régimen son razonablemente estables año a año dentro del período `2020-2024`.

6.2. Evaluación de robustez: threshold por régimen pooled vs mediana anual

```
In [91]: # 6.2. Evaluación de robustez:
#         threshold por régimen pooled vs mediana anual
#         =====
#
# Objetivo:
# Evaluar si los thresholds por régimen calculados con todo el período
# 2020-2024 están demasiado influenciados por años extremos.
#
# Se comparan dos metodologías:
#
# 1. regime_pooled_2020_2024
```

```

#   Calcula el threshold por régimen usando todas las observaciones
#   válidas de 2020-2024 juntas.
#
# 2. regime_median_year_2020_2024
#   Calcula primero thresholds por régimen y año, y luego toma la
#   mediana de esos thresholds anuales.
#
# En este bloque:
# - No se construyen targets.
# - No se usa 2025-2026.
# - No se selecciona todavía un threshold definitivo.
# =====

import numpy as np
import pandas as pd

# -----
# Validaciones previas
# -----

required_objects = [
    "df_thresholds_by_regime",
    "df_thresholds_by_regime_year",
    "THRESHOLD_PERCENTILES_MAIN",
]

missing_objects = [
    obj for obj in required_objects
    if obj not in globals()
]

if missing_objects:
    raise NameError(
        "Faltan objetos necesarios para ejecutar el punto 6.2: "
        f"{missing_objects}"
    )

if df_thresholds_by_regime.empty:
    raise ValueError("df_thresholds_by_regime está vacío.")

if df_thresholds_by_regime_year.empty:
    raise ValueError("df_thresholds_by_regime_year está vacío.")

print("Tablas necesarias cargadas correctamente.")
print("Se comparará threshold por régimen pooled contra mediana anual por régimen")

```

Tablas necesarias cargadas correctamente.

Se comparará threshold por régimen pooled contra mediana anual por régimen.

6.2.1. Construcción de tabla pooled por régimen

```

In [92]: # 6.2.1. Construcción de tabla pooled por régimen
# =====

df_thresholds_regime_pooled = (
    df_thresholds_by_regime[
        df_thresholds_by_regime["percentile"].isin(THRESHOLD_PERCENTILES_MAIN)
    ]
    .copy()
)

```

```

        .reset_index(drop=True)
    )

    df_thresholds_regime_pooled["threshold_method"] = "regime_pooled_2020_2024"
    df_thresholds_regime_pooled["method_role"] = "primary_candidate"

    # -----
    # Columnas principales
    # -----

    pooled_cols = [
        "threshold_method",
        "method_role",
        "threshold_scope",
        "selection_period",
        "horizon",
        "percentile",
        "regime_id",
        "regime_name",
        "threshold_up_pts",
        "threshold_down_pts",
        "threshold_common_pts",
        "n_valid",
        "candidate_for_threshold",
    ]

    df_thresholds_regime_pooled = df_thresholds_regime_pooled[pooled_cols].copy()

    print("Tabla pooled por régimen creada.")
    print("Esta tabla calcula thresholds usando todas las observaciones válidas de 2020-2024")

    print(f"\nFilas: {df_thresholds_regime_pooled.shape[0]:,}")
    print(f"Columnas: {df_thresholds_regime_pooled.shape[1]:,}")

    display(df_thresholds_regime_pooled)

```

Tabla pooled por régimen creada.

Esta tabla calcula thresholds usando todas las observaciones válidas de 2020-2024 juntas.

Filas: 39

Columnas: 13

	threshold_method	method_role	threshold_scope	selection_period
0	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
1	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
2	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
3	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
4	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
5	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
6	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
7	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
8	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
9	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
10	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
11	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
12	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
13	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
14	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
15	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
16	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
17	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
18	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
19	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
20	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
21	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
22	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
23	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
24	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
25	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
26	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
27	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
28	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
29	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
30	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
31	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
32	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024

	threshold_method	method_role	threshold_scope	selection_period
33	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
34	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
35	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
36	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
37	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024
38	regime_pooled_2020_2024	primary_candidate	regime	development_2020_2024

6.2.2. Construcción de mediana anual por régimen

```
In [93]: # 6.2.2. Construcción de mediana anual por régimen
# =====
#
# Objetivo:
# Calcular una versión robusta de los thresholds por régimen.
#
# En lugar de usar todas las observaciones 2020-2024 juntas,
# primero se calculan thresholds por año y régimen, y luego se toma
# la mediana de esos thresholds anuales.
# =====
# -----
# Filtrar percentiles principales
# -----

df_regime_year_main = (
    df_thresholds_by_regime_year[
        df_thresholds_by_regime_year["percentile"].isin(THRESHOLD_PERCENTILES_MA
    )
    .copy()
    .reset_index(drop=True)
)

required_cols = [
    "horizon",
    "percentile",
    "year",
    "regime_id",
    "regime_name",
    "threshold_up_pts",
    "threshold_down_pts",
    "threshold_common_pts",
    "n_valid",
]

missing_cols = [
    col for col in required_cols
    if col not in df_regime_year_main.columns
]

if missing_cols:
    raise ValueError(
```

```

        "Faltan columnas necesarias en df_regime_year_main: "
        f"{missing_cols}"
    )

# -----
# Mediana de thresholds anuales por régimen
# -----

df_thresholds_regime_median_year = (
    df_regime_year_main
    .groupby(
        [
            "horizon",
            "percentile",
            "regime_id",
            "regime_name",
        ],
        observed=True
    )
    .agg(
        threshold_up_pts=("threshold_up_pts", "median"),
        threshold_down_pts=("threshold_down_pts", "median"),
        threshold_common_pts=("threshold_common_pts", "median"),

        mean_threshold_up_pts=("threshold_up_pts", "mean"),
        mean_threshold_down_pts=("threshold_down_pts", "mean"),
        mean_threshold_common_pts=("threshold_common_pts", "mean"),

        min_year_threshold_common_pts=("threshold_common_pts", "min"),
        max_year_threshold_common_pts=("threshold_common_pts", "max"),

        n_years=("year", "nunique"),
        min_year=("year", "min"),
        max_year=("year", "max"),
        median_n_valid=("n_valid", "median"),
        total_n_valid=("n_valid", "sum"),
    )
    .reset_index()
)

df_thresholds_regime_median_year["threshold_method"] = "regime_median_year_2020_"
df_thresholds_regime_median_year["method_role"] = "robustness_check"
df_thresholds_regime_median_year["threshold_scope"] = "regime"
df_thresholds_regime_median_year["selection_period"] = "development_2020_2024"
df_thresholds_regime_median_year["candidate_for_threshold"] = True

# -----
# Reordenar columnas
# -----

median_year_cols = [
    "threshold_method",
    "method_role",
    "threshold_scope",
    "selection_period",
    "horizon",
    "percentile",
    "regime_id",
    "regime_name",
    "threshold_up_pts",

```

```

        "threshold_down_pts",
        "threshold_common_pts",
        "mean_threshold_up_pts",
        "mean_threshold_down_pts",
        "mean_threshold_common_pts",
        "min_year_threshold_common_pts",
        "max_year_threshold_common_pts",
        "n_years",
        "min_year",
        "max_year",
        "median_n_valid",
        "total_n_valid",
        "candidate_for_threshold",
    ]

df_thresholds_regime_median_year = (
    df_thresholds_regime_median_year[median_year_cols]
    .sort_values(
        [
            "horizon",
            "regime_id",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

print("Tabla robusta por régimen creada usando mediana de thresholds anuales.")
print("Esta tabla servirá como control para evaluar si el pooled está influenciado por años extremos.")

print(f"\nFilas: {df_thresholds_regime_median_year.shape[0]:,}")
print(f"Columnas: {df_thresholds_regime_median_year.shape[1]:,}")

display(df_thresholds_regime_median_year)

```

Tabla robusta por régimen creada usando mediana de thresholds anuales.
 Esta tabla servirá como control para evaluar si el pooled está influenciado por años extremos.

Filas: 39

Columnas: 22

	threshold_method	method_role	threshold_scope	selection_peric
0	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
1	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
2	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
3	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
4	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
5	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
6	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
7	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
8	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
9	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
10	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
11	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
12	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
13	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
14	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
15	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
16	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
17	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
18	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
19	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
20	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
21	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
22	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
23	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
24	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
25	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
26	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
27	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
28	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
29	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
30	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
31	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
32	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024

	threshold_method	method_role	threshold_scope	selection_peric
33	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
34	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
35	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
36	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
37	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
38	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024

39 rows × 22 columns

6.2.3. Comparación pooled vs mediana anual

```
In [94]: # 6.2.3. Comparación pooled vs mediana anual
# =====
#
# Objetivo:
# Comparar directamente:
#
# - regime_pooled_2020_2024
# - regime_median_year_2020_2024
#
# para cada combinación:
#
# - horizon
# - percentile
# - regime_id
# =====
#
# -----
# Preparar tabla pooled
# -----

df_pooled_compare = (
    df_thresholds_regime_pooled[
        [
            "horizon",
            "percentile",
            "regime_id",
            "regime_name",
            "threshold_up_pts",
            "threshold_down_pts",
            "threshold_common_pts",
            "n_valid",
        ]
    ]
    .rename(
        columns={
            "threshold_up_pts": "pooled_threshold_up_pts",
            "threshold_down_pts": "pooled_threshold_down_pts",
            "threshold_common_pts": "pooled_threshold_common_pts",
            "n_valid": "pooled_n_valid",
        }
    )
)
```

```

    )
    .copy()
)

# -----
# Preparar tabla mediana anual
# -----

df_median_year_compare = (
    df_thresholds_regime_median_year[
        [
            "horizon",
            "percentile",
            "regime_id",
            "regime_name",
            "threshold_up_pts",
            "threshold_down_pts",
            "threshold_common_pts",
            "n_years",
            "median_n_valid",
            "total_n_valid",
            "min_year_threshold_common_pts",
            "max_year_threshold_common_pts",
        ]
    ]
    .rename(
        columns={
            "threshold_up_pts": "median_year_threshold_up_pts",
            "threshold_down_pts": "median_year_threshold_down_pts",
            "threshold_common_pts": "median_year_threshold_common_pts",
        }
    )
    .copy()
)

# -----
# Unir ambas metodologías
# -----

df_threshold_pooled_vs_median_year = (
    df_pooled_compare
    .merge(
        df_median_year_compare,
        on=[
            "horizon",
            "percentile",
            "regime_id",
            "regime_name",
        ],
        how="left"
    )
)

# -----
# Calcular diferencias
# -----

for threshold_type in ["up", "down", "common"]:

    pooled_col = f"pooled_threshold_{threshold_type}_pts"

```

```

median_col = f"median_year_threshold_{threshold_type}_pts"

diff_col = f"diff_pooled_vs_median_year_{threshold_type}_pts"
abs_diff_col = f"abs_diff_pooled_vs_median_year_{threshold_type}_pts"
ratio_col = f"ratio_pooled_vs_median_year_{threshold_type}"
pct_col = f"diff_pooled_vs_median_year_{threshold_type}_pct"

df_threshold_pooled_vs_median_year[diff_col] = (
    df_threshold_pooled_vs_median_year[pooled_col]
    - df_threshold_pooled_vs_median_year[median_col]
)

df_threshold_pooled_vs_median_year[abs_diff_col] = (
    df_threshold_pooled_vs_median_year[diff_col]
    .abs()
)

df_threshold_pooled_vs_median_year[ratio_col] = np.where(
    df_threshold_pooled_vs_median_year[median_col] > 0,
    (
        df_threshold_pooled_vs_median_year[pooled_col]
        / df_threshold_pooled_vs_median_year[median_col]
    ),
    np.nan
)

df_threshold_pooled_vs_median_year[pct_col] = np.where(
    df_threshold_pooled_vs_median_year[median_col] > 0,
    (
        (
            df_threshold_pooled_vs_median_year[pooled_col]
            / df_threshold_pooled_vs_median_year[median_col]
        )
        - 1
    )
    * 100,
    np.nan
)

df_threshold_pooled_vs_median_year = (
    df_threshold_pooled_vs_median_year
    .replace([np.inf, -np.inf], np.nan)
)

# -----
# Ordenar resultados
# -----

df_threshold_pooled_vs_median_year = (
    df_threshold_pooled_vs_median_year
    .sort_values(
        [
            "horizon",
            "regime_id",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

```

```
print("Comparación entre threshold por régimen pooled y mediana anual creada.")  
print("Esta tabla permite evaluar si el pooled se aleja mucho del valor robusto  
  
print(f"\nTabla generada: df_threshold_pooled_vs_median_year")  
print(f"Filas: {df_threshold_pooled_vs_median_year.shape[0]:,}")  
print(f"Columnas: {df_threshold_pooled_vs_median_year.shape[1]:,}")  
  
display(df_threshold_pooled_vs_median_year)
```

Comparación entre threshold por régimen pooled y mediana anual creada.
Esta tabla permite evaluar si el pooled se aleja mucho del valor robusto año a año.

Tabla generada: df_threshold_pooled_vs_median_year
Filas: 39
Columnas: 28

	horizon	percentile	regime_id	regime_name	pooled_threshold_up_pts	pooled_thresh
0	30	40	0	Overnight	6.25	
1	30	50	0	Overnight	8.50	
2	30	60	0	Overnight	11.25	
3	30	40	1	Pre-market	10.50	
4	30	50	1	Pre-market	15.00	
5	30	60	1	Pre-market	20.50	
6	30	40	2	Opening	19.00	
7	30	50	2	Opening	26.00	
8	30	60	2	Opening	34.00	
9	30	40	3	Regular	11.75	
10	30	50	3	Regular	16.00	
11	30	60	3	Regular	20.75	
12	30	40	4	Closing	13.00	
13	30	50	4	Closing	18.00	
14	30	60	4	Closing	23.75	
15	60	40	0	Overnight	9.75	
16	60	50	0	Overnight	13.25	
17	60	60	0	Overnight	17.00	
18	60	40	1	Pre-market	24.25	
19	60	50	1	Pre-market	32.75	
20	60	60	1	Pre-market	43.25	
21	60	40	2	Opening	27.00	
22	60	50	2	Opening	36.25	
23	60	60	2	Opening	46.75	
24	60	40	3	Regular	18.00	
25	60	50	3	Regular	23.75	
26	60	60	3	Regular	30.75	
27	90	40	0	Overnight	13.25	
28	90	50	0	Overnight	17.50	
29	90	60	0	Overnight	23.00	
30	90	40	1	Pre-market	33.50	
31	90	50	1	Pre-market	45.25	
32	90	60	1	Pre-market	59.50	

	horizon	percentile	regime_id	regime_name	pooled_threshold_up_pts	pooled_thresh
33	90	40	2	Opening		32.00
34	90	50	2	Opening		42.75
35	90	60	2	Opening		55.25
36	90	40	3	Regular		22.50
37	90	50	3	Regular		29.75
38	90	60	3	Regular		38.25

39 rows × 28 columns

6.2.4. Resumen de robustez usando threshold_common_pts

```
In [95]: # 6.2.4. Resumen de robustez usando threshold_common_pts
# =====
#
# Objetivo:
# Resumir la diferencia entre pooled y mediana anual usando
# threshold_common_pts, que será el threshold principal para medir
# magnitud de movimiento.
# =====

df_threshold_pooled_vs_median_year_common_summary = (
    df_threshold_pooled_vs_median_year
    .groupby(
        [
            "horizon",
            "percentile",
        ],
        observed=True
    )
    .agg(
        n_regimes=("regime_id", "nunique"),
        mean_abs_diff_common_pct=("diff_pooled_vs_median_year_common_pct", lambda
        median_abs_diff_common_pct=("diff_pooled_vs_median_year_common_pct", lambda
        max_abs_diff_common_pct=("diff_pooled_vs_median_year_common_pct", lambda
        mean_abs_diff_common_pts=("abs_diff_pooled_vs_median_year_common_pts", "
        median_abs_diff_common_pts=("abs_diff_pooled_vs_median_year_common_pts",
        max_abs_diff_common_pts=("abs_diff_pooled_vs_median_year_common_pts", "r
        min_ratio_common=("ratio_pooled_vs_median_year_common", "min"),
        max_ratio_common=("ratio_pooled_vs_median_year_common", "max"),
    )
    .reset_index()
)

df_threshold_pooled_vs_median_year_common_summary = (
    df_threshold_pooled_vs_median_year_common_summary
    .sort_values(
        [
            "horizon",
            "percentile",
        ]
    )
)
```

```

        .reset_index(drop=True)
    )

    print("Resumen de robustez pooled vs mediana anual usando threshold_common_pts.")
    print("Valores bajos de diferencia indican que el threshold pooled no está demas")

    display(df_threshold_pooled_vs_median_year_common_summary)

```

Resumen de robustez pooled vs mediana anual usando threshold_common_pts.

Valores bajos de diferencia indican que el threshold pooled no está demasiado alejado del control robusto anual.

	horizon	percentile	n_regimes	mean_abs_diff_common_pct	median_abs_diff_common_p
0	30	40	5	7.347381	7.4468
1	30	50	5	9.259495	9.7345
2	30	60	5	8.598218	7.3394
3	60	40	4	6.816208	6.3692
4	60	50	4	9.242230	9.2079
5	60	60	4	9.873024	9.4601
6	90	40	4	7.595111	7.5142
7	90	50	4	8.657606	8.6667
8	90	60	4	10.407075	10.2170

6.2.5. Vista compacta por régimen

```

In [96]: # 6.2.5. Vista compacta por régimen
# =====
#
# Objetivo:
# Ver directamente, para cada régimen, cuánto se aleja el threshold
# pooled respecto de la mediana anual.
# =====

compact_cols = [
    "horizon",
    "percentile",
    "regime_id",
    "regime_name",
    "pooled_threshold_common_pts",
    "median_year_threshold_common_pts",
    "diff_pooled_vs_median_year_common_pts",
    "diff_pooled_vs_median_year_common_pct",
    "ratio_pooled_vs_median_year_common",
    "min_year_threshold_common_pts",
    "max_year_threshold_common_pts",
    "pooled_n_valid",
    "n_years",
]

df_threshold_pooled_vs_median_year_common = (
    df_threshold_pooled_vs_median_year[compact_cols]

```

```
.copy()
.sort_values(
    [
        "horizon",
        "regime_id",
        "percentile",
    ]
)
.reset_index(drop=True)
)

print("Vista compacta de threshold pooled vs mediana anual por régimen.")
print("Esta tabla muestra si el threshold pooled está alineado con el valor robu

display(df_threshold_pooled_vs_median_year_common)
```

Vista compacta de threshold pooled vs mediana anual por régimen.

Esta tabla muestra si el threshold pooled está alineado con el valor robusto de los thresholds anuales.

	horizon	percentile	regime_id	regime_name	pooled_threshold_common_pts	median_
0	30	40	0	Overnight	14.00	
1	30	50	0	Overnight	16.50	
2	30	60	0	Overnight	19.25	
3	30	40	1	Pre-market	25.25	
4	30	50	1	Pre-market	31.00	
5	30	60	1	Pre-market	38.25	
6	30	40	2	Opening	42.75	
7	30	50	2	Opening	50.00	
8	30	60	2	Opening	58.50	
9	30	40	3	Regular	25.50	
10	30	50	3	Regular	30.00	
11	30	60	3	Regular	35.50	
12	30	40	4	Closing	30.00	
13	30	50	4	Closing	35.50	
14	30	60	4	Closing	41.75	
15	60	40	0	Overnight	21.00	
16	60	50	0	Overnight	24.75	
17	60	60	0	Overnight	29.25	
18	60	40	1	Pre-market	54.00	
19	60	50	1	Pre-market	63.25	
20	60	60	1	Pre-market	74.00	
21	60	40	2	Opening	58.50	
22	60	50	2	Opening	68.50	
23	60	60	2	Opening	79.75	
24	60	40	3	Regular	37.25	
25	60	50	3	Regular	44.00	
26	60	60	3	Regular	51.50	
27	90	40	0	Overnight	28.00	
28	90	50	0	Overnight	33.00	
29	90	60	0	Overnight	39.25	
30	90	40	1	Pre-market	73.75	
31	90	50	1	Pre-market	85.50	
32	90	60	1	Pre-market	99.00	

	horizon	percentile	regime_id	regime_name	pooled_threshold_common_pts	median_
33	90	40	2	Opening	68.75	
34	90	50	2	Opening	80.25	
35	90	60	2	Opening	93.50	
36	90	40	3	Regular	46.25	
37	90	50	3	Regular	54.25	
38	90	60	3	Regular	63.50	

6.2.6. Observaciones parciales de robustez pooled vs mediana anual

La comparación entre `regime_pooled_2020_2024` y `regime_median_year_2020_2024` muestra que los thresholds por régimen calculados con todo el período de desarrollo están razonablemente alineados con la versión robusta basada en la mediana anual.

En todos los casos, el threshold pooled queda por encima de la mediana anual. Esto indica que el cálculo usando todas las observaciones de `2020-2024` tiende a producir thresholds ligeramente más exigentes que el cálculo robusto por mediana de años.

Sin embargo, las diferencias no son extremas. Para `threshold_common_pts`, la diferencia media absoluta se ubica aproximadamente entre `6.8%` y `10.4%`, dependiendo del horizonte y percentil. La diferencia máxima observada se mantiene por debajo de aproximadamente `14%`.

Esto es importante porque sugiere que el threshold pooled no está distorsionado de forma severa por años extremos. Aunque `2022` fue un año de mayor amplitud, su efecto no parece suficiente para invalidar el uso del cálculo pooled por régimen.

Los ratios entre el threshold pooled y la mediana anual se mantienen en un rango acotado, aproximadamente entre `1.03` y `1.12`. Esto significa que, en términos prácticos, el threshold pooled suele estar entre un `3%` y un `12%` por encima de la mediana anual.

Desde una perspectiva operativa, esta diferencia puede interpretarse como una mayor exigencia del método pooled. Es decir, al usar todos los datos de `2020-2024`, los thresholds resultantes filtran un poco más los movimientos pequeños que la versión basada en mediana anual.

Algunos casos muestran diferencias algo mayores, como `Closing` en `H30 p50`, `Pre-market` en `H30 p60`, `Pre-market` en `H60 p60` y `Pre-market` en `H90 p60`. Aun así, estas diferencias siguen siendo moderadas y no cambian la conclusión general.

También se observa que cada combinación por régimen cuenta con `5` años disponibles, lo cual permite una comparación temporal equilibrada dentro del período de desarrollo.

En el caso de `Closing`, la comparación aparece solo para `H30`, lo cual es consistente con la restricción de no cruzar días: para horizontes más largos no existe suficiente ventana futura dentro del cierre de la sesión.

La conclusión parcial del punto 6.2 es que los thresholds por régimen calculados con el método pooled son suficientemente robustos frente a la comparación año a año.

Por lo tanto, se puede mantener:

`regime_pooled_2020_2024`

como metodología candidata principal, y conservar:

`t"`

`regime_median_year_2020_2024`

como control de robustez o análisis de sensibilidad.

En síntesis, el análisis del punto 6.2 refuerza la decisión metodológica preliminar: los thresholds por `regime_id` son más representativos que los thresholds globales y, además, su versión pooled no se aleja de forma problemática de la mediana anual.

7. Guardado de thresholds seleccionados

El objetivo de este punto es construir y guardar las tablas finales de thresholds que serán utilizadas en el siguiente stage para construir targets operativos.

En los puntos anteriores se calcularon thresholds bajo distintas metodologías y se analizó su estabilidad. A partir de ese diagnóstico, se definió una prioridad metodológica clara.

La metodología principal será utilizar thresholds por régimen intradiario, calculados sobre todo el período de desarrollo `2020-2024`. Esta versión se denomina:

`regime_pooled_2020_2024`

Esta metodología calcula los thresholds por:

`horizon + percentile + regime_id`

y permite capturar las diferencias estructurales entre los distintos momentos de la sesión, como `Overnight`, `Pre-market`, `Opening`, `Regular` y `Closing`.

Los thresholds globales se conservarán como benchmark de comparación. Esta versión se denomina:

`global_pooled_2020_2024`

Su objetivo no es ser la metodología principal, sino permitir comparar si los thresholds por régimen generan targets más coherentes que una regla global uniforme.

Finalmente, se conservará una versión robusta basada en la mediana anual de thresholds por régimen. Esta versión se denomina:

`regime_median_year_2020_2024`

Esta tabla funciona como control de robustez para verificar que los thresholds por régimen calculados con el método pooled no estén excesivamente influenciados por años extremos.

Los thresholds que se evaluarán inicialmente en el Stage 04 serán:

1. `regime_pooled_2020_2024 + threshold_common_pts + p40/p50/p60 + H30/H60/H90`
→ metodología principal
2. `global_pooled_2020_2024 + threshold_common_pts + p40/p50/p60 + H30/H60/H90`
→ benchmark
3. `regime_median_year_2020_2024 + threshold_common_pts + p40/p50/p60 + H30/H60/H90`
→ control de robustez

Los percentiles se interpretarán de la siguiente manera:

p40 → versión más flexible
p50 → versión base
p60 → versión más exigente

Aunque la primera evaluación de targets utilizará `threshold_common_pts`, las columnas `threshold_up_pts` y `threshold_down_pts` también se conservarán en las tablas finales para permitir análisis posteriores con thresholds direccionales.

No se utilizarán inicialmente thresholds por `regime_year`, `regime_year_quarter`, `regime_contract` o `regime_contract_family` como metodología principal, ya que esos enfoques presentan mayor riesgo de sobreajuste y se mantienen únicamente como diagnóstico de sensibilidad.

La salida principal de este punto será:

`df_thresholds_final_candidates`

Esta tabla consolidará los thresholds candidatos que serán utilizados en el Stage 04 sin recalcularlos ni incorporar información del período `2025-2026`.

7.1. Construcción de tablas finales de thresholds

```
In [97]: # 7.1. Construcción de tablas finales de thresholds
# =====
#
# Objetivo:
# Construir las tablas finales de thresholds que serán utilizadas
# en el Stage 04 para construir targets operativos.
#
# Se crearán:
# - df_thresholds_global_benchmark
# - df_thresholds_regime_primary
# - df_thresholds_regime_robust_check
# - df_thresholds_final_candidates
#
# Regla metodológica:
```

```

# Los thresholds fueron calculados usando únicamente 2020-2024.
# El período 2025-2026 no participa en la selección.
# =====

import numpy as np
import pandas as pd
from pathlib import Path
import json

# -----
# Validaciones previas
# -----

required_objects = [
    "df_thresholds_global",
    "df_thresholds_by_regime",
    "df_thresholds_by_regime_year",
    "THRESHOLD_HORIZONS",
    "THRESHOLD_PERCENTILES_MAIN",
]

missing_objects = [
    obj for obj in required_objects
    if obj not in globals()
]

if missing_objects:
    raise NameError(
        "Faltan objetos necesarios para construir las tablas finales: "
        f"{missing_objects}"
    )

if df_thresholds_global.empty:
    raise ValueError("df_thresholds_global está vacío.")

if df_thresholds_by_regime.empty:
    raise ValueError("df_thresholds_by_regime está vacío.")

if df_thresholds_by_regime_year.empty:
    raise ValueError("df_thresholds_by_regime_year está vacío.")

# -----
# Configuración final
# -----

THRESHOLD_FINAL_PERCENTILES = THRESHOLD_PERCENTILES_MAIN
STAGE04_INITIAL_THRESHOLD_TYPE = "common"
STAGE04_INITIAL_THRESHOLD_COL = "threshold_common_pts"

print("Configuración final de thresholds para Stage 04:")
print(f"Horizontes: {THRESHOLD_HORIZONS}")
print(f"Percentiles principales: {THRESHOLD_FINAL_PERCENTILES}")
print(f"Threshold inicial a evaluar: {STAGE04_INITIAL_THRESHOLD_COL}")

```

Configuración final de thresholds para Stage 04:
 Horizontes: [30, 60, 90]
 Percentiles principales: [40, 50, 60]
 Threshold inicial a evaluar: threshold_common_pts

7.1.1. Tabla benchmark global

```

In [98]: # 7.1.1. Tabla benchmark global
# =====

df_thresholds_global_benchmark = (
    df_thresholds_global[
        df_thresholds_global["percentile"].isin(THRESHOLD_FINAL_PERCENTILES)
    ]
    .copy()
    .reset_index(drop=True)
)

df_thresholds_global_benchmark["threshold_method"] = "global_pooled_2020_2024"
df_thresholds_global_benchmark["method_role"] = "benchmark"
df_thresholds_global_benchmark["is_primary_method"] = False
df_thresholds_global_benchmark["is_benchmark"] = True
df_thresholds_global_benchmark["is_robustness_check"] = False
df_thresholds_global_benchmark["include_in_stage04_evaluation"] = True
df_thresholds_global_benchmark["stage04_priority"] = 2
df_thresholds_global_benchmark["stage04_threshold_type"] = STAGE04_INITIAL_THRES
df_thresholds_global_benchmark["stage04_threshold_col"] = STAGE04_INITIAL_THRESH
df_thresholds_global_benchmark["notes"] = (
    "Benchmark global por horizonte y percentil. "
    "No segmenta por régimen intradiario."
)

# Columnas no aplicables al global
df_thresholds_global_benchmark["regime_id"] = np.nan
df_thresholds_global_benchmark["regime_name"] = np.nan
df_thresholds_global_benchmark["n_years"] = np.nan
df_thresholds_global_benchmark["min_year"] = np.nan
df_thresholds_global_benchmark["max_year"] = np.nan
df_thresholds_global_benchmark["median_n_valid"] = np.nan
df_thresholds_global_benchmark["total_n_valid"] = np.nan

print("Tabla benchmark global creada.")
print(f"Filas: {df_thresholds_global_benchmark.shape[0]:,}")
print(f"Columnas: {df_thresholds_global_benchmark.shape[1]:,}")

display(df_thresholds_global_benchmark)

```

Tabla benchmark global creada.

Filas: 9

Columnas: 26

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	threshc
0	global	development_2020_2024	30	40	9.25	
1	global	development_2020_2024	30	50	13.00	
2	global	development_2020_2024	30	60	17.25	
3	global	development_2020_2024	60	40	14.50	
4	global	development_2020_2024	60	50	19.75	
5	global	development_2020_2024	60	60	26.25	
6	global	development_2020_2024	90	40	18.75	
7	global	development_2020_2024	90	50	25.50	
8	global	development_2020_2024	90	60	33.75	

9 rows × 26 columns



7.1.2. Tabla principal por régimen pooled

```

In [99]: # 7.1.2. Tabla principal por régimen pooled
# =====

df_thresholds_regime_primary = (
    df_thresholds_by_regime[
        df_thresholds_by_regime["percentile"].isin(THRESHOLD_FINAL_PERCENTILES)
    ]
    .copy()
    .reset_index(drop=True)
)

df_thresholds_regime_primary["threshold_method"] = "regime_pooled_2020_2024"
df_thresholds_regime_primary["method_role"] = "primary"
df_thresholds_regime_primary["is_primary_method"] = True
df_thresholds_regime_primary["is_benchmark"] = False
df_thresholds_regime_primary["is_robustness_check"] = False
df_thresholds_regime_primary["include_in_stage04_evaluation"] = True
df_thresholds_regime_primary["stage04_priority"] = 1
df_thresholds_regime_primary["stage04_threshold_type"] = STAGE04_INITIAL_THRESHOLD
df_thresholds_regime_primary["stage04_threshold_col"] = STAGE04_INITIAL_THRESHOLD
df_thresholds_regime_primary["notes"] = (
    "Metodología principal. Thresholds por horizonte, percentil y régimen intradía  

    calculados con todas las observaciones válidas de 2020-2024."
)

df_thresholds_regime_primary["n_years"] = np.nan
df_thresholds_regime_primary["min_year"] = np.nan
df_thresholds_regime_primary["max_year"] = np.nan
df_thresholds_regime_primary["median_n_valid"] = np.nan
df_thresholds_regime_primary["total_n_valid"] = np.nan

print("Tabla principal por régimen pooled creada.")
print(f"Filas: {df_thresholds_regime_primary.shape[0]:,}")
print(f"Columnas: {df_thresholds_regime_primary.shape[1]:,}")

display(df_thresholds_regime_primary)

```

Tabla principal por régimen pooled creada.

Filas: 39

Columnas: 26

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thresh
0	regime	development_2020_2024	30	40	6.25	
1	regime	development_2020_2024	30	50	8.50	
2	regime	development_2020_2024	30	60	11.25	
3	regime	development_2020_2024	30	40	10.50	
4	regime	development_2020_2024	30	50	15.00	
5	regime	development_2020_2024	30	60	20.50	
6	regime	development_2020_2024	30	40	19.00	
7	regime	development_2020_2024	30	50	26.00	
8	regime	development_2020_2024	30	60	34.00	
9	regime	development_2020_2024	30	40	11.75	

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thresh
10	regime	development_2020_2024	30	50	16.00	
11	regime	development_2020_2024	30	60	20.75	
12	regime	development_2020_2024	30	40	13.00	
13	regime	development_2020_2024	30	50	18.00	
14	regime	development_2020_2024	30	60	23.75	
15	regime	development_2020_2024	60	40	9.75	
16	regime	development_2020_2024	60	50	13.25	
17	regime	development_2020_2024	60	60	17.00	
18	regime	development_2020_2024	60	40	24.25	
19	regime	development_2020_2024	60	50	32.75	

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thresh
20	regime	development_2020_2024	60	60	43.25	
21	regime	development_2020_2024	60	40	27.00	
22	regime	development_2020_2024	60	50	36.25	
23	regime	development_2020_2024	60	60	46.75	
24	regime	development_2020_2024	60	40	18.00	
25	regime	development_2020_2024	60	50	23.75	
26	regime	development_2020_2024	60	60	30.75	
27	regime	development_2020_2024	90	40	13.25	
28	regime	development_2020_2024	90	50	17.50	
29	regime	development_2020_2024	90	60	23.00	

	threshold_scope	selection_period	horizon	percentile	threshold_up_pts	thresh
30	regime	development_2020_2024	90	40	33.50	
31	regime	development_2020_2024	90	50	45.25	
32	regime	development_2020_2024	90	60	59.50	
33	regime	development_2020_2024	90	40	32.00	
34	regime	development_2020_2024	90	50	42.75	
35	regime	development_2020_2024	90	60	55.25	
36	regime	development_2020_2024	90	40	22.50	
37	regime	development_2020_2024	90	50	29.75	
38	regime	development_2020_2024	90	60	38.25	

39 rows × 26 columns

7.1.3. Tabla robusta por mediana anual

```

In [100... # 7.1.3. Tabla robusta por mediana anual
# =====
#
# Objetivo:
# Construir una tabla de control robusto.
#
# Primero se usan los thresholds por régimen y año.
# Luego se toma la mediana de esos thresholds anuales.
# =====

df_regime_year_main = (
    df_thresholds_by_regime_year[
        df_thresholds_by_regime_year["percentile"].isin(THRESHOLD_FINAL_PERCENTI
    ]
    .copy()
    .reset_index(drop=True)
)

required_cols = [
    "horizon",
    "percentile",
    "year",
    "regime_id",
    "regime_name",
    "threshold_up_pts",
    "threshold_down_pts",
    "threshold_common_pts",
    "n_valid",
]

missing_cols = [
    col for col in required_cols
    if col not in df_regime_year_main.columns
]

if missing_cols:
    raise ValueError(
        "Faltan columnas necesarias en df_regime_year_main: "
        f"{missing_cols}"
    )

df_thresholds_regime_robust_check = (
    df_regime_year_main
    .groupby(
        [
            "horizon",
            "percentile",
            "regime_id",
            "regime_name",
        ],
        observed=True
    )
    .agg(
        threshold_up_pts=("threshold_up_pts", "median"),
        threshold_down_pts=("threshold_down_pts", "median"),
        threshold_common_pts=("threshold_common_pts", "median"),
        n_years=("year", "nunique"),
        min_year=("year", "min"),
        max_year=("year", "max"),
    )

```

```

        median_n_valid=("n_valid", "median"),
        total_n_valid=("n_valid", "sum"),
    )
    .reset_index()
)

df_thresholds_regime_robust_check["threshold_method"] = "regime_median_year_2020"
df_thresholds_regime_robust_check["threshold_scope"] = "regime"
df_thresholds_regime_robust_check["selection_period"] = "development_2020_2024"
df_thresholds_regime_robust_check["method_role"] = "robustness_check"
df_thresholds_regime_robust_check["is_primary_method"] = False
df_thresholds_regime_robust_check["is_benchmark"] = False
df_thresholds_regime_robust_check["is_robustness_check"] = True
df_thresholds_regime_robust_check["include_in_stage04_evaluation"] = True
df_thresholds_regime_robust_check["stage04_priority"] = 3
df_thresholds_regime_robust_check["stage04_threshold_type"] = STAGE04_INITIAL_TH
df_thresholds_regime_robust_check["stage04_threshold_col"] = STAGE04_INITIAL_THR
df_thresholds_regime_robust_check["candidate_for_threshold"] = True
df_thresholds_regime_robust_check["n_valid"] = df_thresholds_regime_robust_check
df_thresholds_regime_robust_check["notes"] = (
    "Control robusto. Thresholds por régimen calculados como mediana "
    "de los thresholds anuales entre 2020 y 2024."
)

df_thresholds_regime_robust_check = (
    df_thresholds_regime_robust_check
    .sort_values(
        [
            "horizon",
            "regime_id",
            "percentile",
        ]
    )
    .reset_index(drop=True)
)

print("Tabla robusta por régimen usando mediana anual creada.")
print(f"Filas: {df_thresholds_regime_robust_check.shape[0]:,}")
print(f"Columnas: {df_thresholds_regime_robust_check.shape[1]:,}")

display(df_thresholds_regime_robust_check)

```

Tabla robusta por régimen usando mediana anual creada.

Filas: 39

Columnas: 26

	horizon	percentile	regime_id	regime_name	threshold_up_pts	threshold_down_pts
0	30	40	0	Overnight	5.75	5.500
1	30	50	0	Overnight	7.75	7.750
2	30	60	0	Overnight	10.25	10.000
3	30	40	1	Pre-market	11.00	9.750
4	30	50	1	Pre-market	15.75	13.500
5	30	60	1	Pre-market	20.75	17.750
6	30	40	2	Opening	18.50	17.250
7	30	50	2	Opening	25.00	23.750
8	30	60	2	Opening	32.75	32.250

	horizon	percentile	regime_id	regime_name	threshold_up_pts	threshold_down_pts
9	30	40	3	Regular	11.75	9.750
10	30	50	3	Regular	15.75	13.750
11	30	60	3	Regular	20.25	18.500
12	30	40	4	Closing	13.00	12.050
13	30	50	4	Closing	19.00	18.250
14	30	60	4	Closing	23.50	23.000
15	60	40	0	Overnight	9.00	8.750
16	60	50	0	Overnight	12.00	11.750

	horizon	percentile	regime_id	regime_name	threshold_up_pts	threshold_down_pts
17	60	60	0	Overnight	15.50	15.250
18	60	40	1	Pre-market	22.75	20.500
19	60	50	1	Pre-market	30.25	28.625
20	60	60	1	Pre-market	39.75	38.000
21	60	40	2	Opening	26.25	25.000
22	60	50	2	Opening	35.00	34.750
23	60	60	2	Opening	45.25	45.500
24	60	40	3	Regular	18.50	14.500
25	60	50	3	Regular	24.25	20.250

	horizon	percentile	regime_id	regime_name	threshold_up_pts	threshold_down_pts
26	60	60	3	Regular	30.50	27.250
27	90	40	0	Overnight	12.25	11.500
28	90	50	0	Overnight	16.25	15.500
29	90	60	0	Overnight	20.75	21.000
30	90	40	1	Pre-market	31.75	28.750
31	90	50	1	Pre-market	43.00	39.750
32	90	60	1	Pre-market	58.25	53.250
33	90	40	2	Opening	31.25	29.000

	horizon	percentile	regime_id	regime_name	threshold_up_pts	threshold_down_pts
34	90	50	2	Opening	42.00	39.500
35	90	60	2	Opening	53.85	51.000
36	90	40	3	Regular	23.25	18.000
37	90	50	3	Regular	30.50	25.000
38	90	60	3	Regular	38.50	33.250

39 rows × 26 columns

7.1.4. Consolidación de candidatos finales

In [101...

```
# 7.1.4. Consolidación de candidatos finales
# =====

final_candidate_cols = [
    "threshold_method",
    "method_role",
    "threshold_scope",
    "selection_period",
    "horizon",
    "percentile",
    "regime_id",
    "regime_name",
    "threshold_up_pts",
    "threshold_down_pts",
    "threshold_common_pts",
    "n_valid",
    "n_years",
```

```

    "min_year",
    "max_year",
    "median_n_valid",
    "total_n_valid",
    "candidate_for_threshold",
    "stage04_threshold_type",
    "stage04_threshold_col",
    "is_primary_method",
    "is_benchmark",
    "is_robustness_check",
    "include_in_stage04_evaluation",
    "stage04_priority",
    "notes",
]

def standardize_final_threshold_table(df, final_cols):
    """
    Asegura que todas las tablas tengan las mismas columnas
    antes de consolidarlas.
    """

    out = df.copy()

    for col in final_cols:
        if col not in out.columns:
            out[col] = np.nan

    return out[final_cols].copy()

df_thresholds_final_candidates = pd.concat(
    [
        standardize_final_threshold_table(
            df_thresholds_regime_primary,
            final_candidate_cols
        ),
        standardize_final_threshold_table(
            df_thresholds_global_benchmark,
            final_candidate_cols
        ),
        standardize_final_threshold_table(
            df_thresholds_regime_robust_check,
            final_candidate_cols
        ),
    ],
    ignore_index=True
)

# -----
# Crear ID único de threshold
# -----

def build_threshold_candidate_id(row):
    """
    Construye un identificador único para cada threshold candidato.
    """

    method = row["threshold_method"]
    h = int(row["horizon"])
    p = int(row["percentile"])

```

```

    if row["threshold_scope"] == "global":
        return f"{method}_H{h}_p{p}"

    regime_id = int(row["regime_id"])
    return f"{method}_H{h}_p{p}_regime{regime_id}"

df_thresholds_final_candidates["threshold_candidate_id"] = (
    df_thresholds_final_candidates
    .apply(build_threshold_candidate_id, axis=1)
)

# -----
# Ordenar tabla final
# -----

df_thresholds_final_candidates = (
    df_thresholds_final_candidates
    .sort_values(
        [
            "stage04_priority",
            "horizon",
            "regime_id",
            "percentile",
            "threshold_method",
        ],
        na_position="last"
    )
    .reset_index(drop=True)
)

print("Tabla final de thresholds candidatos creada.")
print("Esta tabla consolida metodología principal, benchmark y control robusto.")
print(f"Filas: {df_thresholds_final_candidates.shape[0]:,}")
print(f"Columnas: {df_thresholds_final_candidates.shape[1]:,}")

display(df_thresholds_final_candidates)

```

Tabla final de thresholds candidatos creada.

Esta tabla consolida metodología principal, benchmark y control robusto.

Filas: 87

Columnas: 27

	threshold_method	method_role	threshold_scope	selection_peric
0	regime_pooled_2020_2024	primary	regime	development_2020_2024
1	regime_pooled_2020_2024	primary	regime	development_2020_2024
2	regime_pooled_2020_2024	primary	regime	development_2020_2024
3	regime_pooled_2020_2024	primary	regime	development_2020_2024
4	regime_pooled_2020_2024	primary	regime	development_2020_2024
...	...	...	...	...
82	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
83	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
84	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
85	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024
86	regime_median_year_2020_2024	robustness_check	regime	development_2020_2024

threshold_method	method_role	threshold_scope	selection_peric
------------------	-------------	-----------------	-----------------

87 rows × 27 columns

7.2. Validación final de consistencia

```
In [102... # 7.2. Validación final de consistencia
# =====
#
# Objetivo:
# Verificar que las tablas finales no tengan problemas básicos
# antes de guardarlas:
#
# - IDs duplicados
# - thresholds faltantes
# - thresholds no positivos
# - combinaciones faltantes
# =====

# -----
# Validaciones principales
# -----

if df_thresholds_final_candidates.empty:
    raise ValueError("df_thresholds_final_candidates está vacío.")

duplicated_ids = (
    df_thresholds_final_candidates["threshold_candidate_id"]
    .duplicated()
    .sum()
)

missing_common = (
    df_thresholds_final_candidates["threshold_common_pts"]
    .isna()
    .sum()
)

non_positive_common = (
    df_thresholds_final_candidates["threshold_common_pts"] <= 0
).sum()

missing_up = (
    df_thresholds_final_candidates["threshold_up_pts"]
    .isna()
    .sum()
)

missing_down = (
    df_thresholds_final_candidates["threshold_down_pts"]
    .isna()
    .sum()
)
```

```

# -----
# Resumen por metodología
# -----

df_thresholds_final_validation_summary = (
    df_thresholds_final_candidates
    .groupby(
        [
            "threshold_method",
            "method_role",
            "threshold_scope",
        ],
        observed=True
    )
    .agg(
        rows=("threshold_candidate_id", "size"),
        unique_ids=("threshold_candidate_id", "nunique"),
        horizons=("horizon", "nunique"),
        percentiles=("percentile", "nunique"),
        regimes=("regime_id", "nunique"),
        min_threshold_common_pts=("threshold_common_pts", "min"),
        max_threshold_common_pts=("threshold_common_pts", "max"),
        missing_threshold_common_pts=("threshold_common_pts", lambda x: x.isna()
    )
    .reset_index()
    .sort_values("method_role")
    .reset_index(drop=True)
)

print("Validación final de consistencia completada.")
print(f"IDs duplicados: {duplicated_ids}")
print(f"Threshold common faltantes: {missing_common}")
print(f"Threshold common no positivos: {non_positive_common}")
print(f"Threshold up faltantes: {missing_up}")
print(f"Threshold down faltantes: {missing_down}")

display(df_thresholds_final_validation_summary)

# -----
# Reglas de corte
# -----

if duplicated_ids > 0:
    raise ValueError("Existen threshold_candidate_id duplicados.")

if missing_common > 0:
    raise ValueError("Existen thresholds common faltantes.")

if non_positive_common > 0:
    raise ValueError("Existen thresholds common no positivos.")

print("Las tablas finales están listas para guardarse.")

```

Validación final de consistencia completada.

IDs duplicados: 0

Threshold common faltantes: 0

Threshold common no positivos: 0

Threshold up faltantes: 0

Threshold down faltantes: 0

	threshold_method	method_role	threshold_scope	rows	unique_ids	hoi
0	global_pooled_2020_2024	benchmark	global	9	9	
1	regime_pooled_2020_2024	primary	regime	39	39	
2	regime_median_year_2020_2024	robustness_check	regime	39	39	

Las tablas finales están listas para guardarse.

7.3. Guardado de tablas finales

In [103...

```
# 7.3. Guardado de tablas finales
# =====
#
# Objetivo:
# Guardar las tablas finales de thresholds para utilizarlas
# en el Stage 04.
# =====

# -----
# Definir carpeta de salida
# -----

if "MNQ_THRESHOLDS_PATH" not in globals():
    if "PROJECT_ROOT" in globals():
        MNQ_THRESHOLDS_PATH = PROJECT_ROOT / "data" / "03_mnq_thresholds"
    else:
        MNQ_THRESHOLDS_PATH = Path("data") / "03_mnq_thresholds"

MNQ_THRESHOLDS_PATH.mkdir(parents=True, exist_ok=True)

# -----
# Definir rutas
# -----

MNQ_THRESHOLDS_GLOBAL_BENCHMARK_PARQUET = (
    MNQ_THRESHOLDS_PATH / "mnq_thresholds_global_benchmark.parquet"
)

MNQ_THRESHOLDS_REGIME_PRIMARY_PARQUET = (
    MNQ_THRESHOLDS_PATH / "mnq_thresholds_regime_primary.parquet"
)

MNQ_THRESHOLDS_REGIME_ROBUST_CHECK_PARQUET = (
    MNQ_THRESHOLDS_PATH / "mnq_thresholds_regime_robust_check.parquet"
)

MNQ_THRESHOLDS_FINAL_CANDIDATES_PARQUET = (
    MNQ_THRESHOLDS_PATH / "mnq_thresholds_final_candidates.parquet"
)

MNQ_THRESHOLDS_FINAL_CANDIDATES_CSV = (
    MNQ_THRESHOLDS_PATH / "mnq_thresholds_final_candidates.csv"
)

MNQ_THRESHOLDS_FINAL_SUMMARY_JSON = (
    MNQ_THRESHOLDS_PATH / "mnq_thresholds_final_summary.json"
```

```

)

# -----
# Guardar tablas
# -----

df_thresholds_global_benchmark.to_parquet(
    MNQ_THRESHOLDS_GLOBAL_BENCHMARK_PARQUET,
    index=False
)

df_thresholds_regime_primary.to_parquet(
    MNQ_THRESHOLDS_REGIME_PRIMARY_PARQUET,
    index=False
)

df_thresholds_regime_robust_check.to_parquet(
    MNQ_THRESHOLDS_REGIME_ROBUST_CHECK_PARQUET,
    index=False
)

df_thresholds_final_candidates.to_parquet(
    MNQ_THRESHOLDS_FINAL_CANDIDATES_PARQUET,
    index=False
)

df_thresholds_final_candidates.to_csv(
    MNQ_THRESHOLDS_FINAL_CANDIDATES_CSV,
    index=False
)

# -----
# Resumen metodológico en JSON
# -----

thresholds_final_summary = {
    "stage": "S03_threshold_calibration",
    "section": "7_thresholds_selected_save",
    "description": (
        "Guardado de thresholds candidatos para construir targets operativos "
        "en el Stage 04."
    ),
    "selection_period": "development_2020_2024",
    "final_test_period": "final_test_2025_2026",
    "horizons": THRESHOLD_HORIZONS,
    "percentiles": THRESHOLD_FINAL_PERCENTILES,
    "initial_threshold_type_for_stage04": STAGE04_INITIAL_THRESHOLD_TYPE,
    "initial_threshold_column_for_stage04": STAGE04_INITIAL_THRESHOLD_COL,
    "methods": {
        "primary": {
            "threshold_method": "regime_pooled_2020_2024",
            "description": (
                "Thresholds por horizonte, percentil y régimen intradiario, "
                "calculados usando todas las observaciones válidas de 2020-2024."
            ),
        },
        "benchmark": {
            "threshold_method": "global_pooled_2020_2024",
            "description": (
                "Thresholds globales por horizonte y percentil. "

```

```

        "Se guardan como benchmark de comparación."
    ),
},
"robustness_check": {
    "threshold_method": "regime_median_year_2020_2024",
    "description": (
        "Thresholds por régimen calculados como mediana de thresholds an
        "entre 2020 y 2024."
    ),
},
},
"not_used_as_primary": [
    "regime_year",
    "regime_year_quarter",
    "regime_contract",
    "regime_contract_family",
],
"saved_files": {
    "global_benchmark": str(MNQ_THRESHOLDS_GLOBAL_BENCHMARK_PARQUET),
    "regime_primary": str(MNQ_THRESHOLDS_REGIME_PRIMARY_PARQUET),
    "regime_robust_check": str(MNQ_THRESHOLDS_REGIME_ROBUST_CHECK_PARQUET),
    "final_candidates_parquet": str(MNQ_THRESHOLDS_FINAL_CANDIDATES_PARQUET),
    "final_candidates_csv": str(MNQ_THRESHOLDS_FINAL_CANDIDATES_CSV),
    "summary_json": str(MNQ_THRESHOLDS_FINAL_SUMMARY_JSON),
},
}

with MNQ_THRESHOLDS_FINAL_SUMMARY_JSON.open("w", encoding="utf-8") as f:
    json.dump(
        thresholds_final_summary,
        f,
        indent=2,
        ensure_ascii=False
    )

# -----
# Confirmación de guardado
# -----

print("Tablas finales de thresholds guardadas correctamente.")

print("\nBenchmark global:")
print(MNQ_THRESHOLDS_GLOBAL_BENCHMARK_PARQUET)

print("\nMetodología principal por régimen:")
print(MNQ_THRESHOLDS_REGIME_PRIMARY_PARQUET)

print("\nControl robusto por mediana anual:")
print(MNQ_THRESHOLDS_REGIME_ROBUST_CHECK_PARQUET)

print("\nTabla consolidada final:")
print(MNQ_THRESHOLDS_FINAL_CANDIDATES_PARQUET)

print("\nTabla consolidada final en CSV:")
print(MNQ_THRESHOLDS_FINAL_CANDIDATES_CSV)

print("\nResumen metodológico:")
print(MNQ_THRESHOLDS_FINAL_SUMMARY_JSON)

```

Tablas finales de thresholds guardadas correctamente.

Benchmark global:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_thresholds_global_benchmark.parquet

Metodología principal por régimen:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_thresholds_regime_primary.parquet

Control robusto por mediana anual:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_thresholds_regime_robust_check.parquet

Tabla consolidada final:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_thresholds_final_candidates.parquet

Tabla consolidada final en CSV:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_thresholds_final_candidates.csv

Resumen metodológico:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\03_mnq_thresholds\mnq_thresholds_final_summary.json